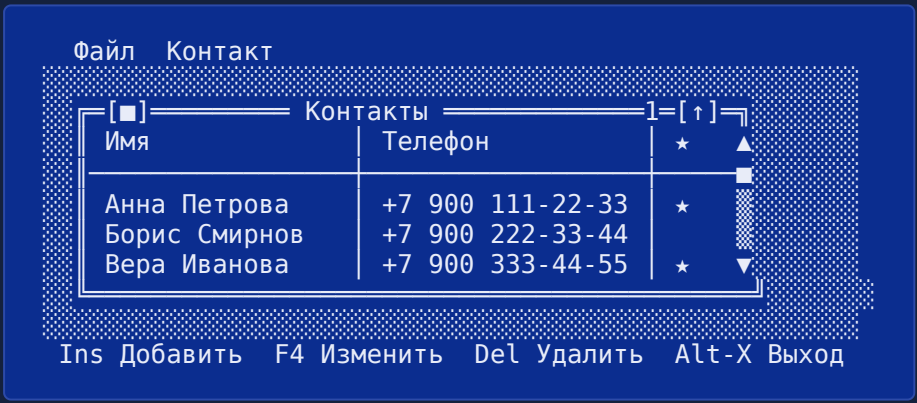
БИБЛИОТЕКА ТЕКСТОВОГО ИНТЕРФЕЙСА ДЛЯ RUST

tvision-rs

Руководство пользователя



Уроки для начинающих · справочник по каждому классу · данные в JSON и SQLite

08.11.2026 · ревизия 92c095e · github.com/cat386/tvision-rs

Содержание

Введение 5

ЧАСТЬ 1. УРОКИ

Урок 1. Первая программа 8
- Что такое текстовый интерфейс 8
- Что нужно 8
- Шаг 1. Создайте проект 8
- Шаг 2. Напишите программу 8
- Шаг 3. Запустите 9
- Что здесь происходит 9
- Словарик 10
- Попробуйте 10
- Если что-то не так 10

Урок 2. Меню и команды 11
- Как устроены команды 11
- Программа 11
- Разбор 12
- Попробуйте 14
- Важно запомнить 14

Урок 3. Окна 15
- Программа 15
- Разбор 16
- Попробуйте 17
- Важно запомнить 18

Урок 4. Координаты 19
- Клетки, а не пиксели 19
- Прямоугольник 19
- Чьи это координаты 19
- Рамка окна 20
- Попробуйте мышью 20
- Растягивание 22
- Попробуйте 23
- Важно запомнить 23

Урок 5. Первый диалог 24
- Программа 24
- Разбор 25
- Попробуйте 26
- Важно запомнить 26

Урок 6. Анкета: флажки, переключатели, список 28
- Программа 28
- Новые элементы 30
- Обмен данными 31
- Попробуйте 32
- Важно запомнить 32

Урок 7. Своё представление 33
- Программа 33

Из чего состоит представление 34
Как представление получает события 36
Попробуйте 36
Важно запомнить 36

Урок 8. Своё окно с данными 38
Программа 38
Разбор 40
Попробуйте 42
Важно запомнить 42

Урок 9. Собираем всё вместе: «Контакты» 43
Карта программы 43
Таблица 43
Enter в таблице 44
Проверка ввода в диалоге 44
Запрет недоступных команд 45
Файлы 45
Вопрос при выходе 45
Попробуйте 45

Урок 10. Тесты 47
«Невидимый» терминал 47
Запустить и проверить 47
Пример целиком 47
Окна, которые никто не закрыл 48
Что ещё удобно 48
Попробуйте 48
Что дальше 48

Урок 11. MVC: данные в JSON и SQLite 49
Что такое MVC 49
Шаг 1. Опишите поля 49
Шаг 2. Откройте модель 50
Шаг 3. Окно и контроллер 51
Где лежат данные 52
Свои команды над моделью 52
Несколько окон 53
Попробуйте 53
Важно запомнить 53

Урок 12. Рецепты: данные 54
Рецепт 1. Своя команда меняет запись 54
Рецепт 2. Команда над всеми записями 54
Рецепт 3. Сводка в заголовке окна 55
Рецепт 4. Своё окно над моделью 55
Рецепт 5. Правка без контроллера 56
Рецепт 6. Поиск по записям 56
Рецепт 7. Своё хранилище 57
Рецепт 8. Запросы SQL без MVC 58
Рецепт 9. Окно из строки ввода и таблицы 58
Рецепт 10. Настройки в JSON 59
Попробуйте 60

Урок 13. Раскладки: без координат 61
Четыре вида раскладок 61
Первая раскладка: диалог входа 62

Откуда берутся размеры 62
Пропорции: окно «Почта» 63
Сетка: калькулятор 64
Промежутки и поля 64
Скрыть и показать 64
Раскладка и grow_mode вместе 65
Попробуйте 65
Важно запомнить 65

ЧАСТЬ 2. СПРАВОЧНИК

Справочник: точки, прямоугольники, цвета, текст 67
Справочник: события, клавиши, команды 73
Справочник: представления 78
Справочник: программа, рабочий стол, окна, диалоги 90
Справочник: меню и строка состояния 96
Справочник: проверка ввода 100
Справочник: элементы управления 104
Справочник: таблица, дерево, прокрутка 111
Справочник: текстовый редактор 119
Справочник: готовые диалоги — файлы, цвета, справка 123
Справочник: файловые панели и операции с файлами 128
Справочник: терминал, буфер обмена, время, язык 133
Справочник: данные — JSON, SQLite, MVC 138
Справочник: раскладки 146

ПРИЛОЖЕНИЕ

Указатель классов 151

Введение

`tvision-rs` — библиотека для программ с **текстовым интерфейсом**: окнами, меню, диалогами и кнопками прямо в терминале, как в Norton Commander или Turbo Pascal. Это перевод на Rust знаменитой библиотеки Turbo Vision.



Для кого это руководство

Для тех, кто **немного знает Rust** (умеет создать проект через `cargo new`, понимает структуры, `impl` и `match`), но **никогда не работал** с Turbo Vision и текстовыми интерфейсами. Всё остальное объясняется по ходу дела.

Если вы знаете Turbo Vision для C++ или Pascal, почти все классы называются так же, только без буквы `T`: `TButton` → `Button`, `TDialog` → `Dialog`. Отличия собраны в главе «Перенос программ».

Часть 1. Уроки

Уроки идут по порядку, каждый — на 15–30 минут. В каждом — готовая маленькая программа: её можно запустить одной командой (`cargo run --example …`) и разобрать строку за строкой. В конце урока — задания «Попробуйте» и короткий список «Важно запомнить».

Урок	Чему научитесь	Программа
1. Первая программа	создать проект, запустить пустую программу, словарик терминов	`learn_01_hello`
2. Меню и команды	меню, строка состояния, свои команды, окно сообщения	`learn_02_menu`
3. Окна	открывать, двигать, распахивать и закрывать окна	`learn_03_windows`
4. Координаты	где что находится; раскладка без ошибок «на одну клетку»	`learn_04_coords`
5. Первый диалог	спросить у пользователя имя и прочитать ответ	`learn_05_dialog`
6. Анкета	флажки, переключатели, список, проверка ввода, обмен данными	`learn_06_form`
7. Своё представление	нарисовать свой элемент и научить его отвечать на клавиши и мышь	`learn_07_view`
8. Своё окно с данными	окно, которое хранит данные и отвечает на команды	`learn_08_todo`
9. «Контакты»	всё вместе: таблица, файлы, запрет команд, вопрос при выходе	`guide_contacts`
10. Тесты	проверять интерфейс через `cargo test`	—
11. MVC: данные в JSON и SQLite	программа из описания полей: модель, окно, контроллер; файл JSON или база SQLite	`mvc_contacts`
12. Рецепты: данные	свои команды над моделью, своё окно, поиск, своё хранилище (CSV), консоль SQL, настройки в JSON	`mvc_todo`, `mvc_books`, `mvc_csv`, `sqlite_query`, `json_settings`

Урок	Чему научитесь	Программа
13. Раскладки: без координат	`HBox`, `VBox`, `Grid`, `Form`, пружины — как в Qt: элементы расставляются и растягиваются сами	`layout_demo`

Как запустить программу урока: скачайте репозиторий библиотеки и выполните в его каталоге, например, `cargo run --example learn_02_menu`.

Часть 2. Справочник

Когда уроки пройдены — здесь можно найти любой класс. Для каждого: **что это**, **когда нужен**, **как создать**, **что важно знать**, главные методы и пример. Редко нужные методы перечислены отдельно, чтобы не мешать.

Страница	Что там
Точки, прямоугольники, цвета, текст	`Point`, `Rect`, цвета, палитры, ширина текста
События, клавиши, команды	`Event`, `Key`, `KeyEvent`, `MouseEvent`, `cm`, `kb`; куда попадает событие
Представления	`View`, `ViewState`, `Ctx`, `DrawCtx`, `Group`, флаги, **координаты подробно**
Программа, окна, диалоги	`Program`, `AppHandler`, `Desktop`, `Window`, `Dialog`
Меню и строка состояния	`Menu`, `MenuBar`, контекстное меню, `StatusLine`
Проверка ввода	маски, диапазоны, фильтры, своя проверка
Элементы управления	кнопки, поля, флажки, списки, история, выпадающий список, полоса выполнения, окна сообщений
Таблица, дерево, прокрутка	`Table`, `TreeView`, `Outline`, `Scroller`, `Terminal`
Текстовый редактор	`EditWindow`, `Editor`, `Memo`
Готовые диалоги	выбор файла, каталога, цветов; справка по F1
Файловые панели	панели как в Norton Commander, копирование в фоне
Терминал, буфер обмена, время, язык	тестовый терминал, буфер обмена, русский язык надписей
Данные: JSON, SQLite, MVC	разбор JSON, запросы к SQLite, `Model`, `ModelWindow`, `CrudController`
Раскладки	`HBox`, `VBox`, `Grid`, `Form`, `Spacer`, политики размера, `set_layout`, `adjust_size`

Все примеры кода в справочнике проверены: они компилируются, а проверки `assert!` в них выполняются.

Что ещё почитать

- **Примеры** в каталоге `examples/`: `demo` (все элементы диалогов), `tvdemo` (демонстрационная программа Turbo Vision), `tvedit` (текстовый редактор), `table`, `treeview`, `json_table`, `mvc_contacts` (записи в JSON или SQLite), `mvc_todo`, `mvc_books`, `mvc_csv`, `sqlite_query`, `json_settings` (рецепты урока 12), `layout_demo` (раскладки), `lfm` (файловый менеджер).
- **Тематические главы** — как библиотека устроена внутри: архитектура, события, цвета и палитры, перенос программ.

ЧАСТЬ 1

Уроки

По порядку, от пустой программы до приложения с данными. К каждому уроку — готовая программа `cargo run --example …`.

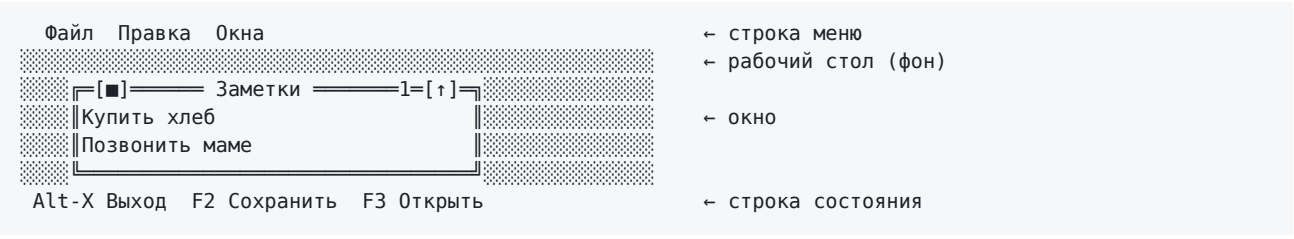
Урок 1. Первая программа

В этом уроке вы создадите проект, подключите библиотеку и запустите программу, которая занимает весь терминал и закрывается по Alt+X. Это займёт минут десять.

Что такое текстовый интерфейс

Обычные консольные программы печатают строки одну за другой: вывод «уезжает» вверх, как лента. Программа с **текстовым интерфейсом** ведёт себя иначе: она занимает **весь экран терминала** и рисует на нём окна, меню и кнопки — из символов. Мышь и клавиатура при этом работают, как в обычной графической программе: можно щёлкать, таскать окна, выбирать пункты меню.

Так выглядели программы для DOS: Norton Commander, Turbo Pascal, Far. Многие из них были написаны на библиотеке **Turbo Vision** фирмы Borland. `tvision-rs` — её перевод на Rust.



Что нужно

- Rust (`cargo` и `rustc`) не очень старой версии — 1.70 или новее. Проверить: `cargo --version`; обновить: `rustup update`.
- Любой терминал: на Linux и macOS — стандартный, на Windows — Windows Terminal или обычная консоль Windows 10 и новее.

Других библиотек и программ не нужно — `tvision-rs` ни от чего не зависит.

Шаг 1. Создайте проект

```
cargo new hello-tv
cd hello-tv
```

Откройте файл `Cargo.toml` и допишите в раздел `[dependencies]` строку:

```
[dependencies]
tvision-rs = { git = "https://github.com/cat386/tvision-rs" }
```

> Пакет называется `tvision-rs`, а в коде библиотека называется короче — `tvision`. Поэтому ниже `use tvision::*`.

Шаг 2. Напишите программу

Замените содержимое `src/main.rs` этим кодом:

```rust
//! Урок 1. Самая простая программа: пустой рабочий стол и строка состояния.
//!
//! Запуск: `cargo run --example learn_01_hello`. Выход — Alt+X или щелчок
//! по надписи «Alt-X Выход» внизу экрана.

// Подключаем всё нужное из библиотеки одной строкой.
use tvision::*;

fn main() -> std::io::Result<()> {
    // Строка состояния — нижняя строка экрана. В ней один пункт:
    // надпись «Alt-X Выход», клавиша Alt+X и команда «выйти из программы».
    let status = vec![StatusDef::all(vec![StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT)])];

    // Создаём программу: она займёт весь терминал.
    // Первый аргумент — меню (его пока нет), второй — строка состояния.
    let mut app = Program::with_terminal(None, Some(status))?;

    // Запускаем. `run` возвращается, когда пользователь выходит из программы;
    // терминал при этом возвращается в обычный вид.
    app.run(&mut program::NoHandler);
    Ok(())
}
```

Шаг 3. Запустите

```
cargo run
```

Первая сборка займёт немного времени. Потом терминал очистится и покажет серо-полосатый **рабочий стол** и внизу — **строку состояния** с надписью «Alt-X Выход». Нажмите Alt+X (или щёлкните по надписи мышью) — программа закроется, и терминал вернётся в обычный вид.

> Тот же пример есть в репозитории библиотеки: `cargo run --example learn_01_hello`.

Что здесь происходит

Разберём программу по строчкам.

`use tvision::*;` — подключает всё из библиотеки: типы, функции, константы. Дальше в уроках эта строка будет всегда.

Строка состояния.

```rust
let status = vec![StatusDef::all(vec![StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT)])];
```

Читается изнутри наружу:

- `StatusItem::new(надпись, клавиша, команда)` — один пункт строки состояния. Надпись `"~Alt-X~ Выход"`: часть между тильдами `~` будет выделена цветом. Клавиша `kb::ALT_X` — Alt+X. Команда `cm::QUIT` — «выйти из программы».
- `StatusDef::all(пункты)` — набор пунктов, который показывается всегда. (Можно сделать несколько наборов для разных случаев — это потом.)
- `vec![…]` — список наборов. У нас один.

Программа.

```rust
let mut app = Program::with_terminal(None, Some(status))?;
```

`Program` — это вся программа целиком: экран, рабочий стол, меню, строка состояния. `with_terminal` переводит терминал в полноэкранный режим. Первый аргумент — меню: его пока нет, поэтому `None`. Второй — строка состояния. Знак `?` в конце передаёт ошибку наружу, если терминал не удалось подготовить (например, программу запустили без терминала).

Запуск.

```
app.run(&mut program::NoHandler);
```

`run` запускает **цикл событий** — сердце любой программы с интерфейсом. Программа ждёт, пока пользователь что-нибудь сделает (нажмёт клавишу, щёлкнет мышью), обрабатывает это и перерисовывает экран — и так снова и снова, пока не придёт команда выхода. Тогда `run` возвращается.

`program::NoHandler` — «ничего не делающий» обработчик. В следующем уроке мы напишем свой и научим программу реагировать на команды.

Словарик

Эти слова будут встречаться во всех уроках.

Слово	Что значит
представление (view)	всё, что видно на экране: окно, кнопка, строка меню, рабочий стол
событие (event)	что-то произошло: нажата клавиша, щёлкнула мышь, выбрана команда
команда (command)	число, означающее действие: «выйти», «открыть файл», «OK». Пункты меню и кнопки посылают команды
рабочий стол (desktop)	область между строкой меню и строкой состояния, на которой лежат окна
строка состояния (status line)	нижняя строка с подсказками клавиш
модальное окно	окно, которое нужно закрыть, прежде чем вернуться к остальной программе (диалог «Сохранить?»)

Попробуйте

1. Добавьте второй пункт в строку состояния: `StatusItem::new("~F10~ Меню", kb::F10, cm::MENU)`. Пока меню нет, он ничего не делает — но надпись появится.
2. Поменяйте надпись на `"Выход ~Alt+X~"` и посмотрите, какая часть выделяется.

Если что-то не так

- **Программа сразу завершается с ошибкой «стандартный ввод не является терминалом»** — её запустили не в терминале (например, из окна вывода редактора кода). Запустите `cargo run` в настоящем терминале.
- **После аварийного завершения терминал «сломался»** (не видно набираемых символов) — наберите вслепую `reset` и Enter.
- **Русские буквы — «кракозябры»** — в терминале не UTF-8. В Linux и macOS это почти не встречается; в старой консоли Windows выполните `chcp 65001`.

Следующий урок — меню и команды.

Урок 2. Меню и команды

Теперь у программы появятся меню наверху и свой код, который отвечает на выбор пунктов. По F2 программа будет здороваться.

Как устроены команды

Когда пользователь выбирает пункт меню, нажимает кнопку или клавишу из строки состояния, программа получает **команду** — просто число. Одни команды обрабатывает сама библиотека (`cm::QUIT` — выход, `cm::CLOSE` — закрыть окно), другие придумываете вы.

Свои команды не обработает никто, кроме вашего кода. Этот код живёт в **обработчике приложения** — структуре, для которой вы реализуете трейт `AppHandler`. Программа передаёт ему все команды, которые не забрали окна и кнопки.

```
пункт меню «Привет» ──► команда CM_HELLO ──► окна? нет ──► ваш AppHandler
```

Программа

Файл `examples/learn_02_menu.rs`:

```rust
//! Урок 2. Меню, строка состояния и свои команды.
//!
//! Запуск: `cargo run --example learn_02_menu`. F10 — меню, F2 — приветствие,
//! Alt+X — выход.

use tvision::*;

// Свои команды. Это просто числа; главное — не пересечься со стандартными
// (`cm::QUIT`, `cm::OK`…), которые заняли номера меньше 100.
const CM_HELLO: u16 = 100;
const CM_ABOUT: u16 = 101;

/// Наше приложение. Здесь живут данные программы — пока только счётчик.
struct App {
    greetings: u32,
}

// `AppHandler` — «мозг» программы: сюда приходят команды из меню и строки
// состояния, которые не обработал никто другой.
impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        match ev.command() {
            Some(CM_HELLO) => {
                self.greetings += 1;
                let text = format!("\x03Привет! Это приветствие номер {}.", self.greetings);
                message_box(app, &text, mf::INFORMATION | mf::OK_BUTTON);
            }
            Some(CM_ABOUT) => {
                message_box(app, "\x03Урок 2\n\x03Меню и команды", mf::INFORMATION | mf::OK_BUTTON);
            }
            // Чужие события не трогаем — просто выходим.
            _ => return,
        }
        // Команда обработана: помечаем событие, чтобы оно не пошло дальше.
        ev.clear();
    }
}

/// Строка меню: два выпадающих меню.
```

```rust
fn menu() -> Menu {
    Menu::new()
        .submenu(
            "~Ф~айл", // тильды выделяют горячую букву: Alt+Ф откроет это меню
            Menu::new()
                .item_key("~П~ривет", CM_HELLO, kb::F2, "F2") // пункт, клавиша F2, подпись «F2»
                .separator() // черта-разделитель
                .item_key("~В~ыход", cm::QUIT, kb::ALT_X, "Alt-X"),
        )
        .submenu("~С~правка", Menu::new().item("~О~ программе", CM_ABOUT))
}

/// Строка состояния.
fn status() -> Vec<StatusDef> {
    vec![StatusDef::all(vec![
        StatusItem::new("~F2~ Привет", kb::F2, CM_HELLO),
        StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT),
        StatusItem::new("", kb::F10, cm::MENU), // без надписи: F10 открывает меню
    ])]
}

fn main() -> std::io::Result<()> {
    // Кнопки окон сообщений («Да», «Отмена»…) — по-русски.
    locale::set_language(locale::Language::Ru);
    let mut app = Program::with_terminal(Some(menu()), Some(status()))?;
    app.run(&mut App { greetings: 0 });
    Ok(())
}
```

Запуск: `cargo run --example learn_02_menu`. Нажмите F10 или щёлкните по слову «Файл» — откроется меню. F2 — окно с приветствием, Enter или Esc — закрыть его.

Разбор

Свои команды

```rust
const CM_HELLO: u16 = 100;
const CM_ABOUT: u16 = 101;
```

Команда — число типа `u16`. Стандартные команды библиотеки (`cm::QUIT`, `cm::OK` и другие) заняли номера меньше 100, поэтому свои начинаем со 100. Имена `CM_…` — традиция из Turbo Vision («command»).

Обработчик приложения

```rust
struct App {
    greetings: u32,
}

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        match ev.command() {
            Some(CM_HELLO) => { … }
            Some(CM_ABOUT) => { … }
            _ => return,
        }
        ev.clear();
    }
}
```

- `App` — наша структура. В ней можно хранить любые данные программы; здесь — сколько раз мы поздоровались.

- `handle_event` вызывается для каждого необработанного события. Два параметра: `app` — сама программа (через неё показывают окна и диалоги), `ev` — событие.
- `ev.command()` возвращает номер команды, если событие — команда, или `None`, если это что-то другое.
- **`ev.clear()`** — «я это обработал». Событие становится пустым и дальше не идёт. Если забыть `clear`, ничего страшного не случится, но привыкайте ставить его всегда — в окнах и диалогах это важно.
- `_ => return` — чужие события пропускаем, не трогая.

Окно сообщения

```
message_box(app, &text, mf::INFORMATION | mf::OK_BUTTON);
```

`message_box` показывает небольшое окно с текстом и кнопками и ждёт, пока его закроют. Флаги `mf::…` выбирают заголовок (`INFORMATION` — «Информация», `WARNING`, `ERROR`, `CONFIRMATION`) и кнопки (`OK_BUTTON`, `YES_BUTTON`, `NO_BUTTON`, `CANCEL_BUTTON`). Флаги складываются знаком `|`. Функция возвращает команду нажатой кнопки — например, `cm::YES`.

Символ `\x03` в начале строки — «расположить текст по центру». `\n` — перенос строки.

Меню

```
Menu::new()
    .submenu("~Ф~айл", Menu::new()
        .item_key("~П~ривет", CM_HELLO, kb::F2, "F2")
        .separator()
        .item_key("~В~ыход", cm::QUIT, kb::ALT_X, "Alt-X"))
    .submenu("~С~правка", Menu::new().item("~О~ программе", CM_ABOUT))
```

Меню собирается **цепочкой вызовов**: каждый метод добавляет что-то и возвращает меню дальше.

- `submenu(название, меню)` — выпадающее меню в строке меню. Тильды выделяют **горячую букву**: Alt+Ф откроет «Файл». В открытом меню пункт выбирается просто буквой.
- `item(название, команда)` — пункт меню.
- `item_key(название, команда, клавиша, подпись)` — пункт с **клавишей быстрого вызова**: F2 сработает, даже когда меню закрыто. Подпись `"F2"` только рисуется справа от названия.
- `separator()` — черта между пунктами.

Строка состояния

```
StatusItem::new("~F2~ Привет", kb::F2, CM_HELLO),
StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT),
StatusItem::new("", kb::F10, cm::MENU),
```

Пункт с пустой надписью **не виден**, но его клавиша работает. Так F10 открывает меню (`cm::MENU`), не занимая места внизу.

Язык

```
locale::set_language(locale::Language::Ru);
```

Надписи, которые рисует сама библиотека (кнопки «ОК», «Отмена», заголовки окон сообщений), по умолчанию английские. Эта строка делает их русскими. Надписи вашей программы вы пишете сами на любом языке.

Запуск с обработчиком

```
app.run(&mut App { greetings: 0 });
```

Вместо `NoHandler` передаём свой обработчик. `&mut` — потому что обработчик меняет свои данные (счётчик).

Попробуйте

1. Добавьте в «Справку» пункт «Версия» с командой `CM_VERSION = 102`, который показывает номер версии.
2. Сделайте пункт «Выход» с вопросом: `message_box(…, mf::CONFIRMATION | mf::YES_BUTTON | mf::NO_BUTTON)` и, если ответ `cm::YES`, — `app.quit()`. (Пункт должен посылать свою команду, а не `cm::QUIT`.)
3. Нажмите F2 несколько раз подряд — счётчик растёт.

Важно запомнить

- Свои команды — константы `u16` от 100.
- Команды приходят в `AppHandler::handle_event`; обработали — `ev.clear()`.
- Не назначайте пункту строки состояния Enter, Esc, Tab или буквы: строка состояния видит клавиши раньше всех, и они перестанут работать в диалогах.

Следующий урок — окна.

Урок 3. Окна

Окно — главный строительный блок интерфейса. В этом уроке программа по F3 открывает новые окна с текстом; их можно двигать мышью, растягивать, распахивать на весь экран и закрывать.

Программа

Файл `examples/learn_03_windows.rs`:

```
//! Урок 3. Окна: создать, передвинуть, распахнуть, закрыть.
//!
//! Запуск: `cargo run --example learn_03_windows`. F3 — новое окно,
//! F6 — следующее окно, F5 — распахнуть, Alt+F3 — закрыть, Alt+X — выход.
//! Окна можно таскать мышью за заголовок и тянуть за правый нижний угол.

use tvision::*;

const CM_NEW_WINDOW: u16 = 100;

struct App {
    /// Сколько окон уже открыто за всё время — для заголовков и номеров.
    opened: i32,
}

impl App {
    fn new_window(&mut self, app: &mut Program) {
        self.opened += 1;
        let n = self.opened;

        // Каждое следующее окно — чуть правее и ниже предыдущего.
        let shift = (n - 1) % 8;
        let bounds = Rect::new(2 + shift * 3, 1 + shift, 2 + shift * 3 + 40, 1 + shift + 10);

        // Номер 1–9 виден в рамке; Alt+номер выбирает окно. 0 — без номера.
        let number = if n < 10 { n as i16 } else { 0 };
        let mut w = Window::new(bounds, &format!("Окно {}", n), number);

        // Внутрь окна кладём текст. `interior()` — место внутри рамки.
        let text =
            StaticText::new(w.interior(), "Тащите окно мышью за заголовок, тяните за правый нижний
угол.\n\nF5 — распахнуть, Alt+F3 — закрыть.");
        w.add(text);

        // Окно — на рабочий стол.
        app.insert_window(w);
    }
}

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        if ev.command() == Some(CM_NEW_WINDOW) {
            self.new_window(app);
            ev.clear();
        }
    }
}

fn menu() -> Menu {
    Menu::new()
        .submenu(
            "~Ф~айл",
            Menu::new().item_key("~Н~овое окно", CM_NEW_WINDOW, kb::F3,
"F3").separator().item_key("~В~ыход", cm::QUIT, kb::ALT_X, "Alt-X"),
        )
```

```
        // Команды окон — стандартные: их обрабатывают сами окна и рабочий стол.
        .submenu(
            "~О~кна",
            Menu::new()
                .item("~М~озаика", cm::TILE)
                .item("~К~аскад", cm::CASCADE)
                .separator()
                .item_key("~С~ледующее", cm::NEXT, kb::F6, "F6")
                .item_key("~Р~аспахнуть", cm::ZOOM, kb::F5, "F5")
                .item_key("~З~акрыть", cm::CLOSE, kb::ALT_F3, "Alt-F3"),
        )
}

fn status() -> Vec<StatusDef> {
    vec![StatusDef::all(vec![
        StatusItem::new("~F3~ Новое", kb::F3, CM_NEW_WINDOW),
        StatusItem::new("~F6~ Следующее", kb::F6, cm::NEXT),
        StatusItem::new("~F5~ Распахнуть", kb::F5, cm::ZOOM),
        StatusItem::new("~Alt-F3~ Закрыть", kb::ALT_F3, cm::CLOSE),
        StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT),
        StatusItem::new("", kb::F10, cm::MENU),
    ])]
}

fn main() -> std::io::Result<()> {
    let mut app = Program::with_terminal(Some(menu()), Some(status()))?;
    let mut h = App { opened: 0 };
    h.new_window(&mut app); // одно окно сразу при запуске
    app.run(&mut h);
    Ok(())
}
```

Запуск: `cargo run --example learn_03_windows`. Нажмите F3 несколько раз, потом попробуйте:

- перетащить окно мышью за верхнюю рамку;
- потянуть за правый нижний угол — окно станет больше;
- щёлкнуть по значку `[↑]` справа вверху или нажать F5 — окно распахнётся на весь рабочий стол;
- щёлкнуть по `[■]` слева вверху или нажать Alt+F3 — окно закроется;
- F6 — перейти к следующему окну, Alt+1, Alt+2… — к окну с этим номером;
- меню «Окна» → «Мозаика» — разложить окна плиткой.

Всё это умеют сами окна — нам не пришлось писать для этого ни строчки.

Разбор

Создание окна

```
let mut w = Window::new(bounds, &format!("Окно {}", n), number);
```

`Window::new` принимает три вещи:

- **границы** `bounds` — прямоугольник, который окно займёт на рабочем столе;
- **заголовок** — текст в верхней рамке;
- **номер** — цифра 1–9 в рамке справа; по Alt+цифра окно выбирается. Номер 0 — без номера.

Прямоугольник

```
let bounds = Rect::new(2 + shift * 3, 1 + shift, 2 + shift * 3 + 40, 1 + shift + 10);
```

`Rect::new(x1, y1, x2, y2)` — прямоугольник от левого верхнего угла `(x1, y1)` до правого нижнего `(x2, y2)`. Единица — одна клетка терминала: `x` — столбец, `y` — строка, счёт с нуля, `y` растёт вниз. Окно выше — шириной `40` и высотой `10` клеток.

> Правый нижний угол **не входит** в прямоугольник: ширина `Rect::new(2, 1, 42, 11)` — ровно `42 - 2 = 40`. Подробно — в уроке 4.

Что положить в окно

```
let text = StaticText::new(w.interior(), "…");
w.add(text);
```

- `StaticText` — просто текст. Длинные строки сами переносятся по словам, `\n` — новая строка.
- `w.interior()` — прямоугольник **внутри рамки**. Текст займёт всё окно, кроме рамки.
- `w.add(…)` кладёт представление в окно. Окно становится его **владельцем**: дальше оно само рисует, двигает и удаляет своих «детей».

Окно — на рабочий стол

```
app.insert_window(w);
```

Окно отдаётся программе: оно появляется поверх остальных и получает фокус (его рамка становится двойной — так видно активное окно).

Стандартные команды окон

```
.item("~М~озаика", cm::TILE)
.item_key("~С~ледующее", cm::NEXT, kb::F6, "F6")
.item_key("~Р~аспахнуть", cm::ZOOM, kb::F5, "F5")
.item_key("~З~акрыть", cm::CLOSE, kb::ALT_F3, "Alt-F3")
```

Это команды библиотеки, их не нужно обрабатывать самим: `cm::ZOOM` и `cm::CLOSE` обрабатывает активное окно, `cm::NEXT`, `cm::TILE` и `cm::CASCADE` — рабочий стол. Достаточно поставить их в меню или в строку состояния.

Окно при запуске

```
let mut h = App { opened: 0 };
h.new_window(&mut app);
app.run(&mut h);
```

До `run` можно подготовить экран: открыть окна, задать узор фона и так далее. Здесь мы вызываем свой же метод `new_window`.

Попробуйте

1. Поменяйте узор рабочего стола: до `run` добавьте `app.desktop_mut().set_pattern('▒');`.
2. Сделайте окно, которое нельзя закрыть: `w.flags &= !wf::CLOSE;` — значок `[■]` исчезнет, Alt+F3 перестанет работать.
3. Откройте окно сразу во весь рабочий стол: `let r = app.desktop().group.vs.extent();` и `Window::new(r, …)`.

Важно запомнить

- Окно = рамка + заголовок + номер + содержимое. Содержимое добавляется методом `add`.
- Место внутри рамки — `interior()`.
- Двигать, растягивать, распахивать и закрывать окна библиотека умеет сама; команды `cm::ZOOM`, `cm::CLOSE`, `cm::NEXT`, `cm::TILE` нужно только поставить в меню.

Следующий урок — координаты.

Урок 4. Координаты

Прежде чем раскладывать кнопки и поля по диалогам, нужно разобраться, как в библиотеке задаются положения. Почти все ошибки вида «кнопка уехала на клетку» или «щелчок попадает не туда» — отсюда.

Клетки, а не пиксели

Экран терминала — сетка **клеток**. В каждой клетке помещается один символ. Положение задаётся двумя числами:

- `x` — номер **столбца**, слева направо, начиная с **0**;
- `y` — номер **строки**, **сверху вниз**, начиная с **0**.

```
    x →   0 1 2 3 4 5 6 7
y   0     . . . . . . . .
↓   1     . . . . . . . .
    2     . . . . . ● . .      ● — точка (5, 2)
```

Точка — тип `Point { x, y }`: `Point::new(5, 2)`. Тот же тип используют для **размеров**: тогда `x` — ширина, `y` — высота.

Прямоугольник

`Rect::new(x1, y1, x2, y2)` — прямоугольник от левого верхнего угла `(x1, y1)` до правого нижнего `(x2, y2)`. Самое важное правило:

> **Правый и нижний край не входят в прямоугольник.**

```
Rect::new(2, 1, 7, 4)

   0 1 2 3 4 5 6 7 8
0  . . . . . . . . .
1  . . ■ ■ ■ ■ ■ . .     занимает столбцы 2, 3, 4, 5, 6
2  . . ■ ■ ■ ■ ■ . .     и строки 1, 2, 3
3  . . ■ ■ ■ ■ ■ . .
4  . . . . . . . . .     ширина: 7 − 2 = 5, высота: 4 − 1 = 3
```

Поэтому удобно думать так: **`x2 = x1 + ширина`**, **`y2 = y1 + высота`**.

```rust
// Строка ввода шириной 20 клеток на строке 2, начиная со столбца 10:
Rect::new(10, 2, 10 + 20, 2 + 1)   // = Rect::new(10, 2, 30, 3)

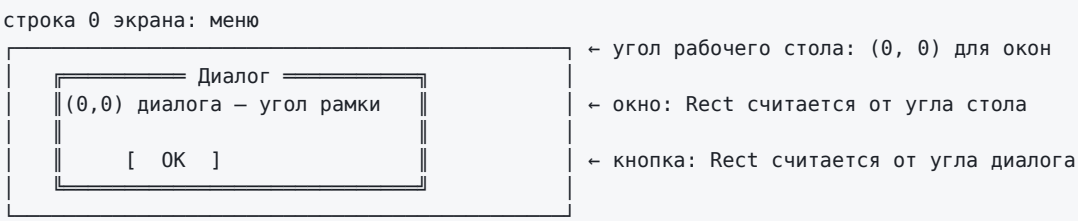
// Кнопка шириной 12 и высотой 2 (у кнопки вторая строка — тень):
Rect::new(5, 6, 5 + 12, 6 + 2)
```

Если два прямоугольника стоят вплотную, у правого `x1` равен `x2` левого — они касаются, но не наезжают друг на друга.

Чьи это координаты

Числа в `Rect` отсчитываются **не от угла экрана**, а от угла того, куда вы кладёте представление, — от его **владельца**:

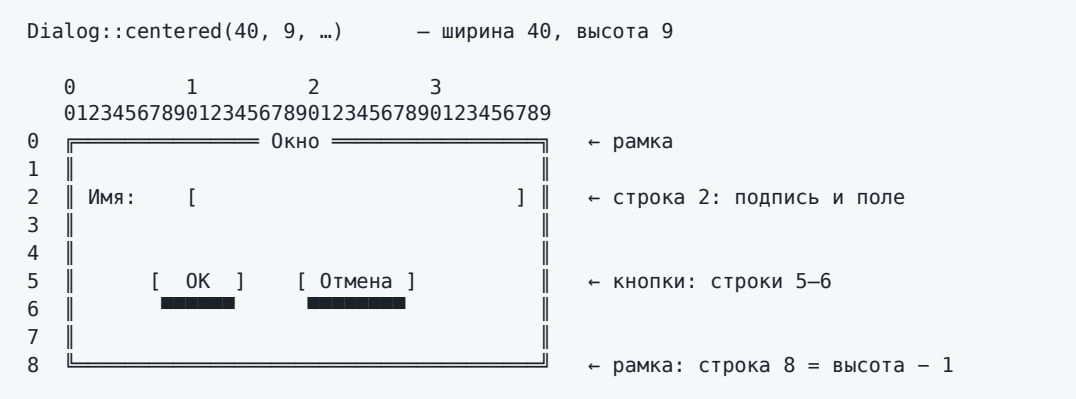
- окно кладут на рабочий стол — его `Rect` считается от угла рабочего стола (а рабочий стол начинается под строкой меню);
- кнопку кладут в диалог — её `Rect` считается от угла диалога.



Благодаря этому, когда пользователь двигает окно, координаты кнопок внутри не меняются: кнопка всегда «в 5 клетках от левого края диалога».

Рамка окна

У окна и диалога **рамка занимает крайние клетки**: строку 0, последнюю строку, столбец 0 и последний столбец. Элементы кладут **внутрь**:



- первая свободная клетка — **(1, 1)**; обычно элементы начинают со столбца 2 и строки 2, чтобы не прилипать к рамке;
- последняя свободная — **(ширина − 2, высота − 2)**;
- всё место внутри рамки сразу даёт `window.interior()`.

Попробуйте мышью

Файл `examples/learn_04_coords.rs`:

```rust
//! Урок 4. Координаты: где что находится.
//!
//! Запуск: `cargo run --example learn_04_coords`. Щёлкайте мышью внутри
//! окна — внизу окна появятся координаты щелчка. F2 — диалог, разложенный
//! по правилам из урока. Alt+X — выход.

use tvision::*;

const CM_LAYOUT: u16 = 100;

/// «Линейка»: рисует номера столбцов и строк и показывает, куда щёлкнули.
/// (Как писать свои представления, разберём в уроке 7 — здесь важны числа.)
struct Ruler {
    vs: ViewState,
    /// Последний щелчок: (свои координаты, координаты владельца-окна).
    click: Option<(Point, Point)>,
```

```
}

impl Ruler {
    fn new(bounds: Rect) -> Self {
        let mut vs = ViewState::new(bounds);
        vs.grow_mode = gf::GROW_HI_X | gf::GROW_HI_Y; // растягиваться вместе с окном
        Ruler { vs, click: None }
    }
}

impl View for Ruler {
    tvision::impl_view_basics!();

    fn draw(&self, dc: &mut DrawCtx) {
        let normal = dc.color(1);
        let w = self.vs.size.x;
        let h = self.vs.size.y;
        // Рисуем в СВОИХ координатах: (0, 0) — левый верхний угол линейки.
        dc.fill(Rect::new(0, 0, w, h), '·', normal);
        for x in 0..w {
            dc.write_char(x, 0, char::from_digit((x % 10) as u32, 10).unwrap(), normal, 1);
        }
        for y in 0..h {
            dc.write_char(0, y, char::from_digit((y % 10) as u32, 10).unwrap(), normal, 1);
        }
        let text = match self.click {
            Some((own, owner)) => format!(" свои: ({}, {})   в окне: ({}, {}) ", own.x, own.y,
owner.x, owner.y),
            None => " щёлкните мышью ".to_string(),
        };
        dc.write_str(2, h - 1, &text, normal);
    }

    fn handle_event(&mut self, ev: &mut Event, _ctx: &mut Ctx) {
        if let Event::MouseDown(m) = ev {
            // До представления позиция доходит в координатах ВЛАДЕЛЬЦА (окна).
            // `make_local` переводит её в свои: вычитает наш `origin`.
            self.click = Some((self.vs.make_local(m.pos), m.pos));
            ev.clear();
        }
    }
}

/// Диалог, разложенный по правилам: подписи слева, поля справа,
/// кнопки высотой 2 внизу по центру.
fn layout_dialog() -> Dialog {
    let (w, h) = (44, 12); // ширина и высота диалога вместе с рамкой
    let mut d = Dialog::centered(w, h, "Разметка");
    let rows = ["~И~мя:", "~Г~ород:", "~П~очта:"];
    for (i, text) in rows.iter().enumerate() {
        let y = 2 + 2 * i as i32; // строки 2, 4, 6: через одну
                                  // Поле: от столбца 12 до `w - 2` (за 2 клетки до правой рамки).
        let field = d.add(InputLine::new(Rect::new(12, y, w - 2, y + 1), 50));
        // Подпись: столбцы 2..12 на той же строке.
        d.add(Label::new(Rect::new(2, y, 12, y + 1), text, Some(field)));
    }
    // Две кнопки шириной 12 с промежутком 2 — ровно по центру.
    let bw = 12;
    let x0 = (w - (bw + 2 + bw)) / 2;
    d.add(Button::new(Rect::new(x0, h - 3, x0 + bw, h - 1), "O~K~", cm::OK, bf::DEFAULT));
    d.add(Button::new(Rect::new(x0 + bw + 2, h - 3, x0 + 2 * bw + 2, h - 1), "Отмена", cm::CANCEL,
bf::NORMAL));
    d
}

struct App;

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        if ev.command() == Some(CM_LAYOUT) {
```

```
            app.exec_dialog(layout_dialog());
            ev.clear();
        }
    }
}

fn open_window(app: &mut Program) {
    // Окно в координатах РАБОЧЕГО СТОЛА: левый верхний угол — (3, 1).
    let mut w = Window::new(Rect::new(3, 1, 63, 18), "Координаты", 1);
    // Линейка занимает всё внутри рамки.
    let inside = w.interior(); // для окна 60×17 это (1, 1)–(59, 16)
    w.add(Ruler::new(inside));
    app.insert_window(w);
}

fn status() -> Vec<StatusDef> {
    vec![StatusDef::all(vec![
        StatusItem::new("~F2~ Диалог", kb::F2, CM_LAYOUT),
        StatusItem::new("~F5~ Распахнуть", kb::F5, cm::ZOOM),
        StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT),
    ])]
}

fn main() -> std::io::Result<()> {
    let mut app = Program::with_terminal(None, Some(status()))?;
    open_window(&mut app);
    app.run(&mut App);
    Ok(())
}
```

Запуск: `cargo run --example learn_04_coords`. Щёлкайте по окну: внизу появятся координаты щелчка — **свои** (от угла линейки) и **в окне** (от угла окна). Они всегда отличаются на 1: линейка лежит в окне в точке `(1, 1)`, сразу за рамкой.

Перетащите окно в другое место и щёлкните в ту же точку линейки — числа не изменятся. Это и значит «координаты от владельца».

F2 открывает диалог, разложенный по правилам этого урока. Посмотрите в функции `layout_dialog`, как положения вычисляются **из ширины и высоты диалога**, а не подбираются числами наугад: так легко поменять размер диалога, и всё останется на местах.

Мышь: два шага

В методе `handle_event` линейки:

```
if let Event::MouseDown(m) = ev {
    self.click = Some((self.vs.make_local(m.pos), m.pos));
```

- `m.pos` — где щёлкнули, **в координатах владельца** (у нас — окна);
- `self.vs.make_local(m.pos)` — то же **в своих координатах**: от левого верхнего угла самого представления.

Для проверки «попали ли в меня» и «в какую клетку» нужны свои координаты — поэтому почти всегда первым делом пишут `make_local`.

Растягивание

Когда окно растягивают, его содержимое по умолчанию **не меняется**. Чтобы элемент растягивался вместе с окном, ему задают `grow_mode`:

```
ruler.vs.grow_mode = gf::GROW_HI_X | gf::GROW_HI_Y; // правый и нижний край следуют за окном
```

Чего хотим	`grow_mode`
растягиваться вместе с окном (список, таблица, текст)	`gf::GROW_HI_X | gf::GROW_HI_Y`
оставаться на месте (поля, подписи)	ничего не задавать
«прилипнуть» к правому нижнему углу (кнопка OK)	`gf::GROW_ALL`

Подробнее о растягивании, центрировании и экранных координатах — в справочнике.

> **Без координат.** Когда элементов много, считать прямоугольники утомительно. Можно поручить это **раскладке**, как в Qt: «эти — в строку, эти — в столбец, здесь пружина» — и библиотека сама расставит и растянет элементы. Это урок 13. Но знать координаты всё равно нужно: свои представления рисуют именно в них.

Попробуйте

1. В `layout_dialog` поменяйте `let (w, h) = (44, 12)` на `(60, 12)` — поля станут длиннее, кнопки останутся по центру.
2. Уберите строчку с `grow_mode` у линейки, распахните окно (F5) — линейка останется маленькой.

Важно запомнить

- `Rect::new(x, y, x + ширина, y + высота)` — правый и нижний край не входят.
- Координаты — от угла **владельца**: окна — от рабочего стола, элементы — от окна.
- У окна рамка — крайние клетки; внутри — `interior()`, от `(1, 1)`.
- У кнопки высота 2.
- Мышь: `self.vs.make_local(m.pos)` — в свои координаты.

Следующий урок — первый диалог.

Урок 5. Первый диалог

Диалог — окно с полями и кнопками, которое что-то спрашивает у пользователя. Сделаем самый простой: «Как вас зовут?» — и поздороваемся по имени.

Программа

Файл `examples/learn_05_dialog.rs`:

```
//! Урок 5. Первый диалог: спросить имя и поздороваться.
//!
//! Запуск: `cargo run --example learn_05_dialog`. F2 — диалог, Alt+X — выход.

use tvision::*;

const CM_ASK: u16 = 100;

struct App {
    /// Имя, которое ввели в прошлый раз (диалог предложит его снова).
    name: String,
}

/// Строим диалог «Знакомство». Возвращаем сам диалог и номер (id) строки
/// ввода — по нему потом достанем введённый текст.
fn name_dialog(old_name: &str) -> (Dialog, ViewId) {
    // Диалог 40×9, появится по центру экрана.
    let mut d = Dialog::centered(40, 9, "Знакомство");

    // Строка ввода: столбцы 10..37, строка 2; не длиннее 30 символов.
    let mut input = InputLine::new(Rect::new(10, 2, 37, 3), 30);
    input.set_text(old_name);
    let input_id = d.add(input);

    // Подпись слева. Связываем её со строкой: Alt+И переведёт туда курсор.
    d.add(Label::new(Rect::new(2, 2, 9, 3), "~И~мя:", Some(input_id)));

    // Кнопки. Высота кнопки — 2 строки (вторая — тень).
    d.add(Button::new(Rect::new(6, 5, 18, 7), "O~K~", cm::OK, bf::DEFAULT)); // DEFAULT — нажимается
по Enter
    d.add(Button::new(Rect::new(21, 5, 33, 7), "Отмена", cm::CANCEL, bf::NORMAL));

    (d, input_id)
}

impl App {
    fn ask(&mut self, app: &mut Program) {
        let (d, input_id) = name_dialog(&self.name);

        // Показать диалог и ждать, пока его закроют. Пока он открыт, остальная
        // программа «замирает» — это и называется модальным окном.
        // Возвращаются: команда, которой закрыли (OK или Cancel), и сам диалог.
        let (cmd, d) = app.exec_dialog(d);
        if cmd != cm::OK {
            return; // нажали «Отмена» или Esc
        }

        // Диалог ещё у нас в руках — достаём из него строку ввода по id.
        let name = d.win.group.find::<InputLine>(input_id).unwrap().text();
        let name = name.trim().to_string();
        if name.is_empty() {
            message_box(app, "\x03Вы не ввели имя.", mf::WARNING | mf::OK_BUTTON);
        } else {
            message_box(app, &format!("\x03Привет, {}!", name), mf::INFORMATION | mf::OK_BUTTON);
            self.name = name;
```

```
        }
    }
}

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        if ev.command() == Some(CM_ASK) {
            self.ask(app);
            ev.clear();
        }
    }
}

fn status() -> Vec<StatusDef> {
    vec![StatusDef::all(vec![StatusItem::new("~F2~ Знакомство", kb::F2, CM_ASK),
StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT)])]
}

fn main() -> std::io::Result<()> {
    locale::set_language(locale::Language::Ru);
    let mut app = Program::with_terminal(None, Some(status()))?;
    let mut h = App { name: String::new() };
    app.put_event(Event::command_event(CM_ASK)); // сразу открыть диалог
    app.run(&mut h);
    Ok(())
}
```

Запуск: `cargo run --example learn_05_dialog`. Диалог откроется сразу. Введите имя и нажмите Enter (или щёлкните «OK»). Esc или «Отмена» — закрыть без ответа. Нажмите F2 ещё раз — диалог предложит имя, которое вы ввели в прошлый раз.

Разбор

Диалог

```
let mut d = Dialog::centered(40, 9, "Знакомство");
```

`Dialog::centered(ширина, высота, заголовок)` — диалог, который при показе встанет по центру экрана. В ширину и высоту входит рамка.

Строка ввода

```
let mut input = InputLine::new(Rect::new(10, 2, 37, 3), 30);
input.set_text(old_name);
let input_id = d.add(input);
```

- `InputLine::new(прямоугольник, наибольшая длина)` — поле для ввода одной строки. Высота прямоугольника — 1. Текст длиннее поля прокручивается.
- `set_text` — начальный текст. Он будет **выделен**: если сразу начать печатать, он заменится.
- **`d.add(…)` возвращает id** — номер элемента. Сохраняем его: по нему потом достанем введённый текст.

> Почему id, а не ссылка на поле? После `add` полем владеет диалог, и держать на него ссылку Rust не позволит. Номер можно хранить сколько угодно и найти по нему поле, когда понадобится.

Подпись

```
d.add(Label::new(Rect::new(2, 2, 9, 3), "~И~мя:", Some(input_id)));
```

`Label` — подпись, **привязанная** к полю (третий аргумент — id поля). Щелчок по подписи или Alt+И (горячая буква в тильдах) переводят курсор в поле. Пока поле в фокусе, подпись подсвечена.

Кнопки

```
d.add(Button::new(Rect::new(6, 5, 18, 7), "O~K~", cm::OK, bf::DEFAULT));
d.add(Button::new(Rect::new(21, 5, 33, 7), "Отмена", cm::CANCEL, bf::NORMAL));
```

- `Button::new(прямоугольник, надпись, команда, флаги)`. Высота прямоугольника — **2**: вторая строка — тень кнопки.
- Нажатая кнопка посылает свою команду. `cm::OK` и `cm::CANCEL` — особенные: они **закрывают диалог**.
- `bf::DEFAULT` — кнопка «по умолчанию»: её нажимает Enter. Такая кнопка в диалоге одна.
- Esc закрывает диалог так же, как «Отмена».

Показ диалога и ответ

```
let (cmd, d) = app.exec_dialog(d);
if cmd != cm::OK {
    return;
}
let name = d.win.group.find::<InputLine>(input_id).unwrap().text();
```

- `app.exec_dialog(d)` показывает диалог и **ждёт**, пока его закроют. Пока диалог открыт, остальная программа на щелчки не реагирует — это **модальное** окно.
- Возвращает пару: **команду**, которой диалог закрыли (`cm::OK` или `cm::CANCEL`), и **сам диалог** — уже закрытый, но целый.
- Из закрытого диалога достаём поле по id: `find::<InputLine>(id)` ищет среди элементов диалога представление с этим номером и типом `InputLine`. `.text()` — введённый текст.

`d.win.group` — «содержимое диалога»: у диалога есть окно `win`, а у окна — группа `group` с элементами. Запоминать это не нужно: цепочку `d.win.group.find…` просто копируют.

Открыть диалог сразу

```
app.put_event(Event::command_event(CM_ASK));
```

`put_event` ставит команду в очередь, как будто её выбрали в меню. `run` первым делом её обработает — и диалог откроется при запуске.

Попробуйте

1. Добавьте второе поле «Город» на строке 4 и поздоровайтесь так: «Привет, Аня из Тулы!». Не забудьте: диалог придётся сделать выше, а кнопки — опустить.
2. Сделайте «OK» недефолтной (`bf::NORMAL`), а «Отмену» — дефолтной. Что теперь делает Enter?
3. Вместо своего диалога используйте готовую функцию: `input_box(app, "Знакомство", "~И~мя:", &self.name, 30)` — она возвращает `Some(текст)` или `None`. Для одного поля это короче.

Важно запомнить

- `d.add(…)` возвращает id — сохраняйте id полей, из которых потом нужно прочитать данные.
- Кнопки: высота 2, `cm::OK` и `cm::CANCEL` закрывают диалог, `bf::DEFAULT` — нажимается по Enter.

- `let (cmd, d) = app.exec_dialog(d);` — показать и дождаться; данные читаются из возвращённого `d`.

Следующий урок — анкета: флажки, переключатели, списки.

Урок 6. Анкета: флажки, переключатели, список

В диалоге бывает много полей разных видов. Читать каждое по id, как в прошлом уроке, утомительно — поэтому у диалога есть **обмен данными**: все поля заполняются и читаются одним вызовом.

Программа

Файл `examples/learn_06_form.rs`:

```
//! Урок 6. Анкета: флажки, переключатели, список и обмен данными.
//!
//! Запуск: `cargo run --example learn_06_form`. F2 — анкета, Alt+X — выход.

use tvision::*;

const CM_FORM: u16 = 100;

const DRINKS: [&str; 3] = ["Чай", "Кофе", "Сок"];
const HOBBIES: [&str; 4] = ["Книги", "Спорт", "Музыка", "Игры"];
const CITIES: [&str; 6] = ["Москва", "Тула", "Казань", "Тверь", "Сочи", "Омск"];

/// Данные анкеты — обычная структура Rust.
#[derive(Clone, Debug, Default, PartialEq)]
struct Profile {
    name: String,
    age: String,
    drink: u32,   // номер выбранного переключателя: 0 — чай, 1 — кофе, 2 — сок
    hobbies: u32, // флажки: бит 0 — книги, бит 1 — спорт, …
    city: i64,    // номер строки в списке городов
}

impl Profile {
    /// Превратить в данные для диалога. Порядок — как порядок `add`
    /// элементов, у которых есть данные (подписи и кнопки не считаются).
    fn to_value(&self) -> Value {
        Value::List(vec![
            Value::Str(self.name.clone()), // строка ввода «Имя»
            Value::Str(self.age.clone()),  // строка ввода «Возраст»
            Value::U32(self.drink),        // переключатели
            Value::U32(self.hobbies),      // флажки
            Value::Int(self.city),         // список
        ])
    }

    /// Обратно: из данных диалога.
    fn from_value(v: &Value) -> Profile {
        let l = v.as_list().unwrap_or(&[]);
        Profile {
            name: l.first().and_then(|x| x.as_str()).unwrap_or("").to_string(),
            age: l.get(1).and_then(|x| x.as_str()).unwrap_or("").to_string(),
            drink: l.get(2).and_then(|x| x.as_u32()).unwrap_or(0),
            hobbies: l.get(3).and_then(|x| x.as_u32()).unwrap_or(0),
            city: l.get(4).and_then(|x| x.as_int()).unwrap_or(0),
        }
    }

    /// Описание словами — для окна сообщения.
    fn describe(&self) -> String {
        let hobbies: Vec<&str> = (0..HOBBIES.len()).filter(|i| self.hobbies & (1 << i) != 0).map(|i|
HOBBIES[i]).collect();
        format!(
```

```
            "Имя: {}\nВозраст: {}\nНапиток: {}\nУвлечения: {}\nГород: {}",
            self.name,
            self.age,
            DRINKS[self.drink as usize % DRINKS.len()],
            if hobbies.is_empty() { "нет".to_string() } else { hobbies.join(", ") },
            CITIES[self.city as usize % CITIES.len()]
        )
    }
}

/// Диалог «Анкета».
fn form_dialog(p: &Profile) -> Dialog {
    let mut d = Dialog::centered(56, 19, "Анкета");

    // 1. Имя — обычная строка ввода.
    let name = d.add(InputLine::new(Rect::new(14, 2, 52, 3), 40));
    d.add(Label::new(Rect::new(2, 2, 13, 3), "~И~мя:", Some(name)));

    // 2. Возраст — строка с проверкой: целое от 1 до 120.
    let age = d.add(InputLine::new(Rect::new(14, 4, 20, 5), 3).with_validator(RangeValidator::new(1,
120)));
    d.add(Label::new(Rect::new(2, 4, 13, 5), "~В~озраст:", Some(age)));

    // 3. Переключатели: выбрать можно только один вариант.
    let drink = d.add(Cluster::radio_buttons(Rect::new(3, 7, 20, 10), &["~Ч~ай", "~К~офе", "~С~ок"]));
    d.add(Label::new(Rect::new(2, 6, 13, 7), "Напиток", Some(drink)));

    // 4. Флажки: можно отметить сколько угодно. Высота 2 — значит, 2 строки
    //    в столбце, и 4 флажка лягут в два столбца.
    let hobbies = d.add(Cluster::check_boxes(Rect::new(23, 7, 52, 9), &["~К~ниги", "С~п~орт",
"~М~узыка", "И~г~ры"]));
    d.add(Label::new(Rect::new(22, 6, 35, 7), "Увлечения", Some(hobbies)));

    // 5. Список городов с полосой прокрутки справа.
    let sb = d.add(ScrollBar::new(Rect::new(20, 12, 21, 15)));
    let mut list = ListBox::new(Rect::new(3, 12, 20, 15), 1, Some(sb));
    list.set_items(CITIES.iter().map(|s| s.to_string()).collect());
    let list = d.add(list);
    d.add(Label::new(Rect::new(2, 11, 13, 12), "~Г~ород", Some(list)));

    d.add(Button::new(Rect::new(30, 12, 42, 14), "О~К~", cm::OK, bf::DEFAULT));
    d.add(Button::new(Rect::new(30, 15, 42, 17), "Отмена", cm::CANCEL, bf::NORMAL));

    // Заполнить все элементы разом — значениями из структуры.
    d.set_data(&p.to_value());
    d.select(name); // курсор — в поле «Имя»
    d
}

struct App {
    profile: Profile,
}

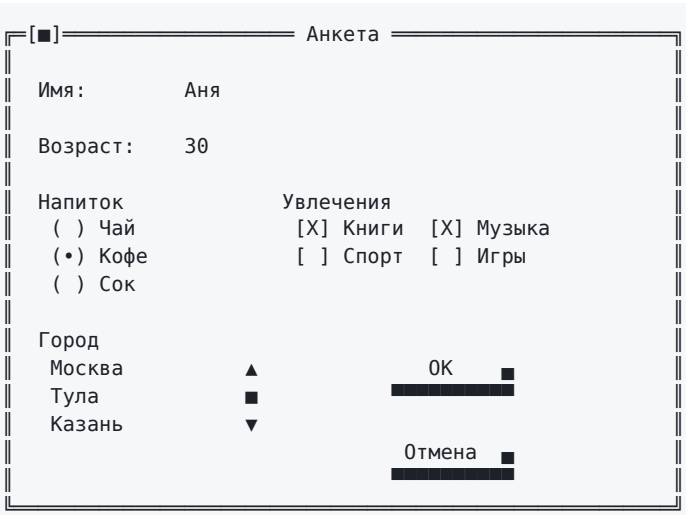
impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        if ev.command() != Some(CM_FORM) {
            return;
        }
        ev.clear();
        let (cmd, d) = app.exec_dialog(form_dialog(&self.profile));
        if cmd == cm::OK {
            // Прочитать все элементы разом.
            self.profile = Profile::from_value(&d.get_data());
            message_box_rect(app, Rect::new(0, 0, 50, 13), &self.profile.describe(), mf::INFORMATION |
mf::OK_BUTTON, true);
        }
    }
}

fn status() -> Vec<StatusDef> {
```

```
    vec![StatusDef::all(vec![StatusItem::new("~F2~ Анкета", kb::F2, CM_FORM), StatusItem::new("~Alt-X~
Выход", kb::ALT_X, cm::QUIT)])]
}

fn main() -> std::io::Result<()> {
    locale::set_language(locale::Language::Ru);
    let mut app = Program::with_terminal(None, Some(status()))?;
    let start = Profile { name: "Аня".into(), age: "30".into(), drink: 1, hobbies: 0b0101, city: 2 };
    app.put_event(Event::command_event(CM_FORM));
    app.run(&mut App { profile: start });
    Ok(())
}
```

Запуск: `cargo run --example learn_06_form`.



Перемещение между полями — Tab и Shift+Tab или мышью. В переключателях и флажках — стрелки и пробел (или горячие буквы). В списке — стрелки. Попробуйте ввести возраст 200 и нажать OK: появится сообщение, а диалог не закроется.

Новые элементы

Переключатели — выбрать один вариант

```
Cluster::radio_buttons(Rect::new(3, 7, 20, 10), &["~Ч~ай", "~К~офе", "~С~ок"])
```

Как кнопки на старом радиоприёмнике: нажал одну — остальные отжались. Значение — **номер** выбранного варианта: 0, 1, 2. Высота прямоугольника — число вариантов (по строке на вариант).

Флажки — отметить сколько угодно

```
Cluster::check_boxes(Rect::new(23, 7, 52, 9), &["~К~ниги", "С~п~орт", "~М~узыка", "И~г~ры"])
```

Значение — **набор битов**: бит 0 — первый флажок, бит 1 — второй и так далее. Отмечены «Книги» и «Музыка» — значение `0b0101` (1 + 4 = 5). Если вариантов больше, чем строк в прямоугольнике, они раскладываются по столбцам: здесь высота 2, поэтому четыре флажка — в два столбца.

Проверить, отмечен ли флажок `i`: `hobbies & (1 << i) != 0`.

Список

```
let sb = d.add(ScrollBar::new(Rect::new(20, 12, 21, 15)));
let mut list = ListBox::new(Rect::new(3, 12, 20, 15), 1, Some(sb));
list.set_items(CITIES.iter().map(|s| s.to_string()).collect());
```

- `ListBox::new(прямоугольник, число столбцов, полоса прокрутки)`.
- Полосу прокрутки (`ScrollBar` шириной 1) кладут рядом **в тот же диалог** и передают списку её id — так они связываются.
- `set_items` — строки списка.
- Значение — номер выбранной строки (тип `i64`).

Проверка ввода

```
InputLine::new(Rect::new(14, 4, 20, 5), 3).with_validator(RangeValidator::new(1, 120))
```

Валидатор проверяет, что ввели. `RangeValidator::new(1, 120)` — целое от 1 до 120. По ходу набора он не пропустит буквы, а при нажатии ОК проверит диапазон и, если число не подходит, покажет сообщение и вернёт курсор в поле. Есть и другие валидаторы — например, маска `PictureValidator::new("##.##.####", true)` для даты; см. справочник.

Обмен данными

Значения элементов

У каждого элемента, который что-то хранит, есть **значение** типа `Value`:

Элемент	Значение
`InputLine` (строка ввода)	`Value::Str(текст)`
переключатели	`Value::U32(номер)`
флажки	`Value::U32(биты)`
`ListBox` (список)	`Value::Int(номер строки)`
подписи, кнопки, полосы прокрутки	значения нет

У диалога значение — **список** значений всех таких элементов **в том порядке, в каком их добавляли** (`Value::List`).

Структура ↔ диалог

Удобно держать данные анкеты в своей структуре и написать два метода перевода:

```
fn to_value(&self) -> Value {
    Value::List(vec![
        Value::Str(self.name.clone()), // 1-й элемент с данными: «Имя»
        Value::Str(self.age.clone()),  // 2-й: «Возраст»
        Value::U32(self.drink),        // 3-й: переключатели
        Value::U32(self.hobbies),      // 4-й: флажки
        Value::Int(self.city),         // 5-й: список
    ])
}
```

`from_value` делает обратное: достаёт значения по номерам. Методы `as_str()`, `as_u32()`, `as_int()` возвращают `Some(…)`, если значение нужного вида, и `None` — если нет; `unwrap_or` даёт запасное значение.

И тогда работа с диалогом — две строки:

```
d.set_data(&p.to_value());                    // заполнить диалог
let p = Profile::from_value(&d.get_data());   // прочитать после OK
```

> **Порядок важен.** Если поменять местами `add` двух полей, поменяется и порядок значений. Держите `to_value` рядом с функцией, которая строит диалог, и меняйте их вместе. Тест `data_roundtrip` в конце файла ловит такие ошибки: он заполняет диалог и проверяет, что прочиталось то же.

Попробуйте

1. Добавьте флажок «Кино» пятым. Что придётся поменять? (Только список надписей `HOBBIES` и подписи флажков — биты сдвигаются сами.)
2. Добавьте между «Возрастом» и «Напитком» поле «Телефон» с фильтром: `FilterValidator::new("0123456789+-() ")` — буквы набрать не получится. Не забудьте добавить его значение в `to_value` и `from_value` **на правильное место**.
3. Сделайте список в два столбца: `ListBox::new(…, 2, …)`.

Важно запомнить

- Переключатели — номер, флажки — биты, список — номер строки, строка ввода — текст.
- `d.set_data(…)` / `d.get_data()` — все поля разом, в порядке `add`.
- Валидатор не пускает неправильные данные дальше кнопки OK.

Следующий урок — своё представление.

Урок 7. Своё представление

Готовых элементов много, но рано или поздно понадобится свой: график, игровое поле, часы, особый индикатор. Сделаем простой — **счётчик**: показывает число, пробел прибавляет 1.

Программа

Файл `examples/learn_07_view.rs`:

```rust
//! Урок 7. Своё представление: счётчик.
//!
//! Запуск: `cargo run --example learn_07_view`. Пробел, Enter или щелчок —
//! прибавить 1; `-` — убавить; `0` — обнулить; Tab — к другому счётчику.
//! Alt+X — выход.

use tvision::*;

/// Счётчик. Любое представление — это структура с полем `vs`
/// (положение, размер, состояние) и своими данными.
pub struct Counter {
    pub vs: ViewState,
    pub value: i32,
}

impl Counter {
    pub fn new(x: i32, y: i32) -> Self {
        // Размер 20×1: одна строка шириной 20 клеток.
        let mut vs = ViewState::new(Rect::new(x, y, x + 20, y + 1));
        // SELECTABLE — может получать фокус (клавиатуру);
        // FIRST_CLICK — первый же щелчок мышью засчитывается.
        vs.options |= of::SELECTABLE | of::FIRST_CLICK;
        Counter { vs, value: 0 }
    }
}

impl View for Counter {
    // Пять служебных методов, одинаковых у всех представлений, пишет макрос.
    tvision::impl_view_basics!();

    /// Нарисовать себя. Вызывается при каждой перерисовке экрана.
    fn draw(&self, dc: &mut DrawCtx) {
        // Цвет берём по номеру из палитры (см. `palette` ниже):
        // 1 — обычный, 2 — когда счётчик в фокусе.
        let color = dc.color(if self.vs.is_focused() { 2 } else { 1 });
        // Сначала закрашиваем всю свою строку, потом пишем текст.
        dc.fill(self.vs.extent(), ' ', color);
        dc.write_str(1, 0, &format!("Счёт: {}", self.value), color);
    }

    /// Палитра: возьмём ту же, что у строки ввода, — так счётчик будет
    /// выглядеть «как поле» в любом диалоге.
    fn palette(&self) -> &'static [u8] {
        color::palettes::INPUT_LINE
    }

    /// Обработать событие: клавишу или мышь.
    fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
        // Стандартное поведение: щелчок по счётчику переводит на него фокус.
        view::default_handle_event(&self.vs, ev, ctx);
        match ev {
            Event::MouseDown(_) => {
                self.value += 1;
                ev.clear(); // событие обработано
            }
```

```
            // Клавиши получает только тот, кто в фокусе.
            Event::KeyDown(k) if self.vs.is_focused() => {
                match k.key {
                    Key::Char(' ') | Key::Enter => self.value += 1,
                    Key::Char('-') => self.value -= 1,
                    Key::Char('0') => self.value = 0,
                    _ => return, // чужая клавиша (например, Tab) — пусть обработают другие
                }
                ev.clear();
            }
            _ => {}
        }
    }
}

fn open_window(app: &mut Program) {
    let mut w = Window::new(Rect::new(10, 3, 46, 11), "Счётчики", 1);
    w.add(StaticText::new(Rect::new(2, 1, 34, 2), "Пробел +1, «-» −1, Tab — дальше"));
    w.add(Counter::new(2, 3));
    w.add(Counter::new(2, 5));
    app.insert_window(w);
}

fn main() -> std::io::Result<()> {
    let status = vec![StatusDef::all(vec![StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT)])];
    let mut app = Program::with_terminal(None, Some(status))?;
    open_window(&mut app);
    app.run(&mut program::NoHandler);
    Ok(())
}
```

Запуск: `cargo run --example learn_07_view`. В окне два счётчика. Пробел или Enter прибавляет единицу, `-` вычитает, `0` обнуляет. Tab переводит фокус на другой счётчик — он подсвечивается. Щелчок мышью тоже прибавляет.

Из чего состоит представление

Любое представление — это:

1. **структура** с полем `vs: ViewState` и своими данными;
2. **реализация трейта** `View`: как себя рисовать (`draw`) и как отвечать на события (`handle_event`).

Структура

```
pub struct Counter {
    pub vs: ViewState,
    pub value: i32,
}
```

`ViewState` — общие для всех представлений сведения: где оно лежит (`origin`), какого размера (`size`), в фокусе ли (`is_focused()`), какие у него флаги. Создаётся из прямоугольника: `ViewState::new(rect)`.

Флаги

```
vs.options |= of::SELECTABLE | of::FIRST_CLICK;
```

- `of::SELECTABLE` — представление может получить **фокус**, то есть клавиатуру. Без этого флага Tab его пропускает (так ведут себя подписи и тексты).
- `of::FIRST_CLICK` — первый же щелчок мышью засчитывается. Без него первый щелчок только переводит фокус.

Служебные методы

```
impl View for Counter {
    tvision::impl_view_basics!();
```

Трейту `View` нужны пять одинаковых у всех служебных методов (доступ к `vs` и приведение типов). Макрос `impl_view_basics!` пишет их за вас. Он работает, если поле называется именно `vs`.

Рисование

```
fn draw(&self, dc: &mut DrawCtx) {
    let color = dc.color(if self.vs.is_focused() { 2 } else { 1 });
    dc.fill(self.vs.extent(), ' ', color);
    dc.write_str(1, 0, &format!("Счёт: {}", self.value), color);
}
```

`draw` вызывается при каждой перерисовке экрана — после любого события. Вам не нужно следить, изменилось ли что-то: просто рисуйте текущее состояние.

- `dc` — «холст» представления. Координаты на нём **свои**: `(0, 0)` — левый верхний угол счётчика, где бы он ни лежал на экране.
- `dc.fill(прямоугольник, символ, цвет)` — залить. `self.vs.extent()` — весь свой прямоугольник. **Закрашивайте всю свою область**: что не нарисовано, останется от прошлого кадра.
- `dc.write_str(x, y, текст, цвет)` — написать строку.
- `dc.color(n)` — цвет номер `n` из **палитры** представления.

Палитра — откуда берутся цвета

```
fn palette(&self) -> &'static [u8] {
    color::palettes::INPUT_LINE
}
```

Цвета в Turbo Vision задаются не напрямую, а **номерами**. Счётчик говорит «мне цвет номер 1», палитра строки ввода переводит его в номер цвета диалога, палитра диалога — в номер цвета программы, а там уже настоящий цвет. Зачем так сложно? Чтобы один и тот же элемент выглядел правильно и в сером диалоге, и в синем окне, а пользователь мог перекрасить всю программу в одном месте.

На практике: возьмите палитру готового элемента, который похож на ваш (`INPUT_LINE`, `STATIC_TEXT`, `BUTTON`, `LIST_VIEWER`), и используйте номера 1, 2… как у него. Для `INPUT_LINE` 1 — обычный, 2 — в фокусе. Если нужен конкретный цвет, можно не брать его из палитры, а задать сам: `color::attr(14, 1)` — жёлтый на синем.

События

```
fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
    view::default_handle_event(&self.vs, ev, ctx);
    match ev {
        Event::MouseDown(_) => { self.value += 1; ev.clear(); }
        Event::KeyDown(k) if self.vs.is_focused() => {
            match k.key {
                Key::Char(' ') | Key::Enter => self.value += 1,
                Key::Char('-') => self.value -= 1,
                Key::Char('0') => self.value = 0,
                _ => return,
            }
            ev.clear();
        }
        _ => {}
    }
}
```

- Первой строкой вызываем **стандартное поведение**: щелчок по представлению переводит на него фокус.
- `Event` — перечисление: `KeyDown` (клавиша), `MouseDown` (нажата кнопка мыши), `Command` (команда) и другие.
- `k.key` — какая клавиша: `Key::Char(c)` для букв и знаков, `Key::Enter`, `Key::Up`, `Key::F(5)` и т. д.; `k.mods` — зажатые Shift/Ctrl/Alt.
- **Чужие клавиши не трогаем** (`_ => return`) — иначе перестанет работать Tab, Alt+X и всё остальное.
- Своё обработали — `ev.clear()`.

`ctx` — «связь с внешним миром»: через него представление может послать команду (`ctx.post_command`), закрыть окно, открыть диалог. В этом уроке он не нужен; о нём — в справочнике.

Как представление получает события

- **Клавиши** идут только представлению **в фокусе** — тому, что выбрано в активном окне. Поэтому в коде проверка `if self.vs.is_focused()`.
- **Мышь** идёт тому, над кем курсор.
- Если представление не обработало событие (не вызвало `clear`), его получают другие: окно, программа, обработчик приложения.

Попробуйте

1. Сделайте счётчик шире и нарисуйте полоску: `'█'` столько раз, каково значение (`dc.write_char(x, 0, '█', color, self.value)`).
2. Добавьте клавиши ↑ и ↓ (`Key::Up`, `Key::Down`).
3. Положите счётчик в диалог из урока 5 и добавьте ему обмен данными: методы `has_data` (вернуть `true`), `get_data` (`Value::Int(self.value as i64)`) и `set_data`. Пример — в справочнике.

Важно запомнить

- Представление = структура с `vs: ViewState` + `impl View` с `impl_view_basics!()`, `draw` и (обычно) `handle_event`.
- В `draw` координаты свои, от `(0, 0)`; закрашивайте всю область.
- Цвета — номерами через `palette()`, удобно взять палитру похожего элемента.
- Клавиши — только в фокусе; чужие клавиши не трогайте; своё — `clear()`.

Следующий урок — своё окно с данными.

Урок 8. Своё окно с данными

В настоящей программе окно не просто показывает текст — оно хранит **данные**: список дел, открытый файл, таблицу. Сделаем окно «Дела» со списком, в который можно добавлять дела, отмечать сделанные и удалять. Таких окон можно открыть несколько — у каждого свои данные.

Программа

Файл `examples/learn_08_todo.rs`:

```
//! Урок 8. Своё окно с данными: список дел.
//!
//! Запуск: `cargo run --example learn_08_todo`. Ins — новое дело,
//! F4 — отметить сделанным, Del — удалить, F3 — ещё одно окно-список,
//! Alt+X — выход.

use tvision::*;

const CM_ADD: u16 = 100;
const CM_DONE: u16 = 101;
const CM_DELETE: u16 = 102;
const CM_NEW_LIST: u16 = 103;

/// Одно дело.
#[derive(Clone, Debug, PartialEq)]
pub struct Task {
    pub text: String,
    pub done: bool,
}

/// Окно «Дела»: обычное окно + список на экране + наши данные.
pub struct TodoWindow {
    /// Стандартное окно: рамка, заголовок, дети.
    pub win: Window,
    /// Номер (id) списка внутри окна — по нему список находится.
    pub list: ViewId,
    /// Данные: что показывает список.
    pub tasks: Vec<Task>,
}

impl TodoWindow {
    pub fn new(bounds: Rect, title: &str, number: i16) -> Self {
        let mut win = Window::new(bounds, title, number);
        let sb = win.standard_scroll_bar(true); // полоса прокрутки на правой рамке
        let mut list = ListBox::new(win.interior(), 1, Some(sb));
        list.vs.grow_mode = gf::GROW_HI_X | gf::GROW_HI_Y; // тянуться вместе с окном
        let list = win.add(list);
        TodoWindow { win, list, tasks: Vec::new() }
    }

    /// Переложить `tasks` в список на экране.
    pub fn refresh(&mut self) {
        let lines: Vec<String> = self.tasks.iter().map(|t| format!("[{}] {}", if t.done { 'x' } else {
' ' }, t.text)).collect();
        let list = self.win.group.find_mut::<ListBox>(self.list).unwrap();
        let keep = list.focused;
        list.set_items(lines);
        list.focus_item(keep.min(self.tasks.len().saturating_sub(1)));
    }

    /// Номер дела под курсором.
    pub fn current(&self) -> Option<usize> {
        let list = self.win.group.find::<ListBox>(self.list)?;
```

```
        (list.focused < self.tasks.len()).then_some(list.focused)
    }
}

// Чтобы окно попало на рабочий стол, оно должно быть представлением.
// Всю работу делаем руками стандартного окна `win` — «передаём» её ему.
impl View for TodoWindow {
    // Служебные методы и всё про детей — у группы окна.
    tvision::impl_group_basics!(win.group);

    // «Да, это окно» — тогда работают Alt+F3, F5, Alt+номер.
    fn as_window(&self) -> Option<&Window> {
        Some(&self.win)
    }
    fn draw(&self, dc: &mut DrawCtx) {
        self.win.draw_window(dc);
    }
    fn palette(&self) -> &'static [u8] {
        self.win.palette
    }
    fn set_state(&mut self, flag: u16, enable: bool) {
        self.win.set_window_state(flag, enable);
    }
    fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
        self.win.handle_window_event(ev, ctx);
    }
}

struct App {
    lists: i16,
}

impl App {
    /// Активное окно «Дела», если оно сейчас сверху.
    fn active(app: &mut Program) -> Option<&mut TodoWindow> {
        // Окна хранятся как «какое-то представление» (`Box<dyn View>`);
        // `downcast_mut` проверяет, что это именно `TodoWindow`.
        app.desktop_mut().group.current_view_mut()?.as_any_mut().downcast_mut::<TodoWindow>()
    }

    fn new_list(&mut self, app: &mut Program) {
        self.lists += 1;
        let n = self.lists;
        let r = Rect::new(2 + 4 * (n as i32 - 1), 1 + (n as i32 - 1), 46 + 4 * (n as i32 - 1), 15 + (n
as i32 - 1));
        let mut w = TodoWindow::new(r, &format!("Дела {}", n), n);
        w.tasks = vec![Task { text: "Прочитать урок 8".into(), done: false }];
        w.refresh();
        app.insert_window(w);
    }

    fn add(&mut self, app: &mut Program) {
        // Сначала спрашиваем текст — окно в это время не трогаем.
        let Some(text) = input_box(app, "Новое дело", "~Ч~то сделать:", "", 80) else { return };
        if text.trim().is_empty() {
            return;
        }
        // Потом снова находим окно и меняем данные.
        if let Some(w) = Self::active(app) {
            w.tasks.push(Task { text: text.trim().to_string(), done: false });
            w.refresh();
            let last = w.tasks.len() - 1;
            w.win.group.find_mut::<ListBox>(w.list).unwrap().focus_item(last);
        }
    }

    fn done(&mut self, app: &mut Program) {
        if let Some(w) = Self::active(app) {
            if let Some(i) = w.current() {
                w.tasks[i].done = !w.tasks[i].done;
```

```
                w.refresh();
            }
        }
    }

    fn delete(&mut self, app: &mut Program) {
        if let Some(w) = Self::active(app) {
            if let Some(i) = w.current() {
                w.tasks.remove(i);
                w.refresh();
            }
        }
    }
}

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        match ev.command() {
            Some(CM_ADD) => self.add(app),
            Some(CM_DONE) => self.done(app),
            Some(CM_DELETE) => self.delete(app),
            Some(CM_NEW_LIST) => self.new_list(app),
            _ => return,
        }
        ev.clear();
    }
}

fn status() -> Vec<StatusDef> {
    vec![StatusDef::all(vec![
        StatusItem::new("~Ins~ Добавить", kb::key(Key::Ins), CM_ADD),
        StatusItem::new("~F4~ Сделано", kb::f(4), CM_DONE),
        StatusItem::new("~Del~ Удалить", kb::key(Key::Del), CM_DELETE),
        StatusItem::new("~F3~ Новый список", kb::F3, CM_NEW_LIST),
        StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT),
        StatusItem::new("", kb::F6, cm::NEXT),
        StatusItem::new("", kb::ALT_F3, cm::CLOSE),
    ])]
}

fn main() -> std::io::Result<()> {
    locale::set_language(locale::Language::Ru);
    let mut app = Program::with_terminal(None, Some(status()))?;
    let mut h = App { lists: 0 };
    h.new_list(&mut app);
    app.run(&mut h);
    Ok(())
}
```

Запуск: `cargo run --example learn_08_todo`.

- Ins — новое дело (откроется окно ввода);
- F4 — отметить дело под курсором сделанным (`[x]`) или снять отметку;
- Del — удалить;
- F3 — ещё одно окно-список; F6 — переключение между окнами.

Разбор

Окно = готовое окно + данные

```
pub struct TodoWindow {
    pub win: Window,
    pub list: ViewId,
    pub tasks: Vec<Task>,
}
```

В Rust нет наследования, как в C++, поэтому «своё окно на основе обычного» делают так: кладут **обычное окно `Window` полем** внутрь своей структуры и добавляют свои данные. Обычное окно умеет рамку, заголовок, перемещение, детей; наша структура — всё остальное.

- `win` — обычное окно;
- `list` — id списка внутри окна (по нему список находится);
- `tasks` — **наши данные**. Список на экране — только их отображение.

Данные → экран

```
pub fn refresh(&mut self) {
    let lines: Vec<String> = self.tasks.iter().map(|t| format!("[{}] {}", …)).collect();
    let list = self.win.group.find_mut::<ListBox>(self.list).unwrap();
    …
    list.set_items(lines);
```

Правило простое: **меняем данные — вызываем `refresh`**. Он заново перекладывает дела в строки списка. Так данные и экран не расходятся.

`self.win.group.find_mut::<ListBox>(self.list)` — найти в окне представление с id `self.list` и типом `ListBox`, чтобы его изменить.

Делаем окно представлением

```
impl View for TodoWindow {
    tvision::impl_group_basics!(win.group);
    fn as_window(&self) -> Option<&Window> { Some(&self.win) }
    fn draw(&self, dc: &mut DrawCtx) { self.win.draw_window(dc); }
    fn palette(&self) -> &'static [u8] { self.win.palette }
    fn set_state(&mut self, flag: u16, enable: bool) { self.win.set_window_state(flag, enable); }
    fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) { self.win.handle_window_event(ev, ctx);
}
}
```

Чтобы окно можно было положить на рабочий стол, оно должно быть представлением (`View`). Всю работу мы **передаём** обычному окну `win`:

- `impl_group_basics!(win.group)` — как `impl_view_basics!` из урока 7, но для представления с детьми: «служебное и дети — у `win.group`»;
- `as_window` — «я окно»: без этого не заработают Alt+F3, F5 и Alt+номер;
- `draw`, `palette`, `set_state`, `handle_event` — просто вызывают то же у `win`.

Это шаблон: его можно копировать в любое своё окно, меняя только имя структуры. Своя обработка событий дописывается в `handle_event` после строки `self.win.handle_window_event(ev, ctx);`.

Найти своё окно

```
fn active(app: &mut Program) -> Option<&mut TodoWindow> {
    app.desktop_mut().group.current_view_mut()?.as_any_mut().downcast_mut::<TodoWindow>()
}
```

Команды (Ins, F4, Del) приходят в обработчик приложения. Ему нужно найти окно, к которому они относятся, — активное:

- `app.desktop_mut().group.current_view_mut()` — активное окно рабочего стола, но «какое-то представление» (`dyn View`): программа не знает наших типов;

- `.as_any_mut().downcast_mut::<TodoWindow>()` — «это `TodoWindow`?». Если да — получаем наше окно со всеми полями, если нет — `None`.

Сначала спросить, потом менять

```
fn add(&mut self, app: &mut Program) {
    let Some(text) = input_box(app, "Новое дело", "~Ч~то сделать:", "", 80) else { return };
    …
    if let Some(w) = Self::active(app) {
        w.tasks.push(…);
        w.refresh();
```

Обратите внимание на порядок: **сначала** показываем окно ввода и получаем текст, **потом** находим окно и меняем данные. Наоборот не получится: пока мы держим ссылку на окно (`w`), Rust не даст передать `app` в `input_box` — ведь окно лежит внутри `app`. Этот приём — «скопировать нужное, показать диалог, снова найти окно» — встречается во всех программах на библиотеке.

Попробуйте

1. Добавьте команду «Переименовать» (F2): спросите новый текст через `input_box` с текущим текстом дела и запишите его в `tasks`.
2. Покажите в заголовке окна число несделанных дел: `self.win.title = format!("Дела: {}", осталось);` в `refresh`.
3. Перед удалением спросите подтверждение: `message_box(…, mf::CONFIRMATION | mf::YES_BUTTON | mf::NO_BUTTON) == cm::YES`.

Важно запомнить

- Своё окно = `Window` полем + свои данные + шаблонная реализация `View`.
- Меняем данные → `refresh()` → экран.
- Активное окно своего типа: `current_view_mut()?.as_any_mut().downcast_mut::<T>()`.
- Сначала диалог, потом изменения: не держите ссылку на окно, пока открыт диалог.

Следующий урок — собираем всё вместе: «Контакты».

Урок 9. Собираем всё вместе: «Контакты»

Теперь у вас есть всё, чтобы написать настоящее приложение. Разберём записную книжку «Контакты»: таблица контактов в окне, диалог правки с проверкой ввода, добавление и удаление, сохранение в файл и вопрос «Сохранить изменения?» при выходе.



Программа большая (около 500 строк вместе с комментариями и тестами), поэтому здесь — не весь код, а **карта**: из каких частей она состоит и что в каждой нового. Откройте рядом файл `examples/guide_contacts.rs` и запустите:

```
cargo run --example guide_contacts
```

Попробуйте: Ins — добавить, Enter или F4 — изменить, Del — удалить, F2 — сохранить (в первый раз спросит имя файла), Alt+X — выйти (спросит, сохранить ли изменения).

Карта программы

Часть	Что делает	Знакомо из урока
константы `CM_…`	свои команды	2
`struct Contact` + `to_value` / `from_value`	данные и обмен с диалогом	6
`to_line` / `from_line`, `load_contacts`, `save_contacts`	файл: строка на контакт	обычный Rust
`contact_dialog`	диалог с полями и проверкой	5, 6
`struct ContactsWindow` + `impl View`	своё окно с таблицей и данными	8
`struct App` + `impl AppHandler`	команды: добавить, изменить, удалить, файлы, выход	2, 8
`fn idle`	запрет недоступных команд	**новое**
`menu`, `status`, `main`	меню, строка состояния, запуск	2
`mod tests`	тесты	10

Ниже — то, чего ещё не было в уроках.

Таблица

В уроке 8 данные показывал список `ListBox`. Для нескольких столбцов есть **таблица** `Table`:

```
let model = VecTableModel::new(&["Имя", "Телефон", "Почта", "Группа", "★"], Vec::new());
let cols = vec![
    TableColumn::new(22),
    TableColumn::new(18),
    TableColumn::new(22),
    TableColumn::new(8),
    TableColumn::new(3).align(Align::Center),
];
let mut table = Table::new(win.interior(), Box::new(model), Some(v), Some(h))
    .with_columns(cols)
    .selection(SelectionMode::Row)
    .striped(true)
    .sortable(false);
table.vs.grow_mode = gf::GROW_HI_X | gf::GROW_HI_Y;
```

- Таблица берёт ячейки из **модели**. `VecTableModel` — самая простая модель: заголовки и строки как `Vec<Vec<String>>`.
- `TableColumn::new(ширина)` — описание столбца; `.align(…)` — выравнивание.
- Настройки цепочкой: `.selection(SelectionMode::Row)` — курсор на всю строку, `.striped(true)` — строки через одну чуть другого цвета, `.sortable(false)` — не сортировать щелчком по заголовку (мы сортируем сами, командой, — так номер строки таблицы всегда равен номеру контакта в `contacts`).
- `v`, `h` — полосы прокрутки на рамке: `win.standard_scroll_bar(true)` и `(false)`.
- `grow_mode` — чтобы таблица растягивалась вместе с окном (урок 4).

Как в уроке 8, есть метод `reload`, который перекладывает `contacts` в таблицу; его вызывают после каждого изменения. Строка под курсором — `table.cursor().0`.

Enter в таблице

Таблица не «съедает» Enter: если она его не обработала, Enter возвращается к окну. Окно этим пользуется — превращает Enter (и двойной щелчок) в команду «Изменить»:

```
fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
    let double = matches!(ev, Event::MouseDown(m) if m.double);
    self.win.handle_window_event(ev, ctx);
    if matches!(ev, Event::KeyDown(k) if k.key == Key::Enter && k.mods == Mods::NONE) || double {
        ctx.post_command(CM_EDIT, Info::None);
        ev.clear();
    }
}
```

`ctx.post_command(команда, Info::None)` ставит команду в очередь — она придёт в `AppHandler`, как будто её выбрали в меню. Так окно и обработчик приложения «разговаривают».

Проверка ввода в диалоге

```
let name = InputLine::new(Rect::new(13, 2, 49, 3), 60)
    .with_validator(FnValidator { f: |s: &str| !s.trim().is_empty(), message: "Имя не может быть
пустым.".into() });
```

`FnValidator` проверяет поле **вашей функцией**: `f` получает текст и возвращает `true`, если всё хорошо; `message` — что сказать, если нет. Проверка срабатывает при нажатии ОК. Телефону поставлен `FilterValidator` — буквы в нём просто не набираются.

Запрет недоступных команд

Если контактов нет, «Изменить» и «Удалить» не имеют смысла. Их можно **запретить**: пункты меню и строки состояния станут серыми, а клавиши перестанут срабатывать.

```
fn idle(&mut self, app: &mut Program) {
    let (has_window, has_rows) = match Self::window(app) {
        Some(w) => (true, !w.contacts.is_empty()),
        None => (false, false),
    };
    for (cmd, on) in [(CM_ADD, has_window), (CM_EDIT, has_rows), (CM_DELETE, has_rows) /* … */] {
        if on { app.enable_command(cmd); } else { app.disable_command(cmd); }
    }
}
```

`idle` — второй метод `AppHandler`: программа вызывает его, когда событий нет. Удобное место, чтобы привести команды в порядок.

> Запрещать можно только команды с номерами до 255 — поэтому свои команды удобно нумеровать от 100 до 255.

Файлы

Чтение и запись — обычный Rust (`std::fs`). От интерфейса нужен только выбор файла — для этого есть готовый диалог:

```
let d = FileDialog::new("*.tsv", "Открыть", "~И~мя", fd::OPEN_BUTTON, 100);
let (cmd, d) = app.exec_dialog(d);
if cmd == cm::CANCEL {
    return;
}
let path = d.file_name();
```

`FileDialog::new(шаблон, заголовок, подпись поля, кнопки, номер истории)`: показывает файлы по шаблону, даёт ходить по каталогам и запоминает введённые имена (номер истории — любое число, у каждого диалога свой). Для «Сохранить как» — кнопка `fd::OK_BUTTON`.

Вопрос при выходе

```
match message_box(app, "\x03Сохранить изменения?", mf::CONFIRMATION | mf::YES_NO_CANCEL) {
    cm::YES => self.save(app, false),
    cm::NO => true,
    _ => false,
}
```

Три кнопки — «Да», «Нет», «Отмена». Выход сделан своей командой `CM_EXIT` (а не `cm::QUIT`), чтобы успеть спросить: только если ответ не «Отмена», вызывается `app.quit()`.

Попробуйте

1. Добавьте поле «День рождения» с маской `PictureValidator::new("##.##.####", true)` и столбец в таблице.
2. Сделайте поиск: команда «Найти» (`input_box`) ставит курсор на первый контакт, имя которого содержит введённый текст (`w.select(i)`).

3. Разрешите сортировку щелчком по заголовку (`.sortable(true)`) — и подумайте, почему после этого «Изменить» правит не тот контакт. (Подсказка: таблица переставила свои строки, а `contacts` — нет.)

Следующий урок — тесты.

Урок 10. Тесты

Программу с интерфейсом тоже можно проверять обычными тестами `cargo test` — без терминала, без человека и без мыши. В каждом примере уроков в конце файла есть такие тесты; разберёмся, как они устроены.

«Невидимый» терминал

Обычно программа создаётся так: `Program::with_terminal(…)` — и рисует в настоящий терминал. Для тестов есть **невидимый терминал** `HeadlessBackend`:

- нажатия клавиш и щелчки берутся из **очереди**, которую заполняет тест;
- экран хранится в памяти — его можно прочитать как текст.

```
use tvision::platform::HeadlessBackend;

let mut b = HeadlessBackend::new(80, 25);   // экран 80×25
b.push_key(kb::F2);                         // «нажать» F2
b.push_key(kb::ENTER);
b.push_click(10, 5);                        // «щёлкнуть» в столбце 10, строке 5
let mut app = Program::new(Box::new(b), Some(menu()), Some(status()));
```

`Program::new(терминал, меню, строка состояния)` — то же, что `with_terminal`, но с любым терминалом.

Запустить и проверить

```
let mut h = App { greetings: 0 };
app.pump(&mut h);                 // обработать всю очередь
assert_eq!(h.greetings, 1);       // проверить данные
assert!(app.screen_text().contains("Файл"));   // проверить экран
```

- `app.pump(&mut обработчик)` — обработать все события из очереди и вернуться (в отличие от `run`, который ждёт выхода);
- проверять можно **данные** (поля обработчика, окна) и **экран**: `app.screen_text()` — весь последний кадр текстом.

Пример целиком

Тест из урока 5: ввести имя в диалог и проверить, что программа его запомнила.

```
#[test]
fn remembers_name() {
    let mut b = HeadlessBackend::new(80, 25);
    b.push_key(kb::F2);                              // открыть диалог
    for c in "Аня".chars() {                         // набрать «Аня»
        b.push_key(KeyEvent::new(Key::Char(c), Mods::NONE));
    }
    b.push_key(kb::ENTER);                           // нажать OK
    let mut app = Program::new(Box::new(b), None, Some(status()));
    let mut h = App { name: String::new() };
    app.pump(&mut h);
    assert_eq!(h.name, "Аня");
}
```

Запуск: `cargo test --example learn_05_dialog`. Все тесты библиотеки и примеров — `cargo test --all-targets`.

Окна, которые никто не закрыл

После ОК пример показывает окно «Привет, Аня!». В тесте его никто не закроет — и тест должен был бы зависнуть. Чтобы этого не случилось, невидимый терминал, когда очередь кончилась, сам «нажимает» Esc: забытые окна закрываются.

Обычно это то, что нужно. Но если ваше окно ждёт чего-то долгого (например, загрузки файла в отдельном потоке), Esc его отменит. Тогда выключите это: `b.esc_budget = 0;` — и следите, чтобы в тесте не оставалось незакрытых окон.

Что ещё удобно

- **Команда без клавиш:** `app.put_event(Event::command_event(CM_HELLO)); app.pump(&mut h);` — как будто выбрали пункт меню.
- **Диалог отдельно:** `let (cmd, d) = app.exec_dialog(my_dialog());` — с очередью клавиш для этого диалога; потом проверить `cmd` и данные `d.get_data()`.
- **Без программы вовсе:** функции вроде `to_value` / `from_value` или раскладку диалога можно проверять напрямую. Тест `dialog_inside_frame` из урока 4 проверяет, что ни один элемент не налез на рамку:

```
let d = layout_dialog();
let s = d.win.group.vs.size;
let inside = Rect::new(1, 1, s.x - 1, s.y - 1);
for c in &d.win.group.children {
    let b = c.vs().bounds();
    assert_eq!(inside.intersect(&b), b, "{:?} залезает на рамку", b);
}
```

Попробуйте

1. Напишите тест для урока 3: F3 три раза — на рабочем столе три окна.
2. Напишите тест для урока 7: щелчок по первому счётчику — на экране «Счёт: 1». (Подсказка: окно лежит в `(10, 3)`, счётчик — в `(2, 3)` окна; рабочий стол начинается со строки 0, потому что меню нет.)

Что дальше

Дальше — MVC: данные в JSON и SQLite: как получить программу для записей из одного описания полей, и рецепты с данными. Дальше — по мере надобности:

- **Справочник** — каждый класс библиотеки: что это, когда нужен, как создать, пример.
- **Примеры** в каталоге `examples/` — от простых до больших: `demo` (все элементы диалогов), `tvdemo` (порт демонстрационной программы Turbo Vision), `tvedit` (текстовый редактор), `table`, `treeview`, `json_table`, `mvc_contacts`, `lfm` (файловый менеджер).
- **Тематические главы** — как библиотека устроена внутри, цвета и палитры, перенос программ с C++ и Pascal.

Урок 11. MVC: данные в JSON и SQLite

В уроке 9 мы написали «Контакты» вручную: своё окно, обмен данными с диалогом, чтение и запись файла. Это полезно, чтобы понять, как всё устроено, — но для каждой новой программы (склад, библиотека книг, список задач) пришлось бы писать то же самое заново.

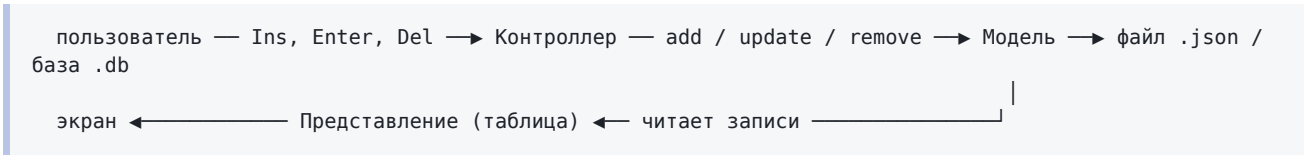
В этом уроке та же программа получится из **описания полей**: какие поля есть у записи и какого они вида. Таблицу, диалог правки, проверку ввода и сохранение в файл JSON или в базу SQLite библиотека сделает сама — модуль `tvision::mvc`.

Программа урока: `cargo run --example mvc_contacts`.

Что такое MVC

Model–View–Controller (модель — представление — контроллер) — способ разделить программу с интерфейсом на три части, чтобы они не мешали друг другу:

Часть	Что делает	В `tvision::mvc`
Модель	хранит данные и знает правила: какие поля обязательны, куда сохранять	`Model` + хранилище (`JsonStorage`, `SqliteStorage`)
Представление	показывает данные; само их не меняет	`ModelWindow` — окно с таблицей
Контроллер	принимает команды пользователя и меняет модель	`CrudController` — «Добавить», «Изменить», «Удалить»



Главная выгода: части можно менять по отдельности. Чтобы хранить данные в базе вместо файла, меняется только хранилище — окно и контроллер об этом даже не узнают.

Шаг 1. Опишите поля

Запись — это строка таблицы. Её поля описывает **схема** (`Schema`): имя таблицы и список полей.

```rust
use tvision::mvc::*;
use tvision::*;

fn schema() -> Schema {
    Schema::new(
        "contacts",                                          // имя таблицы в базе SQLite
        vec![
            Field::text("name", "~И~мя").required().max_len(60).width(22),
            Field::text("phone", "~Т~елефон").max_len(30).width(18),
            Field::text("email", "~П~очта").max_len(60).width(22).check(is_email, "В адресе почты нет
«@».").
            Field::choice("group", "~Г~руппа", &["Семья", "Работа", "Друзья"]),
            Field::bool("favorite", "Из~б~ранный").width(3),
        ],
    )
}

fn is_email(s: &str) -> bool {
    s.contains('@')
}
```

У каждого поля два имени:

- **`"name"`** — для хранения: ключ в JSON, столбец в базе. Латиницей, без пробелов.
- **`"~И~мя"`** — для человека: подпись в диалоге и заголовок столбца. Буква между тильдами — горячая клавиша (Alt+И).

Вид поля решает всё остальное:

Создать	В таблице	В диалоге	Хранится как
`Field::text(…)`	текст	строка ввода	строка
`Field::int(…)`	число справа	строка ввода, в которую можно набрать только цифры	целое
`Field::real(…)`	число справа	строка ввода (запятая или точка)	дробное
`Field::bool(…)`	`✓`	флажок	`true` / `false` (в базе — 1 / 0)
`Field::choice(…, &[варианты])`	текст варианта	переключатели	строка

И уточнения — их можно писать цепочкой:

Метод	Что делает
`.required()`	поле нельзя оставить пустым
`.max_len(n)`	не больше `n` символов
`.width(n)`	ширина столбца таблицы
`.check(функция, сообщение)`	своя проверка: функция получает текст и возвращает `true`, если всё хорошо. Для пустого необязательного поля не вызывается

Шаг 2. Откройте модель

```rust
let model = Model::open(schema(), open_storage("contacts.json")?)?.into_ref();
```

Разберём изнутри наружу:

- `open_storage(путь)` выбирает **хранилище** по расширению файла: `.json` — файл JSON, `.db`, `.sqlite`, `.sqlite3` — база SQLite.
- `Model::open(схема, хранилище)` создаёт модель и сразу читает все записи. Файла ещё нет — не страшно: записей просто ноль, файл появится при первом изменении.
- `.into_ref()` заворачивает модель в `Rc<RefCell<…>>` (тип `ModelRef`). Так ей могут пользоваться сразу двое: окно (чтобы рисовать) и контроллер (чтобы менять).

> **`Rc<RefCell<T>>` за одну минуту.** `Rc` — «общий владелец»: копия `model.clone()` не копирует данные, а даёт ещё одну ссылку на ту же модель. `RefCell` разрешает менять общие данные: `model.borrow()` — прочитать, `model.borrow_mut()` — изменить. Одновременно `borrow_mut` и что-то ещё нельзя — программа остановится с ошибкой, поэтому не держите `borrow` долго (например, на время диалога).

Шаг 3. Окно и контроллер

```
struct App {
    crud: CrudController,
}

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        self.crud.handle(app, ev);           // Добавить, Изменить, Удалить
    }
    fn idle(&mut self, app: &mut Program) {
        self.crud.update_commands(app);      // нет записей — «Изменить» и «Удалить» серые
    }
}

fn main() -> std::io::Result<()> {
    let model = Model::open(schema(), open_storage("contacts.json").unwrap()).unwrap().into_ref();
    let mut app = Program::with_terminal(Some(menu()), Some(status()))?;
    let r = app.desktop().group.vs.extent();                  // весь рабочий стол
    let crud = CrudController::open(&mut app, r, "Контакты", model);
    app.run(&mut App { crud });
    Ok(())
}
```

`CrudController::open` делает две вещи: вставляет на рабочий стол окно `ModelWindow` с таблицей и создаёт контроллер, который знает это окно.

Осталось связать клавиши с командами. Команды записей уже есть в модуле: `CM_RECORD_ADD`, `CM_RECORD_EDIT`, `CM_RECORD_DELETE`.

```
fn status() -> Vec<StatusDef> {
    vec![StatusDef::all(vec![
        StatusItem::new("~Ins~ Добавить", kb::key(Key::Ins), CM_RECORD_ADD),
        StatusItem::new("~F4~ Изменить", kb::f(4), CM_RECORD_EDIT),
        StatusItem::new("~Del~ Удалить", kb::key(Key::Del), CM_RECORD_DELETE),
        StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT),
    ])]
}
```

Enter и двойной щелчок по строке окно превращает в `CM_RECORD_EDIT` само.

Всё. Запустите:

```
cargo run --example mvc_contacts
```

Ins открывает диалог «Новый контакт», собранный по схеме: строки ввода с подписями, переключатели группы, флажок «Избранный». Попробуйте оставить имя пустым или ввести почту без `@` — ОК не закроет диалог, а покажет сообщение. Щелчок по заголовку столбца сортирует таблицу.

Где лежат данные

JSON. Без аргументов пример открывает файл `tvision-mvc-contacts.json` во временном каталоге системы (и заполняет его образцом). Свой файл — `cargo run --example mvc_contacts -- мои.json`. Внутри — массив объектов:

```
[
  {
    "id": 1,
    "name": "Анна Петрова",
    "phone": "+7 900 111-22-33",
    "email": "anna@example.ru",
    "group": "Семья",
    "favorite": true
  }
]
```

Файл переписывается **после каждого изменения** — команды «Сохранить» нет, и при выходе ничего не теряется. `"id"` — номер записи, его ставит хранилище. Файл можно править и в обычном редакторе: объекты без `"id"` получат номер при чтении, а ключи, которых нет в схеме, сохранятся как были.

SQLite. База — один файл, но внутри — настоящая база данных с запросами на SQL. Поддержка SQLite включается **признаком** (feature) `sqlite`, потому что ей нужна системная библиотека `libsqlite3`:

```
cargo run --example mvc_contacts --features sqlite -- контакты.db
```

В своём проекте признак включается в `Cargo.toml`:

```
[dependencies]
tvision-rs = { git = "https://github.com/cat386/tvision-rs", features = ["sqlite"] }
```

> Где взять `libsqlite3`: в macOS она есть всегда; в Ubuntu и Debian — `sudo apt install libsqlite3-dev`; в Fedora — `sudo dnf install sqlite-devel`; в Windows 10 и новее используется встроенная `winsqlite3.dll`.

Таблица `contacts` создаётся сама: столбец `id INTEGER PRIMARY KEY` и по столбцу на поле. Добавили поле в схему — при следующем запуске появится и столбец. Каждое изменение — сразу запрос `INSERT`, `UPDATE` или `DELETE`.

Код программы для JSON и для SQLite **один и тот же** — в этом и смысл модели.

Свои команды над моделью

Контроллер умеет три команды. Остальное вы пишете сами — работая с моделью, а не с таблицей. Например, «Сортировать по имени»:

```
CM_SORT_NAME => {
    self.crud.model.borrow_mut().sort_by(0, true);   // поле 0, по возрастанию
}
```

Таблицу обновлять не нужно: у модели есть счётчик изменений `revision()`, окно замечает, что он вырос, и обновляется само.

Что умеет модель:

Метод	Что делает
`len()`, `get(i)`, `records()`	сколько записей, запись номер `i`, все записи
`add(запись)`	проверить, сохранить, добавить; вернёт номер строки
`update(i, запись)`	заменить запись `i`
`remove(i)`	удалить запись `i`
`sort_by(поле, по_возрастанию)`	сортировать; порядок соблюдается и для новых записей
`reload()`	перечитать файл

Запись (`Record`) — это `id` и значения полей по порядку схемы:

```
let r = Record::new(vec!["Дарья".into(), "+7 999".into(), "".into(), "Друзья".into(), true.into()]);
self.crud.model.borrow_mut().add(r)?;
```

Если запись не прошла проверку или файл не записался, `add` вернёт ошибку с текстом для человека, и модель **ничего не изменит** — данные в памяти и на диске всегда совпадают.

Несколько окон

Можно открыть несколько файлов сразу — каждое окно со своим контроллером. Контроллер берёт команду, только если **его** окно активно, поэтому Del удалит запись именно в том окне, где курсор. В примере F3 открывает ещё один файл (`App` хранит `Vec<CrudController>`).

Попробуйте

1. Добавьте в схему поле `Field::int("year", "~Г~од рождения")` и запустите с тем же файлом. Что стало со старыми записями? (Год у них пустой.) А с базой `.db`?
2. Сделайте телефон обязательным и запретите буквы в нём через `.check(…)`.
3. Добавьте команду «Только избранные»: удалите (`remove`) все записи, у которых `favorite` — «нет». Подсказка: `r.get(4).as_bool()`. Удаляйте с конца, чтобы номера не сдвигались.
4. Напишите тест как в уроке 10: Ins, имя, Enter — и проверьте, что в файле JSON появилась строка с этим именем.

Важно запомнить

- Модель — данные и правила, представление — показ, контроллер — команды. Меняется только модель.
- Поля описываются схемой; диалог, проверка и столбцы получаются из неё.
- `open_storage` выбирает JSON или SQLite по расширению файла; программа от этого не меняется.
- Изменения сохраняются сразу — «Сохранить» не нужно.
- SQLite включается признаком `sqlite` и требует `libsqlite3`.

Следующий урок — рецепты: свои команды над моделью, своё окно, поиск, хранилище CSV, консоль SQL и настройки в JSON. Подробности обо всех типах — в справочнике «Данные: JSON, SQLite, MVC».

Урок 12. Рецепты: данные

Урок 11 показал готовый каркас: схема, модель, окно и контроллер. Здесь — десять рецептов «как сделать, если нужно чуть больше». Каждый взят из готовой программы-примера, которую можно запустить и прочитать целиком.

Пример	Что в нём	Рецепты
`mvc_todo`	список задач: приоритет, срок, «сделано»	1, 2, 3
`mvc_books`	каталог книг со своим окном, поиском и статистикой	4, 5, 6
`mvc_csv`	склад в файле CSV	7
`sqlite_query`	консоль SQL: запрос сверху, результат в таблице	8, 9
`json_settings`	настройки программы в JSON	10

Запуск: `cargo run --example mvc_todo` (и так далее). Примеры с MVC открывают файл во временном каталоге, а с аргументом — ваш: `cargo run --example mvc_books -- книги.json`, или базу: `cargo run --example mvc_books --features sqlite -- книги.db`. `sqlite_query` работает только с признаком `sqlite`.

Рецепт 1. Своя команда меняет запись

Задача. F5 отмечает задачу «сделано» или снимает отметку.

Как. Взять запись из модели, поменять поле и вернуть через `update`. Таблицу трогать не нужно: модель изменилась — окно обновится само.

```
const F_DONE: usize = 3;   // номер поля в схеме — константой, а не «магическим» числом

fn toggle_done(&mut self, app: &mut Program) {
    let Some(i) = self.crud.window(app).and_then(|w| w.current()) else { return };
    let res = {
        let mut m = self.crud.model.borrow_mut();
        let mut r = m.get(i).unwrap().clone();
        r.values[F_DONE] = Data::Bool(!r.get(F_DONE).as_bool());
        m.update(i, r)                                  // проверка + сохранение
    };                                                  // ← borrow_mut закончился здесь
    match res {
        Ok(j) => self.crud.window(app).unwrap().select(j), // курсор — за записью
        Err(e) => { message_box(app, &e, mf::ERROR | mf::OK_BUTTON); }
    }
}
```

Важно. Фигурные скобки вокруг `borrow_mut` — не украшение: модель нужно «отпустить» до `message_box`, потому что окно сообщения перерисовывает экран, а окну для рисования нужен `borrow`. Если запись после изменения переехала (модель отсортирована), `update` вернёт её новый номер `j`.

Рецепт 2. Команда над всеми записями

Задача. «Удалить сделанные».

```
let mut m = self.crud.model.borrow_mut();
for i in (0..m.len()).rev() {                    // с конца!
    if m.get(i).unwrap().get(F_DONE).as_bool() {
        m.remove(i)?;
    }
}
```

Важно. Удаляйте **с конца**: после `remove(i)` записи правее сдвигаются на одну позицию влево, и при проходе с начала каждая вторая подходящая запись была бы пропущена.

Рецепт 3. Сводка в заголовке окна

Задача. «Задачи: осталось 3 из 4» в рамке окна.

Как. В `idle` посчитать по записям и записать в `caption` окна — это заголовок без имени файла; `refresh()` соберёт полный.

```
fn idle(&mut self, app: &mut Program) {
    self.crud.update_commands(app);
    let (left, total) = {
        let m = self.crud.model.borrow();
        (m.records().iter().filter(|r| !r.get(F_DONE).as_bool()).count(), m.len())
    };
    if let Some(w) = self.crud.window(app) {
        let t = format!("Задачи: осталось {} из {}", left, total);
        if w.caption != t {          // не обновлять зря: idle вызывается очень часто
            w.caption = t;
            w.refresh();
        }
    }
}
```

Рецепт 4. Своё окно над моделью

Задача. Под таблицей — строка итогов «Книг: 5, прочитано: 3, на сумму ...», которая обновляется при каждом изменении.

`ModelWindow` такого не умеет, но его «кирпичики» доступны по отдельности: `model_table` строит таблицу над моделью, а `Model::revision()` подсказывает, когда пересчитать итоги.

```
pub struct BooksWindow {
    win: Window,
    table: ViewId,
    summary: ViewId,
    model: ModelRef,
    seen: u64,                       // revision, которую мы уже показали
}

impl BooksWindow {
    pub fn new(bounds: Rect, model: ModelRef) -> Self {
        let mut win = Window::new(bounds, "Книги", 1);
        let v = win.standard_scroll_bar(true);
        let h = win.standard_scroll_bar(false);
        let mut r = win.interior();
        r.b.y -= 1;                                          // последняя строка — для итогов
        let table = win.add(model_table(r, model.clone(), Some(v), Some(h)));
        let i = win.interior();
        let mut s = StaticText::new(Rect::new(i.a.x, i.b.y - 1, i.b.x, i.b.y), "");
        s.vs.grow_mode = gf::GROW_LO_Y | gf::GROW_HI_X | gf::GROW_HI_Y;   // прилипнуть к низу
        let summary = win.add(s);
        let mut w = BooksWindow { win, table, summary, model, seen: u64::MAX };
```

```
        w.sync();
        w
    }

    /// Модель изменилась — обновить итоги и таблицу.
    pub fn sync(&mut self) {
        let rev = self.model.borrow().revision();
        if rev == self.seen { return; }
        self.seen = rev;
        let s = stats(&self.model.borrow());
        let id = self.summary;
        if let Some(t) = self.win.group.find_mut::<StaticText>(id) {
            t.text = format!(" Книг: {}   прочитано: {}   на сумму: {:.2} ₽", s.count, s.read,
s.total_price);
        }
        let id = self.table;
        self.win.group.find_mut::<Table>(id).unwrap().refresh();
    }
}
```

В `handle_event` окна `sync()` вызывается до и после обработки события, а остальное (`draw`, `palette`, `set_state`, Enter → команда) — как в уроке 8.

Про `grow_mode`. `GROW_LO_Y | GROW_HI_Y` — «верхний и нижний края двигаются вместе с нижним краем окна»: строка остаётся внизу, когда окно растягивают. `GROW_HI_X` — растягиваться по ширине.

Рецепт 5. Правка без контроллера

Без `CrudController` команды «Добавить», «Изменить», «Удалить» пишутся сами — это три коротких метода. Диалог по схеме даёт `edit_record`.

```
fn edit(&mut self, app: &mut Program) {
    let Some(i) = self.win(app).and_then(|w| w.current()) else { return };
    // Копия схемы и записи — до диалога: пока он открыт, модель не заимствуем.
    let (schema, rec) = {
        let m = self.model.borrow();
        (m.schema().clone(), m.get(i).unwrap().clone())
    };
    if let Some(new) = edit_record(app, &schema, &rec, "Книга") {
        let r = self.model.borrow_mut().update(i, new);
        match r {
            Ok(j) => self.win(app).unwrap().select(j),
            Err(e) => { message_box(app, &e, mf::ERROR | mf::OK_BUTTON); }
        }
    }
}
```

`add` — то же с `schema.new_record()` и `model.add`, `delete` — вопрос и `model.remove`. Полностью — в `examples/mvc_books.rs`.

Когда писать самому, а когда взять контроллер? Контроллер — когда хватает трёх стандартных команд и окна-таблицы. Сами — когда окно своё (как здесь) или правка нестандартная (например, «изменить» открывает не диалог, а отдельное окно).

Рецепт 6. Поиск по записям

Задача. Ctrl+F — спросить текст и перейти к первой книге, где он есть в названии или авторе; F3 — к следующей.

Поиск — обычная функция над моделью, её легко проверить тестом:

```rust
/// Номер первой записи после `from`, где есть `what`; по кругу.
pub fn find(m: &Model, what: &str, from: usize) -> Option<usize> {
    let what = what.to_lowercase();
    let n = m.len();
    (1..=n).map(|k| (from + k) % n).find(|&i| {
        let r = m.get(i).unwrap();
        r.text(F_TITLE).to_lowercase().contains(&what) ||
r.text(F_AUTHOR).to_lowercase().contains(&what)
    })
}
```

Спросить текст — готовой функцией `input_box(app, заголовок, подпись, начальный_текст, длина)`: она вернёт `Some(текст)` после OK.

Рецепт 7. Своё хранилище

Задача. Хранить записи в файле CSV (его открывают Excel и LibreOffice), а окно, модель и контроллер оставить как есть.

Как. Реализовать трейт `Storage` — пять методов. Удобный способ: держать копию записей в памяти и после каждого изменения переписывать файл целиком.

```rust
pub struct CsvStorage {
    path: PathBuf,
    records: Vec<Record>,
    next_id: i64,
}

impl Storage for CsvStorage {
    fn load(&mut self, schema: &Schema) -> Result<Vec<Record>, String> {
        // прочитать файл (нет файла — Ok(пусто)), найти столбцы по именам
        // из первой строки, каждое значение разобрать `field.parse(текст)`
        // — так проверка та же, что в диалоге
    }
    fn insert(&mut self, schema: &Schema, rec: &mut Record) -> Result<(), String> {
        rec.id = Some(self.next_id);
        self.records.push(rec.clone());
        if let Err(e) = self.save(schema) {      // не записалось —
            self.records.pop();                  // вернуть как было
            rec.id = None;
            return Err(e);
        }
        self.next_id += 1;
        Ok(())
    }
    fn update(&mut self, schema: &Schema, rec: &Record) -> Result<(), String> { /* заменить и save */
}
    fn delete(&mut self, schema: &Schema, id: i64) -> Result<(), String> { /* убрать и save */ }
    fn name(&self) -> String { /* имя файла для заголовка */ }
}

// И всё: дальше как с JSON.
let model = Model::open(schema(), Box::new(CsvStorage::new("склад.csv")))?.into_ref();
```

Правило хорошего хранилища: если сохранить не удалось, верните ошибку и **оставьте всё как было** — модель тогда тоже ничего не изменит, и пользователь увидит сообщение, а не потерянные данные.

Полный код с разбором кавычек CSV — `examples/mvc_csv.rs`.

Рецепт 8. Запросы SQL без MVC

Задача. Выполнить любой запрос, который ввёл пользователь, и показать результат — заранее неизвестно, какие будут столбцы.

`query_rows` возвращает и строки, и имена столбцов. Если столбцов нет, это была команда (`INSERT`, `UPDATE`, `DELETE`), и `changes()` скажет, сколько строк она изменила.

```
use tvision::sqlite::{Connection, Rows};
use tvision::VecTableModel;

pub enum Outcome {
    Table(Rows),
    Changed(usize),
}

pub fn run_sql(db: &Connection, sql: &str) -> Result<Outcome, String> {
    let rows = db.query_rows(sql, &[]).map_err(|e| e.to_string())?;
    if rows.columns.is_empty() {
        Ok(Outcome::Changed(db.changes()))
    } else {
        Ok(Outcome::Table(rows))
    }
}

/// Результат → модель для `Table` (NULL показываем точкой).
fn to_model(rows: &Rows) -> VecTableModel {
    let headers: Vec<&str> = rows.columns.iter().map(String::as_str).collect();
    let cells = rows.rows.iter().map(|r| r.iter().map(|v| if v.is_null() { "·".into() } else {
v.to_text() }).collect()).collect();
    VecTableModel::new(&headers, cells)
}

fn sql_console() {
    let db = Connection::open_in_memory().unwrap();
    db.execute_batch("CREATE TABLE t (a INTEGER, b TEXT); INSERT INTO t VALUES (1, 'x'), (2,
NULL);").unwrap();
    let Ok(Outcome::Table(r)) = run_sql(&db, "SELECT * FROM t") else { panic!() };
    let m = to_model(&r);
    assert_eq!((m.rows.len(), m.rows[1][1].as_str()), (2, "·"));
    assert!(matches!(run_sql(&db, "DELETE FROM t"), Ok(Outcome::Changed(2))));
}
```

Показать новую модель в таблице: `table.set_model(Box::new(модель))`, затем `table.auto_width(200)` — ширина столбцов по содержимому первых 200 строк.

> Здесь запрос пишет сам пользователь — это консоль. В обычной программе данные **никогда** не вклеиваются в текст запроса: только `?` и параметры (см. справочник).

Рецепт 9. Окно из строки ввода и таблицы

Задача. Сверху строка запроса, под ней таблица; Enter в строке — выполнить.

```
let mut win = Window::new(bounds, "SQL", 1);
let i = win.interior();
let mut line = InputLine::new(Rect::new(i.a.x, i.a.y, i.b.x, i.a.y + 1), 1000);
line.vs.grow_mode = gf::GROW_HI_X;                          // растягиваться по ширине
let mut t = Table::new(Rect::new(i.a.x, i.a.y + 1, i.b.x, i.b.y), Box::new(VecTableModel::new(&[],
vec![])), None, None);
t.vs.grow_mode = gf::GROW_HI_X | gf::GROW_HI_Y;
let table = win.add(t);
let input = win.add(line);
win.select(input);                                          // фокус — строке ввода
```

Строка ввода не забирает Enter себе — он возвращается окну. Окно проверяет, что фокус в строке, и посылает команду:

```
fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
    self.win.handle_window_event(ev, ctx);
    let in_input = self.win.group.current_view().map(|v| v.vs().id) == Some(self.input);
    if in_input && matches!(ev, Event::KeyDown(k) if k.key == Key::Enter && k.mods == Mods::NONE) {
        ctx.post_command(CM_RUN, Info::None);
        ev.clear();
    }
}
```

Не привязывайте Enter в строке состояния. Строка состояния видит клавиши раньше всех — даже в диалогах, — и Enter перестал бы нажимать кнопку OK. Для «Выполнить» в строке состояния взята F9.

Рецепт 10. Настройки в JSON

Задача. Хранить настройки программы в файле: прочитать при запуске, записать после изменения. Файл может быть старым (без новых ключей), испорченным руками или содержать чужие ключи — со всем этим нужно обойтись аккуратно.

```
use tvision::json::{parse_json, Json};

#[derive(Debug, PartialEq)]
pub struct Settings {
    pub user: String,
    pub tab_size: i64,
    pub recent: Vec<String>,
}

impl Settings {
    /// Нет ключа или он не того вида — значение по умолчанию.
    pub fn from_json(j: &Json) -> Settings {
        Settings {
            user: j.get("user").and_then(Json::as_str).unwrap_or("").to_string(),
            tab_size: j.get("tab_size").and_then(Json::as_i64).filter(|n|
(1..=16).contains(n)).unwrap_or(4),
            recent: j
                .get("recent")
                .and_then(Json::as_array)
                .map(|a| a.iter().filter_map(|x| x.as_str().map(String::from)).collect())
                .unwrap_or_default(),
        }
    }

    /// Записать свои поля в объект; чужие ключи остаются.
    pub fn write_json(&self, into: &mut Json) {
        into.set("user", Json::from_str(&self.user));
        into.set("tab_size", Json::from_i64(self.tab_size));
        into.set("recent", Json::Arr(self.recent.iter().map(|s| Json::from_str(s)).collect()));
    }
}

fn settings_roundtrip() {
    let mut raw = parse_json(r#"{"plugin": {"on": true}, "tab_size": 99}"#).unwrap();
    let mut s = Settings::from_json(&raw);
    assert_eq!(s, Settings { user: String::new(), tab_size: 4, recent: vec![] });
    s.recent.push("a.txt".into());
    s.write_json(&mut raw);
    let text = raw.pretty(); // → в файл
    assert!(text.contains("\"plugin\"") && text.contains("\"a.txt\""));
}
```

Три правила, которые здесь соблюдены:

1. **Отсутствующий или неверный ключ — значение по умолчанию**, а не ошибка: старый файл настроек продолжит работать.
2. **Чужие ключи не теряются**: мы храним разобранный `Json` целиком и меняем в нём только свои поля (`set`), а не собираем объект заново.
3. **Испорченный файл — ошибка при запуске** (с номером строки из `JsonError`), а не молчаливая перезапись настройками по умолчанию.

Диалог настроек и список недавних файлов — в `examples/json_settings.rs`.

Попробуйте

1. В `mvc_todo` добавьте команду «Просроченные»: перейти к первой несделанной задаче со сроком раньше сегодняшнего. Дата сегодня — `tvision::platform::time::now()`; сроки — строки `ГГГГ-ММ-ДД`, их можно сравнивать как строки.
2. В `mvc_books` добавьте в строку итогов среднюю цену.
3. Сделайте `TsvStorage` (значения через табуляцию) на основе `CsvStorage` и подключите его в `open_model` по расширению `.tsv`.
4. В `sqlite_query` сохраняйте историю запросов в JSON-файл через рецепт 10 и подставляйте предыдущий запрос по Ctrl+↑.

Справочник по всем типам — «Данные: JSON, SQLite, MVC». Следующий урок — раскладки: окна без единой координаты, как в Qt.

Урок 13. Раскладки: без координат

В уроке 4 каждый элемент диалога ставился прямоугольником: `Rect::new(x, y, x + ширина, y + высота)`. Это точно, но утомительно: добавили поле — сдвигайте всё, что ниже; поменяли подпись — пересчитывайте отступы; растянули окно — подбирайте `grow_mode`.

В Qt для этого есть **раскладки** (layouts): вы говорите «эти элементы — в строку, эти — в столбец, а здесь — пружина», а координаты библиотека считает сама. В `tvision-rs` то же самое — модуль `tvision::layout`.

Программа урока: `cargo run --example layout_demo`.

Четыре вида раскладок

Раскладка	Как в Qt	Что делает
`HBox`	`QHBoxLayout`	элементы в строку, слева направо
`VBox`	`QVBoxLayout`	элементы в столбец, сверху вниз
`Grid`	`QGridLayout`	сетка: строки и столбцы, элемент может занимать несколько клеток
`Form`	`QFormLayout`	строки «подпись — поле»: подписи в столбец по самой длинной, поля — дальше

И распорки, как `QSpacerItem`:

Что	Как в Qt	Что делает
`.stretch(1)`	`addStretch(1)`	**пружина**: забирает всё лишнее место
`.space(2)`	`addSpacing(2)`	пустое место ровно в 2 клетки
`Spacer::horizontal()`, `Spacer::vertical()`, `Spacer::fixed(w, h)`	`QSpacerItem`	распорка для сетки и особых случаев

Раскладки вкладываются друг в друга: в столбце может быть строка, в строке — столбец, и так сколько угодно.

Первая раскладка: диалог входа

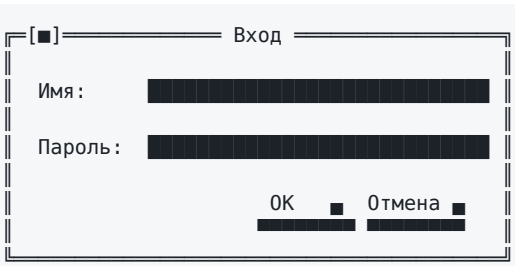
```rust
use tvision::layout::*;
use tvision::*;

fn login() -> Dialog {
    let mut d = Dialog::centered(30, 10, "Вход");          // размер — пока любой

    // Элементы — с пустым прямоугольником: место им найдёт раскладка.
    let user = d.add(InputLine::new(Rect::default(), 30));
    let pass = d.add(InputLine::new(Rect::default(), 30));
    let lu = d.add(Label::new(Rect::default(), "~И~мя:", Some(user)));
    let lp = d.add(Label::new(Rect::default(), "~П~ароль:", Some(pass)));
    let ok = d.add(Button::new(Rect::default(), "О~К~", cm::OK, bf::DEFAULT));
    let cancel = d.add(Button::new(Rect::default(), "Отмена", cm::CANCEL, bf::NORMAL));

    d.set_layout(
        VBox::new()                                        // всё — в столбец:
            .margins(2, 1, 2, 0)                           //   поля: слева 2, сверху 1, справа 2
            .add(Form::new().row(lu, user).row(lp, pass)) //   форма из двух строк
            .space(1)                                      //   пустая строка
            .add(HBox::new()                               //   и ряд кнопок:
                .stretch(1)                                //     пружина слева — кнопки вправо
                .add(ok)
                .add(cancel)),
    );
    d.adjust_size();                                       // размер диалога — по содержимому
    d
}
```

Получится:



Что здесь важно:

- **`Rect::default()`** — пустой прямоугольник. Положение и размер элементам даст раскладка, поэтому считать координаты не нужно.
- **`d.set_layout(…)`** — поставить раскладку. Она сразу расставит элементы и потом будет делать это **при каждом изменении размера** окна.
- **`d.adjust_size()`** — подогнать размер диалога под содержимое (как `adjustSize()` в Qt). Поэтому в `Dialog::centered(30, 10, …)` размер можно написать любой.
- Добавить ещё одно поле — значит дописать `.row(подпись, поле)`. Всё остальное сдвинется само.

Откуда берутся размеры

Каждый элемент знает свой **желаемый размер** — как `sizeHint()` в Qt:

Элемент	Желаемый размер
`Button`	по надписи + 4, не меньше 10; высота 2
`Label`	по тексту; высота 1
`StaticText`	по самой длинной строке × число строк

Элемент	Желаемый размер
`InputLine`	по длине (не больше 30); высота 1
флажки, переключатели	по самому длинному пункту × число пунктов
таблица, список, редактор, дерево	размер, с которым создан (или 10 × 3)

…и как он относится к лишнему месту — **политику** (`Policy`, как `QSizePolicy`):

Политика	Что значит	У кого по умолчанию
`Fixed`	ровно желаемый размер	кнопка (обе стороны), высота строки ввода и подписи
`Preferred`	желаемый лучше всего, но может расти	ширина подписи, флажков
`Expanding`	хочет как можно больше	ширина строки ввода; таблица, список, редактор — в обе стороны
`Minimum`, `Maximum`, `Ignored`	не меньше / не больше желаемого / размер не важен	—

Как делится лишнее место в строке:

1. Если у кого-то задан **коэффициент растяжения** (`stretch`) — место делится по коэффициентам. Пружина `.stretch(1)` — это пустое место с коэффициентом 1.
2. Иначе — тем, у кого `Expanding` (строка ввода, таблица).
3. Иначе — всем, кто может расти.

Если места **не хватает**, сжимаются те, кому можно, — но не меньше их минимума. Окно с раскладкой нельзя сделать меньше её минимума: рамка просто не потянется.

Пропорции: окно «Почта»

Папки слева — четверть ширины, письма и текст справа — три четверти; кнопки внизу справа:

```
w.set_layout(
    VBox::new()
        .add(HBox::new()
            .add(item(folders).stretch(1))                // 1 часть ширины
            .add(item(VBox::new()
                    .spacing(1)
                    .add(item(letters).stretch(2))        // 2/3 высоты
                    .add(item(text).expand().stretch(1))) // 1/3 высоты
                .stretch(3)))                             // 3 части ширины
        .space(1)
        .add(HBox::new().stretch(1).add(reply).add(delete)),
);
```

`item(…)` заворачивает элемент, чтобы что-то уточнить:

Уточнение	Что делает
`.stretch(n)`	коэффициент растяжения
`.expand()`	тянуться во все стороны (политика `Expanding`)
`.fixed(w, h)`, `.fixed_width(w)`, `.fixed_height(h)`	ровно такой размер
`.min(w, h)`, `.max(w, h)`, `.min_width(w)`…	пределы
`.hint(w, h)`	свой желаемый размер

Уточнение	Что делает
`.policy(по_ширине, по_высоте)`	своя политика
`.align_h(Alignment::…)`, `.align_v(…)`, `.center()`	не растягивать, а поставить в начало / по центру / в конец

Растяните окно мышью за правый нижний угол (или Ctrl+F5 и стрелки с Shift) — пропорции сохраняются, кнопки остаются в углу.

Сетка: калькулятор

```
let mut grid = Grid::new()
    .span(0, 0, 1, 3, display)          // табло: строка 0, столбец 0, на 1 строку и 3 столбца
    .cell(0, 3, clear);                 // «C» — в четвёртом столбце
for (i, b) in buttons.iter().enumerate() {
    grid = grid.cell(1 + i / 4, i % 4, item(*b).expand());  // кнопки заполняют клетки
}
for c in 0..4 { grid = grid.col_stretch(c, 1); }    // столбцы поровну
for r in 1..5 { grid = grid.row_stretch(r, 1); }    // строки с кнопками поровну
w.set_layout(grid.margins(1, 0, 1, 0));
```

- `.cell(строка, столбец, элемент)` — в одну клетку;
- `.span(строка, столбец, строк, столбцов, элемент)` — на несколько;
- `.col_stretch(столбец, n)`, `.row_stretch(строка, n)` — как делить лишнее место между столбцами и строками.

Кнопкам задано `.expand()`: обычно кнопка не меняет размер (`Fixed`), а здесь пусть растёт вместе с окном.

Промежутки и поля

Раскладка	Промежуток по умолчанию
`HBox`	1 клетка между элементами
`VBox`	0 — строки вплотную
`Grid`	1 между столбцами, 0 между строками
`Form`	1 между подписью и полем, **1 между строками** (как в диалогах Turbo Vision)

Поменять: `.spacing(n)` (а у сетки отдельно `.hspacing(n)` и `.vspacing(n)`). Поля вокруг: `.margins(слева, сверху, справа, снизу)` или `.margin(n)`. Рамку окна раскладка учитывает сама.

Рядом с распорками промежуток не ставится — как в Qt. Поэтому `.space(2)` даёт ровно 2 клетки.

Скрыть и показать

Скрытый элемент места **не занимает** — остальные сдвигаются:

```
if let Some(v) = d.win.group.view_mut(details) {
    v.vs_mut().state &= !sf::VISIBLE;              // скрыть
}
d.win.group.apply_layout();                        // пересчитать
d.adjust_size();                                   // и подогнать окно, если нужно
```

`apply_layout()` нужен и после других изменений, от которых меняется желаемый размер: например, сменили надпись кнопки.

Раскладка и `grow_mode` вместе

Раскладка управляет только теми элементами, которые в неё добавлены. Остальные (например, полосы прокрутки на рамке окна) двигаются по своему `grow_mode`, как раньше. Раскладку можно поставить и в любую группу — `group.set_layout(…)`; тогда отступа под рамку нет.

Попробуйте

1. В диалог входа добавьте поле «Сервер» с переключателями: `Cluster::radio_buttons(Rect::default(), &["~Р~абочий", "~Д~омашний"])` и строку `.row(подпись, переключатели)`. Ничего пересчитывать не придётся.
2. Поставьте кнопки по центру: пружина и слева, и справа — `HBox::new().stretch(1).add(ok).add(cancel).stretch(1)`.
3. В «Почте» поменяйте пропорции на 1:1 и растяните окно.
4. Сделайте в калькуляторе кнопку «0» шириной в две клетки: `.span(4, 0, 1, 2, ноль)`.

Важно запомнить

- Элементы создаются с `Rect::default()`, место им даёт раскладка.
- `HBox` — строка, `VBox` — столбец, `Grid` — сетка, `Form` — «подпись — поле»; вкладываются друг в друга.
- `.stretch(n)` — пружина, `.space(n)` — ровно `n` клеток.
- `item(элемент)` — уточнить: `.stretch()`, `.expand()`, `.fixed()`, `.align_h()`…
- `set_layout` — раскладка пересчитывается при каждом изменении размера окна; `adjust_size` — размер окна по содержимому.

Подробно о каждом типе — в справочнике «Раскладки».

ЧАСТЬ 2

Справочник

Каждый класс библиотеки: что это, когда нужен, как создать, что важно знать, главные методы и пример.

Справочник: точки, прямоугольники, цвета, текст

Самые базовые типы: из них собирается всё остальное. Если вы прошли уроки, многое здесь уже знакомо — справочник нужен, чтобы быстро уточнить подробности.

На этой странице: `Point` · `Rect` · `Attr` и цвета · `ColorMode` · палитры · `Cell` · `DrawBuffer` · `ScreenBuffer` · функции текста

`Point` — точка

Что это. Два числа: `x` (столбец) и `y` (строка). Счёт с нуля, `y` растёт вниз. Тем же типом задают **размер**: `x` — ширина, `y` — высота.

Аналог в Turbo Vision: `TPoint`.

```rust
use tvision::*;

fn points() {
    let a = Point::new(10, 5); // столбец 10, строка 5
    let b = Point::new(2, 1);
    assert_eq!(a + b, Point::new(12, 6)); // сложение по координатам
    assert_eq!(a - b, Point::new(8, 4));
    let size = Point::new(40, 10); // как размер: ширина 40, высота 10
    assert_eq!(size.x * size.y, 400);
}
```

Что важно знать.

- Координаты — `i32` и могут быть отрицательными (окно, утащенное за левый край экрана).
- Поддерживаются `+`, `-`, `+=`, `-=`.

`Rect` — прямоугольник

Что это. Прямоугольник на экране: левый верхний угол `a` и правый нижний `b`. **Правый и нижний край в прямоугольник не входят**, поэтому ширина — просто `b.x - a.x`.

Когда нужен. Почти везде: им задают положение и размер окна, кнопки, поля — любого представления.

Аналог: `TRect`.

```rust
use tvision::*;

fn rects() {
    let r = Rect::new(2, 1, 12, 4); // столбцы 2..11, строки 1..3
    assert_eq!((r.width(), r.height()), (10, 3));
    assert!(r.contains(Point::new(11, 3)));
    assert!(!r.contains(Point::new(12, 3))); // правый край — уже снаружи

    let inner = r.grown(-1, -1); // сжать на клетку с каждой стороны
    assert_eq!(inner, Rect::new(3, 2, 11, 3));
    let moved = r.moved(5, 0); // сдвинуть на 5 вправо
    assert_eq!(moved, Rect::new(7, 1, 17, 4));
    assert_eq!(r.intersect(&moved), Rect::new(7, 1, 12, 4)); // общая часть
    assert_eq!(r.union(&moved), Rect::new(2, 1, 17, 4)); // охватывающий
}
```

Как создать.

- `Rect::new(x1, y1, x2, y2)` — по углам. Удобная запись: `Rect::new(x, y, x + ширина, y + высота)`.
- `Rect::from_origin_size(угол, размер)` — по углу и размеру.

Главное.

Метод	Что делает
`width()`, `height()`, `size()`	ширина, высота, размер (`Point`)
`contains(точка)`	попадает ли точка внутрь
`is_empty()`	пустой ли (ширина или высота ≤ 0)
`moved(dx, dy)`	сдвинутая копия
`grown(dx, dy)`	шире на `dx` и выше на `dy` **с каждой стороны**; отрицательные — уже
`intersect(&r)`	общая часть двух прямоугольников
`union(&r)`	наименьший прямоугольник, охватывающий оба
`a`, `b`	углы (их можно менять напрямую)

`moved` и `grown` возвращают новый прямоугольник, исходный не меняется.

Как прямоугольники складываются в раскладку окон — урок 4 и координаты подробно.

`Attr` — цвет

Что это. Цвет одной клетки: цвет символа и цвет фона в одном числе (`u32`). Младший байт устроен как в DOS: младшие 4 бита — цвет текста, старшие 4 — фон. `0x1E` — жёлтый (`E`) на синем (`1`).

16 основных цветов:

№	Цвет	№	Цвет
0	чёрный	8	тёмно-серый
1	синий	9	ярко-синий
2	зелёный	10	ярко-зелёный
3	бирюзовый	11	ярко-бирюзовый
4	красный	12	ярко-красный
5	малиновый	13	ярко-малиновый

№	Цвет	№	Цвет
6	коричневый	14	жёлтый
7	светло-серый	15	белый

Кроме них можно использовать **256 цветов** современных терминалов (палитра xterm) — они хранятся в старших битах того же числа.

Как получить цвет. Функции из модуля `tvision::color`:

```
use tvision::color::*;

fn colors() {
    let a = attr(14, 1); // жёлтый на синем, из 16 цветов
    assert_eq!(a, 0x1E);
    assert_eq!((fore(a), back(a)), (14, 1)); // разобрать обратно
    assert_eq!(with_back(a, 4), 0x4E); // тот же текст на красном

    let b = attr256(208, 236); // оранжевый на тёмно-сером, из 256 цветов
    assert_eq!((fg256(b), bg256(b)), (208, 236));

    let c = rgb256(0xFFD700, 0x303030); // ближайшие к цветам RGB
    assert_eq!(fg256(c), 220);
}
```

Чаще всего нужны:

Функция	Что делает
`attr(текст, фон)`	цвет из 16 основных (номера 0–15)
`attr256(текст, фон)`	цвет из 256 (номера 0–255)
`rgb256(0xRRGGBB, 0xRRGGBB)`	ближайшие из 256 к цветам RGB
`with_fore(a, c)`, `with_back(a, c)`	заменить цвет текста / фона
`fore(a)`, `back(a)`	номер цвета текста / фона (0–15)

Реже нужны: `fg256`, `bg256`, `is_fg256`, `is_bg256` (номера из 256); `with_fg256`, `with_bg256`; `bios`, `to_bios` (свести к 16 цветам); `xterm_rgb`, `rgb_to_xterm`, `xterm_to_bios`, `bios_to_ansi`, `ansi_to_bios` (переводы между системами цветов); `fg_rgb`, `bg_rgb` (цвет в RGB; второй аргумент — `VGA_RGB` или `GRUVBOX_RGB`); `contrast256` (чёрный или белый — что читается на фоне); `shade_bg` (фон «чуть темнее» — для полосатых таблиц); `remap` (перекрасить палитру). Константы: `FG_256`, `BG_256`, `ERROR_ATTR`, `SHADOW_ATTR`, `SHADOW_ATTR_256`, `VGA_RGB`, `GRUVBOX_RGB`.

Что важно знать. Обычно цвета **не задают числами**, а берут из палитры по номеру (`dc.color(1)` в методе `draw`) — так программа выглядит единообразно и её можно перекрасить. Свои числа нужны для особых случаев: красная надпись «Ошибка», раскраска строк таблицы.

`AttrPair` — пара цветов

Что это. Два цвета в одном числе (`u64`): обычный и **выделенный**. Нужен для текстов с горячей буквой (`"~Ф~айл"`): буква между тильдами рисуется вторым цветом. Создать — `color::pair(обычный, выделенный)`; разобрать — `lo(p)`, `hi(p)`, `unpair(p)`.

`ColorMode` — сколько цветов у терминала

Что это. `ColorMode::Ansi16` (16 цветов) или `ColorMode::Xterm256` (256 цветов). Библиотека определяет режим сама по переменным окружения терминала; если терминал умеет только 16, цвета из 256 заменяются ближайшими.

```rust
use tvision::color::{color_mode, set_color_mode, ColorMode};

fn force_16() {
    set_color_mode(Some(ColorMode::Ansi16)); // всегда 16 цветов
    assert_eq!(color_mode(), ColorMode::Ansi16);
    set_color_mode(None); // снова определять самому
}
```

Функции: `color_mode()` — текущий режим, `set_color_mode(Some(…))` — задать, `set_color_mode(None)` — определять самому, `detect_color_mode()` — определить заново, `color_mode_from(…)` — то же по заданным строкам (для тестов). Без изменения программы режим задаёт переменная окружения `TVISION_COLORS=16` или `TVISION_COLORS=256`.

Палитры

Что это. Палитра — список **номеров цветов**. У каждого вида представления своя палитра: кнопка говорит «мне цвет номер 1», её палитра переводит это в номер цвета диалога, палитра диалога — в номер цвета программы, и только **палитра программы** хранит настоящие цвета.

Зачем так: одна и та же кнопка правильно выглядит и в сером диалоге, и в синем окне, а чтобы перекрасить всю программу, достаточно поменять одну палитру.

Где лежат. Модуль `tvision::color::palettes`:

Константа	Что
`GRUVBOX`	палитра программы по умолчанию — 256 цветов, тема gruvbox
`APP_COLOR`	классические цвета Borland (синий рабочий стол, серые диалоги)
`APP_BLACK_WHITE`, `APP_MONOCHROME`	чёрно-белая и монохромная
`BLUE_WINDOW`, `CYAN_WINDOW`, `GRAY_WINDOW`	палитры окон
`GRAY_DIALOG`, `BLUE_DIALOG`, `CYAN_DIALOG`	палитры диалогов
`BUTTON`, `INPUT_LINE`, `STATIC_TEXT`, `LABEL`, `CLUSTER`, `LIST_VIEWER`, `SCROLL_BAR`, `SCROLLER`, `EDITOR`, `MEMO`, …	палитры элементов — их удобно брать для своих представлений

```rust
use tvision::*;

fn classic(app: &mut Program) {
    // Классические цвета Turbo Vision вместо gruvbox.
    app.palette = color::palettes::APP_COLOR.to_vec();
}
```

Палитра программы — обычный `Vec` в поле `app.palette`: её можно менять на ходу (это делает диалог цветов `ColorDialog`) и сохранять в файл.

`Cell` — клетка экрана

Что это. Символ и его цвет: поля `ch: char` и `attr: Attr`. `Cell::new(символ, цвет)`. У иероглифов и эмодзи, которые занимают две клетки, вторая клетка хранит `'\0'` (`is_wide_tail()` — `true`).

`DrawBuffer` — строка для рисования

Что это. Одна строка клеток, которую сначала заполняют, а потом целиком выводят: `dc.write_line(x, y, ширина, высота, &буфер)`. Удобна, когда в строке несколько кусков разного цвета.

Аналог: `TDrawBuffer`.

```rust
use tvision::*;

/// «[ OK ]  Сохранить» с выделенной буквой «С».
fn line(width: usize) -> DrawBuffer {
    let mut b = DrawBuffer::new(width); // пробелы
    b.move_char(0, ' ', 0x70, width); // залить серым
    let x = b.move_str(0, "[ OK ]", 0x2F); // вернёт ширину: 6
    b.move_cstr(x + 2, "~С~охранить", color::pair(0x70, 0x7E)); // «С» — вторым цветом
    b
}
```

Метод	Что делает
`DrawBuffer::new(ширина)`	строка из пробелов
`move_char(x, символ, цвет, сколько)`	повторить символ
`move_str(x, текст, цвет)`	написать текст; возвращает его **ширину**
`move_str_limited(x, текст, цвет, макс)`	не шире `макс` клеток
`move_cstr(x, текст, пара)`	текст с горячей буквой в тильдах
`put_char(x, c)`, `put_attribute(x, цвет)`	поменять только символ / только цвет
`move_buf(x, &клетки, цвет)`	скопировать готовые клетки

Цвет `0` в `move_str` и `move_char` значит «не менять цвет»: так пишут текст поверх уже закрашенного фона.

`ScreenBuffer` — весь экран

Что это. Все клетки экрана. Программа рисует кадр в него, а потом выводит в терминал только то, что изменилось. В своём коде нужен в основном в тестах:

```rust
use tvision::*;

fn check_frame(app: &Program) {
    let s: &ScreenBuffer = app.screen();
    let top = s.line_text(0); // первая строка экрана текстом
    let cell = s.get(2, 0); // клетка (x = 2, y = 0)
    let _ = (top, cell.attr, s.to_text(), s.width, s.height);
}
```

Методы: `ScreenBuffer::new(ш, в)`, `get(x, y)`, `get_mut(x, y)`, `clear(цвет)`, `line_text(y)`, `to_text()`; поля `width`, `height`, `cells`. Удобнее всего — `app.screen_text()`: весь экран одной строкой.

Функции текста

Модуль `tvision::text`. Нужны, чтобы считать **ширину на экране** — она не всегда равна числу символов: иероглифы занимают две клетки, а тильды в `"~Ф~айл"` не рисуются вовсе.

```
use tvision::text::*;

fn texts() {
    assert_eq!(str_width("Привет"), 6);
    assert_eq!(str_width("漢字"), 4); // по две клетки
    assert_eq!(cstr_width("~Ф~айл"), 4); // тильды не считаются
    assert_eq!(strip_tildes("~Ф~айл"), "Файл");
    assert_eq!(hotkey("~Ф~айл"), Some('Ф'));
    assert!(same_hotkey('a', 'Ф')); // Alt+A в латинской раскладке = Alt+Ф
    assert_eq!(truncate_width("Привет, мир", 6), "Привет");
    assert_eq!(wrap_text("один два три", 8), vec!["один два", "три"]);
}
```

Функция	Что делает
`str_width(s)`	ширина строки в клетках
`char_width(c)`	ширина символа: 0, 1 или 2
`cstr_width(s)`	ширина без тильд
`strip_tildes(s)`	убрать тильды
`hotkey(s)`	горячая буква (в верхнем регистре)
`same_hotkey(нажата, горячая)`	совпадает ли буква с учётом раскладки (й↔q, ф↔a)
`truncate_width(s, макс)`	начало строки не шире `макс` клеток
`wrap_text(s, ширина)`	разбить на строки по словам
`to_upper(c)`	заглавная буква (и кириллица)

> Ширину на экране считайте `str_width`, а не `s.len()` (это байты: у «Привет» их 12) и не `s.chars().count()`.

Справочник: события, клавиши, команды

Программа с интерфейсом всё время ждёт, что сделает пользователь. Каждое его действие — нажатие клавиши, щелчок, выбор пункта меню — превращается в **событие**, и программа передаёт его тому, кому оно предназначено.

На этой странице: `Event` · `Key` · `Mods` · `KeyEvent` · `kb` · `MouseEvent` · `Message` и `Info` · `cm` · путь события

`Event` — событие

Что это. Перечисление: что именно произошло.

Вариант	Что произошло
`KeyDown(KeyEvent)`	нажата клавиша
`MouseDown(m)`	нажата кнопка мыши
`MouseUp(m)`	кнопка отпущена
`MouseMove(m)`	мышь сдвинулась
`MouseAuto(m)`	кнопка удерживается (событие повторяется — для прокрутки)
`MouseWheel(m)`	повёрнуто колесо
`Command(Message)`	команда: из меню, от кнопки, от строки состояния
`Broadcast(Message)`	сообщение «всем»: одно представление сообщает соседям, что что-то изменилось
`Nothing`	пусто: событие уже обработано

Аналог: `TEvent`.

Как обрабатывать. В методе `handle_event` своего представления или обработчика приложения:

```
use tvision::*;

/// Возвращает, что сделали, — для проверки.
fn handle(ev: &mut Event) -> &'static str {
    // Клавиша?
    if let Some(k) = ev.key() {
        if k.is(kb::F2) {
            ev.clear(); // обработано — дальше не пойдёт
            return "F2";
        }
    }
    // Команда?
    if ev.command() == Some(cm::OK) {
        ev.clear();
        return "OK";
    }
    "не наше"
}

fn check() {
    let mut e = Event::KeyDown(kb::F2);
    assert_eq!(handle(&mut e), "F2");
    assert!(e.is_nothing()); // после clear событие пустое
    let mut c = Event::command_event(cm::OK);
    assert_eq!(handle(&mut c), "OK");
}
```

Метод	Что делает
`clear()`	«я обработал»: событие становится пустым и дальше не идёт
`is_nothing()`	обработано ли уже
`key()`	клавиша, если это нажатие клавиши
`command()`	номер команды, если это команда
`broadcast()`	сообщение, если это `Broadcast`
`mouse()`, `mouse_mut()`	данные мыши, если это событие мыши
`is_mouse()`	событие ли мыши
`Event::command_event(cmd)`	создать команду (например, для `app.put_event`)

Что важно знать. Своё событие — `ev.clear()`. Чужое — не трогайте: просто выйдите из функции, и его получит кто-то другой.

`Key` — клавиша

Что это. Какая клавиша нажата (без Shift/Ctrl/Alt):

Вариант	Клавиша
`Char(c)`	буква, цифра, знак, пробел — в той раскладке, что включена
`Enter`, `Esc`, `Tab`, `Backspace`	
`BackTab`	Shift+Tab (некоторые терминалы присылают его так)
`Up`, `Down`, `Left`, `Right`	стрелки
`Home`, `End`, `PgUp`, `PgDn`, `Ins`, `Del`	
`F(n)`	F1–F12: `Key::F(5)`

`Mods` — Shift, Ctrl, Alt

Что это. Какие клавиши-модификаторы зажаты: поля `shift`, `ctrl`, `alt`. Готовые значения: `Mods::NONE`, `Mods::SHIFT`, `Mods::CTRL`, `Mods::ALT`, `Mods::CTRL_SHIFT`.

`KeyEvent` — нажатие клавиши

Что это. Клавиша и модификаторы: поля `key` и `mods`.

```rust
use tvision::*;

fn keys() {
    let k = KeyEvent::new(Key::Char('x'), Mods::ALT); // Alt+X
    assert!(k.is(kb::ALT_X)); // большие и маленькие буквы не различаются
    assert_eq!(k.name(), "Alt+X"); // для подписей
    assert_eq!(k.text_char(), None); // с Alt — это не ввод текста

    let t = KeyEvent::new(Key::Char('ж'), Mods::NONE);
    assert_eq!(t.text_char(), Some('ж')); // обычная буква — ввод текста
}
```

Метод	Что делает
`KeyEvent::new(клавиша, модификаторы)`	создать

Метод	Что делает
`is(другое)`	то же ли это сочетание
`text_char()`	символ, если это обычный ввод текста (без Ctrl и Alt)
`alt_char()`	буква для Alt+буква
`name()`	название: `"Alt+X"`, `"Ctrl+F5"`, `"F10"`

`kb` — готовые клавиши

Константы и функции для частых сочетаний:

Что	Как записать
клавиша без модификаторов	`kb::key(Key::Ins)`
Alt+буква, Ctrl+буква	`kb::alt('X')`, `kb::ctrl('N')`
F1–F12	`kb::f(9)`; готовые `kb::F1`, `F2`, `F3`, `F5`, `F6`, `F10`
Alt+F, Ctrl+F, Shift+F	`kb::alt_f(3)`, `kb::ctrl_f(5)`, `kb::shift_f(6)`
частые	`kb::ENTER`, `kb::ESC`, `kb::TAB`, `kb::SHIFT_TAB`, `kb::ALT_X`, `kb::ALT_F3`, `kb::CTRL_F5`, `kb::SHIFT_F6`

> Некоторые Ctrl+буква терминал присылает как другие клавиши: Ctrl+J и Ctrl+M — это Enter, Ctrl+I — Tab, Ctrl+H — Backspace. Не назначайте на них команды — они не будут работать.

`MouseEvent` — мышь

Что это. Где мышь и что с кнопками.

Поле	Что
`pos`	где курсор — **в координатах владельца** представления; свои координаты — `self.vs.make_local(m.pos)`
`buttons`	какие кнопки нажаты: `event::MB_LEFT`, `MB_RIGHT`, `MB_MIDDLE` (биты)
`double`	двойной щелчок
`mods`	зажаты ли Shift, Ctrl, Alt
`wheel`	колесо: −1 — вверх, +1 — вниз

```rust
use tvision::event::MB_RIGHT;
use tvision::*;

/// Где щёлкнули правой кнопкой — в своих координатах.
fn right_click(vs: &ViewState, ev: &Event) -> Option<Point> {
    match ev {
        Event::MouseDown(m) if m.buttons & MB_RIGHT != 0 => Some(vs.make_local(m.pos)),
        _ => None,
    }
}
```

Константы `MB_…` лежат в модуле `tvision::event` — их подключают отдельно: `use tvision::event::MB_RIGHT;`.

`Message` и `Info` — команда с данными

`Message` — что лежит внутри `Command` и `Broadcast`: номер команды `command` и данные `info`.

`Info` — данные команды. Обычно их нет (`Info::None`), но иногда нужно сказать «кто послал» или передать число:

Вариант	Когда встречается
`None`	большинство команд
`Id(ViewId)`	кто послал: «в этом списке выбрали строку»
`Int(i64)`, `Str(String)`	свои данные
`Text { id, text }`, `Indicator { … }`, `Progress { id, value }`, `Scroll { … }`	служебные сообщения элементов друг другу

```rust
use tvision::*;

/// Какой список сообщил «выбрана строка».
fn selected_list(ev: &Event) -> Option<ViewId> {
    match ev.broadcast() {
        Some(Message { command: cm::LIST_ITEM_SELECTED, info: Info::Id(id) }) => Some(*id),
        _ => None,
    }
}
```

`cm` — стандартные команды

Модуль `cm` — команды библиотеки. Их не нужно обрабатывать самим: их понимают окна, диалоги, рабочий стол, редактор. Достаточно поставить в меню.

Что	Команды
программа и окна	`QUIT` (выход), `MENU` (открыть меню), `CLOSE`, `ZOOM` (распахнуть), `RESIZE`, `NEXT`, `PREV`, `TILE` (мозаика), `CASCADE` (каскад), `HELP`
ответы диалога	`OK`, `CANCEL`, `YES`, `NO` — закрывают диалог
правка (для редактора и полей)	`CUT`, `COPY`, `PASTE`, `UNDO`, `REDO`, `CLEAR`, `FIND`, `REPLACE`, `SEARCH_AGAIN`, `SELECT_ALL`
файлы (для редактора)	`NEW`, `OPEN`, `SAVE`, `SAVE_AS`
сообщения элементов	`LIST_ITEM_SELECTED`, `TABLE_CURSOR_MOVED`, `TREE_CURSOR_MOVED`, `SCROLL_BAR_CHANGED`, `RECEIVED_FOCUS` …

Все команды с номерами — глава 16.

Свои команды нумеруйте от **100**. Если команду нужно уметь запрещать (делать пункт меню серым), её номер должен быть не больше **255**.

Куда попадает событие

Это полезно знать, когда событие «не доходит».

1. **Клавиши и команды** идут по цепочке **фокуса**: программа → рабочий стол → активное окно → элемент в фокусе. Кто первый обработал (`clear`), тот и забрал. * Строка состояния и строка меню

видят клавиши **раньше** всех — поэтому их клавиши работают везде, даже в диалогах. * Кнопка «по умолчанию» видит Enter **после** элемента в фокусе: если поле Enter не забрало, нажмётся кнопка.

2. **Мышь** идёт тому, кто под курсором (самому верхнему).
3. **Сообщения** (`Broadcast`) получают все **соседи** отправителя — другие дети его окна или диалога.
4. Если **команду** никто не обработал, она приходит в `AppHandler::handle_event` — в ваш обработчик приложения.



Справочник: представления

Представление (view) — всё, что видно на экране: кнопка, поле, окно, рабочий стол. Здесь — как они устроены: трейт `View`, общие поля `ViewState`, связь с программой через `Ctx`, рисование через `DrawCtx` и представления с детьми — `Group`.

Если вы ещё не писали своих представлений, начните с урока 7 и урока 8.

На этой странице: `View` · `ViewState` · `ViewId` · флаги · `Value` · `Ctx` · `Request`, `PlaceFn` · `CommandSet` · `Phase` · `DrawCtx` · `Group` · макросы · вспомогательные функции · координаты подробно

`View`

Что это. Трейт, который реализует каждое представление. Он отвечает на два главных вопроса: **как себя нарисовать** (`draw`) и **что делать с событием** (`handle_event`). Остальные методы нужны реже, и у них есть поведение по умолчанию.

Аналог: класс `TView`. В C++ от него наследовались; в Rust реализуют трейт.

Пример: «оценка» звёздочками

Пять звёзд `★★★☆☆`: стрелки ←/→ и щелчок меняют оценку, а значение участвует в обмене данными диалога, как у обычного поля.

```
use tvision::*;

pub struct Rating {
    pub vs: ViewState,
    pub value: u32, // 0..=5
}

impl Rating {
    pub fn new(x: i32, y: i32) -> Self {
        let mut vs = ViewState::new(Rect::new(x, y, x + 5, y + 1));
        vs.options |= of::SELECTABLE | of::FIRST_CLICK;
        Rating { vs, value: 3 }
    }
}

impl View for Rating {
    tvision::impl_view_basics!();

    fn draw(&self, dc: &mut DrawCtx) {
        let a = dc.color(if self.vs.is_focused() { 2 } else { 1 });
        let s: String = (0..5).map(|i| if i < self.value { '★' } else { '☆' }).collect();
        dc.write_str(0, 0, &s, a);
    }

    fn palette(&self) -> &'static [u8] {
        color::palettes::INPUT_LINE // 1 — обычный цвет, 2 — в фокусе
    }

    fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
        view::default_handle_event(&self.vs, ev, ctx);
        match ev {
            Event::MouseDown(m) => {
                let p = self.vs.make_local(m.pos);
                self.value = (p.x + 1).clamp(0, 5) as u32;
                ev.clear();
            }
```

```
            Event::KeyDown(k) if self.vs.is_focused() => {
                match k.key {
                    Key::Left => self.value = self.value.saturating_sub(1),
                    Key::Right => self.value = (self.value + 1).min(5),
                    _ => return,
                }
                ev.clear();
            }
            _ => {}
        }
    }

    // Обмен данными: значение — число звёзд.
    fn has_data(&self) -> bool {
        true
    }
    fn get_data(&self) -> Value {
        Value::U32(self.value)
    }
    fn set_data(&mut self, v: &Value) {
        self.value = v.as_u32().unwrap_or(0).min(5);
    }
}

fn rating_test() {
    let mut r = Rating::new(0, 0);
    r.set_data(&Value::U32(9));
    assert_eq!(r.get_data(), Value::U32(5)); // больше пяти не бывает
}
```

Методы трейта

Почти всегда нужны:

Метод	Когда вызывается	Что писать
`draw(&self, dc)`	при каждой перерисовке	нарисовать всю свою область
`handle_event(&mut self, ev, ctx)`	пришло событие	обработать своё и вызвать `ev.clear()`
`palette(&self)`	при рисовании	палитра для `dc.color(n)`; можно взять у похожего элемента
служебные `vs`, `vs_mut`, `as_any`, `as_any_mut`, `into_any`	библиотеке нужен доступ	пишет макрос `impl_view_basics!()`

Для элементов диалога:

Метод	Зачем
`has_data`, `get_data`, `set_data`	участвовать в обмене данными диалога (`Value`)
`valid(&mut self, cmd, ctx) -> bool`	можно ли закрыть окно: `false` — не закрывать (так работают валидаторы)

Редко нужны:

Метод	Зачем
`set_state(flag, on)`	узнать, что представление получило или потеряло фокус и т. п.
`update_commands(&self, ctx)`	разрешать и запрещать команды, пока представление в фокусе
`exec_result(tag, cmd, view, ctx)`	получить ответ модального окна, открытого через `ctx.exec` (см. `Ctx`)
`change_bounds(r)`, `size_limits(owner)`	изменение размера и его пределы
`size_hint()`, `size_policy()`	желаемый размер и политика для раскладок

Метод	Зачем
`help_ctx()`	номер темы справки для F1
`monitor_event(ev)`	видеть все события программы (для отладочного окна)
`as_window()`, `as_group()`, `as_group_mut()`	«я окно», «у меня есть дети» — пишут макросы и шаблон окна
`contains_id(id)`	есть ли `id` среди детей

Как найти представление нужного типа

Представления хранятся как «какое-то представление» — `Box<dyn View>`. Чтобы получить свой тип, есть три способа:

- по id в группе: `group.find::<InputLine>(id)` или `find_mut`;
- по id во всей программе: `app.find::<InputLine>(id)`;
- из `dyn View`: `v.as_any().downcast_ref::<InputLine>()`.

```
use tvision::*;

fn text_of(v: &dyn View) -> Option<String> {
    v.as_any().downcast_ref::<InputLine>().map(|il| il.text())
}
```

`ViewState`

Что это. Общие поля любого представления: где лежит, какого размера, в каком состоянии. Хранится в поле `vs`.

Аналог: поля класса `TView` (`origin`, `size`, `state`, `options`…).

Поле	Что хранит
`id`	номер представления — выдаётся сам
`origin`	левый верхний угол **в координатах владельца**
`size`	ширина и высота
`state`	флаги состояния `sf::…` (видно ли, в фокусе ли)
`options`	флаги поведения `of::…` (может ли получать фокус)
`grow_mode`	как меняться, когда владелец меняет размер (`gf::…`)
`drag_mode`	как перетаскивать (`dm::…`)
`cursor`	где текстовый курсор (в своих координатах)
`help_ctx`	номер темы справки

Метод	Что делает
`ViewState::new(rect)`	создать для прямоугольника; представление видимо
`bounds()`	прямоугольник в координатах владельца
`extent()`	прямоугольник в своих координатах: от `(0, 0)` до размера
`set_bounds(r)`	задать положение и размер

Метод	Что делает
`make_local(p)`	точку владельца — в свои координаты (для мыши)
`is_focused()`	в фокусе (получает клавиши)
`is_selected()`, `is_active()`, `is_visible()`, `is_disabled()`, `is_selectable()`	другие состояния
`get_state(флаг)`	установлен ли флаг
`show_cursor()`, `hide_cursor()`, `set_cursor(x, y)`, `block_cursor()`, `normal_cursor()`	текстовый курсор

`ViewId`

Что это. Номер представления (`u32`). Его возвращают `add` и `insert_window`; по нему потом находят представление (`find`, `find_mut`). Номер выдаётся сам и не повторяется.

> Почему номер, а не ссылка? Представлением владеет окно; пока вы держите ссылку, Rust не даст окну им пользоваться. Номер можно хранить сколько угодно.

Флаги: `sf`, `of`, `gf`, `dm`

`sf` — состояние (`vs.state`)

Меняет программа, вы обычно только **читаете** — через методы `is_focused()` и т. п.

Флаг	Значит
`VISIBLE`	видно
`SELECTED`	выбрано в своём окне
`FOCUSED`	получает клавиши (выбрано, и окно активно)
`ACTIVE`	лежит в активном окне
`DISABLED`	недоступно
`MODAL`	выполняется как модальное
`CURSOR_VIS`, `CURSOR_INS`	курсор виден; курсор-блок
`SHADOW`	рисуется тень (у окон)
`DRAGGING`, `DEFAULT`, `EXPOSED`	служебные

`of` — поведение (`vs.options`)

Задаёте **вы** при создании.

Флаг	Значит
`SELECTABLE`	может получить фокус (по Tab или щелчку)
`FIRST_CLICK`	первый щелчок по невыбранному тоже обрабатывается (иначе он только выбирает)
`TOP_SELECT`	при выборе поднимается наверх (окна)
`TILEABLE`	участвует в «Мозаике» и «Каскаде»

Флаг	Значит
`CENTER_X`, `CENTER_Y`, `CENTERED`	встать по центру при вставке
`PRE_PROCESS`, `POST_PROCESS`	видеть клавиши раньше / позже элемента в фокусе (строка состояния, кнопки)
`VALIDATE`	проверять при уходе фокуса

`gf` — растягивание (`vs.grow_mode`)

Когда окно растягивают, каждый элемент сдвигает свои края по этим флагам: `GROW_LO_X` — левый край, `GROW_LO_Y` — верхний, `GROW_HI_X` — правый, `GROW_HI_Y` — нижний. `GROW_ALL` — все четыре (элемент едет вместе с правым нижним углом), `GROW_REL` — пропорционально. Подробнее — в координатах.

`dm` — перетаскивание (`vs.drag_mode`)

`DRAG_MOVE`, `DRAG_GROW`, `LIMIT_ALL` и т. п. Нужны только окнам; что пользователю можно делать с окном, проще задать флагами окна `wf::…`.

`Value`

Что это. Данные элемента диалога для обмена (`get_data` / `set_data`).

Вариант	У кого
`Str(String)`	строка ввода, многострочное поле, редактор
`U32(u32)`	флажки (биты), переключатели (номер)
`Int(i64)`	список (номер строки), числовое поле, полоса выполнения
`List(Vec<Value>)`	диалог или группа: значения всех элементов по порядку
`None`	данных нет

Методы `as_str()`, `as_u32()`, `as_int()`, `as_list()` возвращают `Some(…)`, если вариант подходит.

```rust
use tvision::*;

fn values() {
    let v = Value::List(vec![Value::Str("Аня".into()), Value::U32(0b101)]);
    let l = v.as_list().unwrap();
    assert_eq!(l[0].as_str(), Some("Аня"));
    assert_eq!(l[1].as_u32(), Some(5));
    assert_eq!(l[1].as_int(), None); // не тот вариант
}
```

Как этим пользоваться — урок 6.

`Ctx`

Что это. «Связь с внешним миром» для представления. Представление не знает ни своего окна, ни программы — когда ему нужно что-то сделать снаружи себя, оно **просит** через `Ctx`, а программа выполняет просьбу сразу после того, как `handle_event` закончится.

`Ctx` передаётся в `handle_event`, `valid`, `exec_result` и `update_commands`.

Чаще всего нужно:

Метод	Что делает
`post_command(cmd, Info::None)`	послать команду — она придёт другим представлениям и в `AppHandler`
`message(cmd, info)`	разослать сообщение соседям по окну
`select(id)`	перевести фокус на соседа
`close(id)`	закрыть (удалить) представление, например своё окно
`exec(Box::new(диалог), Some(self.vs.id), метка)`	открыть модальное окно; ответ придёт в `exec_result`
`with_program(|app| …)`	сделать что-то с программой целиком
`clipboard`	буфер обмена: `ctx.clipboard.set("…")`, `get()`

Реже: `focus(id)` (фокус куда угодно), `select_next(вперёд)`, `make_first(id)`, `end_modal(cmd)`, `put_event(ev)`, `popup_menu(menu, pos)` (контекстное меню; позиция — на экране), `exec_near`, `exec_placed` (окно рядом с другим представлением), `grab_mouse(id)`, `release_mouse()`, `mouse_grabbed_by(id)` (захват мыши для протяжки), `command_enabled`, `enable_command`, `disable_command`, `set_command`, `owner_size()`, поля `screen_size`, `phase`.

Модальное окно изнутри представления

Программа может показать диалог и дождаться ответа (`exec_dialog`), но только у неё есть для этого `&mut Program`. Представление поэтому **просит** открыть окно и получает ответ позже — в метод `exec_result`:

```
use tvision::*;

const TAG_NAME: u16 = 1;

/// Надпись: щелчок спрашивает имя и показывает его.
pub struct AskName {
    pub vs: ViewState,
    pub name: String,
}

impl View for AskName {
    tvision::impl_view_basics!();

    fn draw(&self, dc: &mut DrawCtx) {
        let a = dc.color(1);
        dc.fill(self.vs.extent(), ' ', a);
        let t = if self.name.is_empty() { "[ввести имя]" } else { self.name.as_str() };
        dc.write_str(0, 0, t, a);
    }

    fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
        if let Event::MouseDown(_) = ev {
            let mut d = Dialog::centered(40, 8, "Имя");
            d.add(InputLine::new(Rect::new(3, 2, 37, 3), 30));
            d.add(Button::new(Rect::new(14, 4, 26, 6), "O~K~", cm::OK, bf::DEFAULT));
            // «Откройте, пожалуйста, и ответ пришлите мне с меткой TAG_NAME».
            ctx.exec(Box::new(d), Some(self.vs.id), TAG_NAME);
            ev.clear();
        }
    }

    fn exec_result(&mut self, tag: u16, cmd: u16, view: Box<dyn View>, _ctx: &mut Ctx) {
        if tag == TAG_NAME && cmd == cm::OK {
            let data = view.get_data(); // [Str(имя)]
            self.name = data.as_list().and_then(|l| l.first()).and_then(|v|
```

```
v.as_str()).unwrap_or("").to_string();
        }
    }
}
```

Метка (`tag`) нужна, если представление открывает несколько разных окон: по ней видно, на какое пришёл ответ.

Действие с программой

```rust
use tvision::*;

/// Из любого представления: сменить узор рабочего стола.
fn change_pattern(ctx: &mut Ctx) {
    ctx.with_program(|app| app.desktop_mut().set_pattern('▒'));
}
```

`Request` и `PlaceFn`

Внутреннее устройство `Ctx`, нужно редко. `Request` — просьба к программе (открыть меню, контекстное меню, модальное окно, передать фокус, выполнить функцию); её отправляет `ctx.request(r)`, но обычно вместо этого вызывают готовые методы `exec`, `popup_menu`, `focus`, `with_program`. `PlaceFn` — функция `(границы якоря, размер владельца) → границы окна` для `ctx.exec_placed`: так окно истории встаёт прямо под строкой ввода.

`CommandSet`

Что это. Список запрещённых команд. Хранится в программе; по нему меню и строка состояния решают, какие пункты рисовать серыми. Методы `is_enabled(cmd)`, `enable(cmd)`, `disable(cmd)`. Напрямую обычно не нужен — удобнее `app.enable_command(cmd)` / `app.disable_command(cmd)`. Запретить можно только команды с номерами до 255.

`Phase`

Что это. На каком этапе доставки клавиша (`ctx.phase`): `PreProcess` — её видят строка состояния и меню, `Focused` — элемент в фокусе, `PostProcess` — кнопки «по умолчанию». Нужна редко.

`DrawCtx`

Что это. «Холст» для метода `draw`. Координаты на нём **свои**: `(0, 0)` — левый верхний угол представления. Всё, что выходит за его границы или закрыто другими окнами, обрезается само.

Аналог: методы `writeLine`, `writeStr`, `getColor` у `TView`.

```rust
use tvision::*;

/// Рамка из точек и надпись по центру.
fn draw_box(dc: &mut DrawCtx, text: &str) {
    let a = dc.color(1);
    let size = dc.size;
    dc.fill(Rect::new(0, 0, size.x, size.y), '·', a); // вся область
    let w = text::str_width(text) as i32;
    dc.write_str((size.x - w) / 2, size.y / 2, text, a);
}
```

Метод	Что делает
`color(n)`	цвет номер `n` из палитры представления
`fill(r, символ, цвет)`	залить прямоугольник
`write_str(x, y, текст, цвет)`	написать текст; возвращает его ширину
`write_char(x, y, символ, цвет, сколько)`	повторить символ
`write_line(x, y, ширина, высота, &буфер)`	вывести готовый `DrawBuffer`
`size`	размер представления

Реже: `get_color(0xHHLL)` (пара цветов для текста с горячей буквой), `map_color(n)`, `write_cells(x, y, &клетки)`, `recolor(r, цвет)` (перекрасить уже нарисованное), `shadow(r)` (тень), `owner_palette_len()` (какой длины палитра владельца: 8 — окно, 32 — диалог; так таблица подбирает цвета), `commands` (какие команды запрещены), `origin()`, `clip()`, `new`, `sub` (служебные).

> Представление должно закрашивать **всю** свою область — что не нарисовано, останется от прошлого кадра. Последний столбец — `size.x - 1`, последняя строка — `size.y - 1`.

`Group`

Что это. Представление с детьми. Рабочий стол, окно, диалог — это группы (или построены вокруг группы). Группа рисует детей, раздаёт им события, переводит фокус по Tab.

Аналог: `TGroup`.

Дети лежат в `children` по порядку **наложения**: первый — самый нижний, последний — верхний. Выбранный ребёнок — `current`.

Чаще всего нужно:

Метод	Что делает
`add(v) -> ViewId`	добавить ребёнка
`find::<T>(id)`, `find_mut::<T>(id)`	найти представление типа `T` по id (и среди внуков)
`current_view()`, `current_view_mut()`	выбранный ребёнок (у рабочего стола — активное окно)
`remove(id)`	убрать ребёнка
`get_data()`, `set_data(v)`	данные всех детей разом

Реже: `set_layout(раскладка)`, `apply_layout()`, поле `layout` (раскладки), `Group::new(rect)`, `insert(Box<dyn View>)`, `index_of(id)`, `child(id)`, `view(id)`, `view_mut(id)` (без приведения типа), `select_id(id, ctx)`, `select_index(i, ctx)`, `set_current`, `select_next(вперёд, ctx)`, `reset_current()`, `make_first(i)`, `put_at(i, v)`, `focus_path(id, ctx)` (выбрать представление вместе с его окном), `call_on(id, ctx, f)`, `for_each_view(f)` (обойти всех), `broadcast_children`, `dispatch`, `has_data()`, `geometry_of(id)`, `owner_origin_of(id)` (для расчёта экранных координат), `cursor_pos`, `announce_focus`, `help_ctx()`.

Своя группа. Обычно своё «составное» представление делают на основе окна (урок 8). Если окно не нужно — на основе группы:

```
use tvision::*;
```

```rust
/// Подпись сверху и поле ввода под ней — как один элемент.
pub struct Field {
    pub group: Group,
    pub input: ViewId,
}

impl Field {
    pub fn new(r: Rect, title: &str) -> Self {
        let mut group = Group::new(r);
        group.vs.options |= of::SELECTABLE;
        let w = r.width();
        group.add(StaticText::new(Rect::new(0, 0, w, 1), title)); // координаты — внутри группы
        let input = group.add(InputLine::new(Rect::new(0, 1, w, 2), 80));
        Field { group, input }
    }

    pub fn text(&self) -> String {
        self.group.find::<InputLine>(self.input).map(|il| il.text()).unwrap_or_default()
    }
}

impl View for Field {
    tvision::impl_group_basics!(group);

    fn draw(&self, dc: &mut DrawCtx) {
        self.group.draw_children(dc);
    }
    fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
        self.group.handle_group_event(ev, ctx);
    }
    fn set_state(&mut self, flag: u16, on: bool) {
        self.group.set_group_state(flag, on);
    }
    fn valid(&mut self, cmd: u16, ctx: &mut Ctx) -> bool {
        self.group.valid(cmd, ctx)
    }
}
```

Макросы

Пишут служебные методы `View`, одинаковые у всех. Вызываются **внутри** `impl View for …`.

Макрос	Когда
`impl_view_basics!()`	у вашей структуры есть поле `vs: ViewState`
`impl_group_basics!(путь)`	структура построена вокруг группы: `group`, `win.group` (своё окно), `dlg.win.group` (свой диалог); дополнительно пишет работу с детьми и данными
`impl_view_basics_field!(поле)`	`ViewState` лежит во вложенной структуре: `self.поле.vs`
`impl_group_core!(путь)`	как `impl_group_basics!`, но данные (`has_data` / `get_data` / `set_data`) вы пишете сами

Вспомогательные функции

- `view::default_handle_event(&self.vs, ev, ctx)` — стандартное поведение: щелчок по представлению переводит на него фокус. Вызывайте первой строкой своего `handle_event`.
- `view::calc_bounds(v, сдвиг, размер_владельца)` — новые границы по `grow_mode`, когда владелец меняет размер. Нужна своим группам.
- `view::next_id()` — новый номер представления (его вызывает `ViewState::new`).

- `view::HC_NO_CONTEXT` (0), `view::HC_DRAGGING` (1) — стандартные номера тем справки.

Координаты подробно

Основы — в уроке 4. Здесь — то, что нужно реже.

Три системы координат

У каждой точки три «адреса»:

1. **экранные** — от левого верхнего угла терминала;
2. **владельца** — от угла группы, в которой лежит представление (`vs.origin` и `Rect` в конструкторе — в них);
3. **свои** — от левого верхнего угла самого представления.

Что	В каких координатах
`Rect` в конструкторе, `vs.origin`, `vs.bounds()`	владельца
рисование (`DrawCtx`), `vs.extent()`, `vs.cursor`	свои
`m.pos` мыши в `View::handle_event`	владельца → свои через `make_local`
`m.pos` мыши в `AppHandler::handle_event`	экранные
окно для `app.insert_window`	рабочего стола
элемент в `Dialog::add` / `Window::add`	окна (рамка — строка и столбец 0)
`ctx.popup_menu(…, pos)`	экранные
`Editor::cursor()`, `Table::cursor()`	**не экранные**: строка и номер символа в тексте, строка и столбец данных

Перевод в экранные координаты

```rust
use tvision::platform::HeadlessBackend;
use tvision::*;

/// Границы представления на экране.
fn screen_rect(app: &Program, id: ViewId) -> Option<Rect> {
    let (bounds, _) = app.group.geometry_of(id)?; // у владельца
    let o = app.group.owner_origin_of(id)?; // где на экране угол владельца
    Some(bounds.moved(o.x, o.y))
}

fn layers() {
    let menu = Menu::new().submenu("~Ф~айл", Menu::new().item("~В~ыход", cm::QUIT));
    let mut app = Program::new(Box::new(HeadlessBackend::new(80, 25)), Some(menu), None);
    let mut w = Window::new(Rect::new(4, 2, 40, 12), "Окно", 1); // на рабочем столе
    let button = w.add(Button::new(Rect::new(7, 1, 19, 3), "О~К~", cm::OK, bf::DEFAULT)); // в окне
    let win = app.insert_window(w);
    assert_eq!(screen_rect(&app, win).unwrap().a, Point::new(4, 3)); // стол — со строки 1, под меню
    assert_eq!(screen_rect(&app, button).unwrap().a, Point::new(11, 4)); // 4 + 7, 3 + 1
}
```

Рабочий стол и экран

Окна лежат на рабочем столе, а он — между строкой меню (строка 0) и строкой состояния (последняя строка). На экране 80×25 с меню и строкой состояния стол — 80×23. Размер «во весь стол» берите у стола: `app.desktop().group.vs.extent()`. Размер экрана — `app.ctx.screen_size`.

Центрирование

`Dialog::centered(w, h, …)` и флаг `of::CENTERED` (или `CENTER_X`, `CENTER_Y`) ставят представление по центру владельца **в момент вставки**. Чтобы диалог оставался по центру и после изменения размера терминала, добавьте `grow_mode = gf::GROW_REL`.

```
use tvision::platform::HeadlessBackend;
use tvision::*;

fn centering() {
    let status = vec![StatusDef::all(vec![StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT)])];
    let menu = Menu::new().submenu("~Ф~айл", Menu::new().item("~В~ыход", cm::QUIT));
    let mut app = Program::new(Box::new(HeadlessBackend::new(80, 25)), Some(menu), Some(status));
    assert_eq!(app.desktop().group.vs.size, Point::new(80, 23));
    let id = app.insert_window(Dialog::centered(40, 9, "По центру"));
    let o = app.find::<Dialog>(id).unwrap().win.group.vs.origin;
    assert_eq!(o, Point::new((80 - 40) / 2, (23 - 9) / 2));
}
```

Растягивание

Когда владелец меняет размер на `(dx, dy)`, края ребёнка сдвигаются по флагам `grow_mode`:

Чего хотим	`grow_mode`
растягиваться вместе с окном	`gf::GROW_HI_X | gf::GROW_HI_Y`
оставаться на месте	`0`
прижаться к низу и тянуться по ширине (строка сведений)	`gf::GROW_LO_Y | gf::GROW_HI_Y | gf::GROW_HI_X`
прижаться к правому нижнему углу (кнопка)	`gf::GROW_ALL`

```
use tvision::*;

fn grow() {
    // Окно выросло на (+6, +2): с 30×10 до 36×12.
    let (owner, delta) = (Point::new(36, 12), Point::new(6, 2));
    let mut ok = Button::new(Rect::new(17, 7, 29, 9), "O~K~", cm::OK, bf::DEFAULT);
    ok.vs.grow_mode = gf::GROW_ALL;
    assert_eq!(view::calc_bounds(&ok, delta, owner), Rect::new(23, 9, 35, 11)); // уехала в угол
}
```

Что растягивается само: `Editor`, `TreeView`, `Outline`, `DualPanel` и полосы прокрутки. **Не** растягиваются сами: `Table`, `ListBox`, `FilePanel` и все поля и кнопки — им `grow_mode` задают вручную.

Прокрутка

У прокручиваемых представлений строка на экране и строка данных — разные числа: **строка данных = первая видимая + строка на экране**. Таблица и дерево отдают строку данных (`cursor()`), а не экранную.

Широкие символы

Иероглифы и многие эмодзи занимают **две** клетки. Поэтому ширину текста считают `text::str_width`, а не `chars().count()`. У редактора позиция — номер символа; экранный столбец дают `col_to_x` / `x_to_col`.

Частые ошибки

Симптом	Причина	Как исправить
элемент на клетку шире	ширина как `x2 - x1 + 1`	`Rect::new(x, y, x + ширина, y + высота)`
элемент залез на рамку	начали с 0 или кончили на ширине окна	от 1 до `ширина − 2`, или `interior()`
у кнопки нет тени	высота 1	у кнопки высота 2
щелчок «промахивается»	сравнили `m.pos` со своим прямоугольником	сначала `make_local(m.pos)`
контекстное меню не там	передали свои координаты	нужны экранные
окно залезает под строку состояния	размер взяли у экрана	у рабочего стола
элемент не растёт вместе с окном	нет `grow_mode`	`gf::GROW_HI_X | gf::GROW_HI_Y`

Справочник: программа, рабочий стол, окна, диалоги

Каркас любого приложения: сама программа, рабочий стол, окна и диалоги.

На этой странице: `Program` · `AppHandler` · `NoHandler` · `Desktop` · `Background` · `Window` · `wf` · `Frame`, `FrameInfo` · `Dialog` · `DialogPalette`

`Program` — программа

Что это. Вся программа целиком: экран, строка меню, рабочий стол, строка состояния, цикл событий, палитра, буфер обмена.

Аналог: `TProgram` и `TApplication`.

```
use tvision::*;

struct App;

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        if ev.command() == Some(cm::NEW) {
            message_box(app, "\x03Привет!", mf::INFORMATION | mf::OK_BUTTON);
            ev.clear();
        }
    }
}

fn run() -> std::io::Result<()> {
    let menu = Menu::new().submenu("~Ф~айл", Menu::new().item_key("~П~ривет", cm::NEW, kb::F2,
"F2").item_key("~В~ыход", cm::QUIT, kb::ALT_X, "Alt-X"));
    let status = vec![StatusDef::all(vec![StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT)])];
    let mut app = Program::with_terminal(Some(menu), Some(status))?;
    app.run(&mut App);
    Ok(())
}
```

Как создать

- `Program::with_terminal(меню, строка_состояния)` — в настоящем терминале. Меню — `Option<Menu>`, строка состояния — `Option<Vec<StatusDef>>`; любое можно не задавать (`None`).
- `Program::new(терминал, меню, строка_состояния)` — с любым терминалом, например невидимым для тестов (урок 10).

Запуск

Метод	Что делает
`run(&mut обработчик)`	работать, пока не выйдут (Alt+X, `cm::QUIT`, `quit()`)
`quit()`	выйти из `run`
`put_event(ev)`	поставить событие в очередь (например, команду при запуске)
`pump(&mut обработчик)`	обработать всё, что в очереди, и вернуться (тесты)

Окна и диалоги

Метод	Что делает
`insert_window(окно) -> ViewId`	положить окно на рабочий стол
`exec_dialog(диалог) -> (команда, диалог)`	показать модально, дождаться, вернуть диалог обратно
`exec_dialog_idle(диалог, функция)`	то же, и пока диалог открыт, вызывать `функция` — для фоновой работы с полосой выполнения
`find::<T>(id)`, `find_mut::<T>(id)`	найти представление по id где угодно, даже в открытом диалоге
`desktop()`, `desktop_mut()`	рабочий стол

```rust
use tvision::*;

/// Диалог, который «работает» сам: полоса доходит до конца, и он закрывается.
fn countdown(app: &mut Program) -> u16 {
    let mut d = Dialog::centered(40, 8, "Подождите");
    let bar = d.add(ProgressBar::new(Rect::new(3, 2, 37, 3), 50));
    d.add(Button::new(Rect::new(14, 4, 26, 6), "Отмена", cm::CANCEL, bf::DEFAULT));
    let (cmd, _) = app.exec_dialog_idle(d, |app, _| {
        let b = app.find_mut::<ProgressBar>(bar)?;
        b.step(1);
        b.is_done().then_some(cm::OK) // Some(команда) — закрыть диалог
    });
    cmd
}
```

Команды и настройки

Метод / поле	Что
`enable_command(cmd)`, `disable_command(cmd)`, `command_enabled(cmd)`	разрешить / запретить команду (запрещённые пункты серые)
`palette`	палитра программы — поменяйте, чтобы перекрасить всё
`ctx.clipboard`	буфер обмена
`ctx.screen_size`	размер экрана

Реже: `step()` (один шаг цикла), `get_event()`, `handle(ev, h)`, `redraw()`, `is_running()`, `exec_view` и `exec_view_idle` (то же, что `exec_dialog`, для `Box<dyn View>`), `insert_window_boxed`, `modal_top()`, `focus(id)` (перевести фокус на любое представление), `status_line_mut()`, `help_ctx()` (номер темы справки элемента в фокусе), `screen()`, `screen_text()` (тесты), `group`, `desktop_id`, `menu_bar_id`, `status_line_id`, `double_delay` (интервал двойного щелчка), `reverse_buttons` (мышь для левши), `event_monitor`, `dos_shell(текст)` (выйти в оболочку ОС и вернуться), `suspended(|| …)` (запустить другую программу в терминале — например, `vim` — и вернуться), `program::DOUBLE_CLICK`.

`modal_hook` — команды из открытого диалога

Пока открыт модальный диалог, `AppHandler` **не вызывается**. Если в диалоге есть кнопка со своей командой (не `OK` / `CANCEL`), её ловят в `app.modal_hook`:

```rust
use tvision::*;

const CM_BROWSE: u16 = 130;

/// Диалог «Экспорт» с кнопкой «Обзор…», которая открывает выбор файла.
fn export_dialog(app: &mut Program) -> Option<String> {
    let mut d = Dialog::centered(52, 9, "Экспорт");
    let field = d.add(InputLine::new(Rect::new(3, 3, 36, 4), 200));
    d.add(Button::new(Rect::new(37, 2, 49, 4), "~О~бзор…", CM_BROWSE, bf::NORMAL));
    d.add(Button::new(Rect::new(12, 6, 24, 8), "О~К~", cm::OK, bf::DEFAULT));
    app.modal_hook = Some(Box::new(move |app, ev| {
        if ev.command() == Some(CM_BROWSE) {
            let (cmd, f) = app.exec_dialog(FileDialog::new("*", "Файл", "~И~мя", fd::OK_BUTTON, 0));
            if cmd != cm::CANCEL {
                if let Some(il) = app.find_mut::<InputLine>(field) {
                    il.set_text(&f.file_name().display().to_string());
                }
            }
            ev.clear();
        }
    }));
    let (cmd, d) = app.exec_dialog(d);
    app.modal_hook = None; // не забыть убрать
    (cmd == cm::OK).then(|| d.win.group.find::<InputLine>(field).unwrap().text())
}
```

`AppHandler` — код приложения

Что это. Трейт для вашего обработчика: сюда приходят команды, которые не обработали окна и элементы.

Аналог: переопределённые методы наследника `TApplication`.

Метод	Когда вызывается
`handle_event(&mut self, app, ev)`	пришло необработанное событие (обычно команда из меню)
`idle(&mut self, app)`	событий нет: обновить часы, запретить ненужные команды

Оба метода необязательны. `app` — программа: через неё показывают диалоги, кладут окна, меняют палитру. Во время модального диалога обработчик не вызывается (см. `modal_hook` выше).

`NoHandler`

Пустой обработчик: `app.run(&mut program::NoHandler)` — для программ, где всё делают сами окна. Лежит в модуле `program`.

`Desktop` — рабочий стол

Что это. Область между строкой меню и строкой состояния: фон и окна.

Аналог: `TDesktop`.

```rust
use tvision::*;

fn arrange(app: &mut Program) {
    app.desktop_mut().set_pattern('▒'); // узор фона
    let r = app.desktop().group.vs.extent(); // весь рабочий стол
    app.desktop_mut().tile(r); // окна плиткой
}
```

Метод / поле	Что
`set_pattern(c)`, `pattern()`	символ фона (по умолчанию ▒)
`tile(r)`, `cascade(r)`	разложить окна плиткой / каскадом (окна с флагом `of::TILEABLE`)
`group`	окна — его дети; активное окно — `group.current_view()`
`group.vs.extent()`	прямоугольник всего стола — для окна «во весь стол»

Реже: `tile_columns_first` (плитка сначала по столбцам), `background_id`.

Сам обрабатывает команды `cm::NEXT` (следующее окно), `cm::PREV`, `cm::TILE`, `cm::CASCADE`.

`Background` — фон

Что это. Фон рабочего стола, залитый одним символом. `Background::new(границы, символ)`. Обычно его не трогают, а меняют узор через `desktop.set_pattern(c)`.

`Window` — окно

Что это. Рамка с заголовком, номером и значками `[■]` (закрыть) и `[↑]` (распахнуть) — и место для содержимого. Окна можно двигать, растягивать, распахивать и закрывать — всё это они умеют сами.

Аналог: `TWindow`.

```rust
use tvision::*;

fn list_window(app: &mut Program) {
    let mut w = Window::new(Rect::new(2, 1, 42, 15), "Список", 1);
    let sb = w.standard_scroll_bar(true); // полоса прокрутки на правой рамке
    let mut lb = ListBox::new(w.interior(), 1, Some(sb));
    lb.vs.grow_mode = gf::GROW_HI_X | gf::GROW_HI_Y; // тянуться вместе с окном
    lb.set_items((1..=50).map(|i| format!("Строка {}", i)).collect());
    w.add(lb);
    app.insert_window(w);
}
```

Метод / поле	Что
`Window::new(границы, заголовок, номер)`	окно; номер 1–9 виден в рамке (Alt+номер), 0 — без номера
`add(элемент) -> ViewId`	положить внутрь; координаты — от угла окна, рамка — строка и столбец 0
`interior()`	место внутри рамки
`standard_scroll_bar(вертикальная) -> ViewId`	полоса прокрутки на правой (`true`) или нижней (`false`) рамке
`select(id)`	дать фокус элементу окна (как у диалога) — например, строке ввода в окне со строкой и таблицей
`set_layout(раскладка)`, `adjust_size()`	расставлять элементы раскладкой (пересчёт при изменении размера) / размер окна по раскладке
`title`	заголовок — можно менять на ходу
`flags`	что можно делать с окном (`wf::…`)
`palette`	цвета: `palettes::BLUE_WINDOW` (по умолчанию), `CYAN_WINDOW`, `GRAY_WINDOW`
`group`	содержимое; `group.vs` — положение самого окна

Реже: `insert(Box<dyn View>)`, `zoom(размер)`, `number`, `id()`; `handle_window_event`, `draw_window`, `set_window_state` — для своих окон (урок 8).

Окно само понимает: мышь за заголовок — двигать, за правый нижний угол — менять размер, `cm::ZOOM` (F5) — распахнуть, `cm::CLOSE` (Alt+F3) — закрыть, `cm::RESIZE` (Ctrl+F5) — двигать и тянуть клавишами.

Своё окно

Своё окно с данными — структура с полем `Window` и шаблонной реализацией `View`. Подробно — урок 8:

```
use tvision::*;

pub struct NotesWindow {
    pub win: Window,
    pub notes: Vec<String>,
}

impl View for NotesWindow {
    tvision::impl_group_basics!(win.group);
    fn as_window(&self) -> Option<&Window> {
        Some(&self.win)
    }
    fn draw(&self, dc: &mut DrawCtx) {
        self.win.draw_window(dc);
    }
    fn palette(&self) -> &'static [u8] {
        self.win.palette
    }
    fn set_state(&mut self, flag: u16, on: bool) {
        self.win.set_window_state(flag, on);
    }
    fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
        self.win.handle_window_event(ev, ctx);
        // здесь — своя обработка того, что окно не забрало
    }
}
```

`wf` — что можно делать с окном

Флаги в `window.flags`: `MOVE` — двигать, `GROW` — менять размер, `CLOSE` — закрывать (значок `[■]`), `ZOOM` — распахивать (значок `[↑]`). У окна по умолчанию все четыре, у диалога — `MOVE` и `CLOSE`. Окно, которое нельзя закрыть: `w.flags &= !wf::CLOSE;`.

`window::MIN_WIN_SIZE` — наименьший размер окна (16×6).

`Frame` и `FrameInfo` — рамка

Что это. То, что рисует рамку окна. Окно вызывает его само; вручную нужно, только если вы рисуете окно без `Window`: `Frame::draw_frame(dc, &FrameInfo { title, number, flags, active, dragging, zoomed })`, палитра — `Frame::palette()`. Значки: `frame::CLOSE_ICON` (`[■]`), `ZOOM_ICON` (`[↑]`), `UNZOOM_ICON` (`[↕]`), `DRAG_ICON`, `DRAG_LEFT_ICON`. Активное окно — двойная рамка, неактивное — одинарная.

`Dialog` — диалог

Что это. Окно для вопросов пользователю: серое, с полями и кнопками. Обычно показывается **модально** — пока его не закроют, остальная программа ждёт.

Аналог: `TDialog`.

```rust
use tvision::*;

fn ask_name(app: &mut Program) -> Option<String> {
    let mut d = Dialog::centered(40, 9, "Знакомство");
    let name = d.add(InputLine::new(Rect::new(10, 2, 37, 3), 40));
    d.add(Label::new(Rect::new(2, 2, 9, 3), "~И~мя:", Some(name)));
    d.add(Button::new(Rect::new(6, 5, 18, 7), "O~K~", cm::OK, bf::DEFAULT));
    d.add(Button::new(Rect::new(21, 5, 33, 7), "Отмена", cm::CANCEL, bf::NORMAL));
    let (cmd, d) = app.exec_dialog(d);
    (cmd == cm::OK).then(|| d.win.group.find::<InputLine>(name).unwrap().text())
}
```

Метод / поле	Что
`Dialog::centered(ширина, высота, заголовок)`	диалог, который встанет по центру
`Dialog::new(границы, заголовок)`	в заданном месте
`add(элемент) -> ViewId`	добавить; **порядок добавления = порядок Tab = порядок данных**
`select(id)`	какой элемент получит фокус первым
`set_layout(раскладка)`, `adjust_size()`	элементы без координат — раскладкой; размер диалога по содержимому
`set_data(v)`, `get_data()`	заполнить / прочитать все поля разом
`win.group.find::<T>(id)`	найти элемент — например, после `exec_dialog`

Реже: `set_palette(DialogPalette::…)`, `insert(Box<dyn View>)`, `id()`, `win.title`, `win.flags`.

Как ведёт себя:

- Esc — то же, что «Отмена» (`cm::CANCEL`);
- Enter — нажимает кнопку с флагом `bf::DEFAULT`;
- `cm::OK`, `cm::CANCEL`, `cm::YES`, `cm::NO` закрывают диалог (перед этим поля проверяются валидаторами — кроме «Отмены»);
- другие команды кнопок идут дальше — в `modal_hook` или, если диалог не модальный, в `AppHandler`.

Диалог можно положить и как обычное окно — `app.insert_window(d)`: он останется на рабочем столе, пока его не закроют.

`DialogPalette`

Цвета диалога: `Gray` (серый, по умолчанию), `Blue`, `Cyan`. `d.set_palette(DialogPalette::Blue)`.

Справочник: меню и строка состояния

На этой странице: `Menu` · `MenuItem` · `MenuBar` · контекстное меню · `MenuSession`, `MenuOutcome` · `StatusLine` · `StatusDef` · `StatusItem` · `HintFn`

`Menu` — меню

Что это. Список пунктов меню. Его передают программе, и она делает из него **строку меню** наверху экрана; вложенные меню становятся выпадающими.

Аналог: `TMenu` и функции `newSubMenu`, `newItem`, `newLine`.

```
use tvision::*;

const CM_ABOUT: u16 = 120;

fn menu() -> Menu {
    Menu::new()
        .submenu("~С~правка", Menu::new().item("~О~ программе...", CM_ABOUT))
        .submenu(
            "~Ф~айл",
            Menu::new()
                .item_key("~О~ткрыть...", cm::OPEN, kb::F3, "F3")
                .submenu("~Н~едавние", Menu::new().item("a.txt", 200).item("b.txt", 201)) // вложенное
                .separator()
                .item_key("~В~ыход", cm::QUIT, kb::ALT_X, "Alt-X"),
        )
}

fn check() {
    let m = menu();
    assert_eq!(m.items.len(), 2);
    assert_eq!(m.find_hot_key(&kb::F3), Some(cm::OPEN)); // какая команда у F3
}
```

Метод	Что добавляет
`Menu::new()`	пустое меню
`item(название, команда)`	пункт
`item_key(название, команда, клавиша, "подпись")`	пункт с клавишей быстрого вызова (работает и при закрытом меню); подпись рисуется справа
`submenu(название, меню)`	вложенное меню
`submenu_key(название, клавиша, меню)`	вложенное меню, которое открывается ещё и клавишей
`separator()`	черту
`help(номер)`	номер темы справки для **последнего** добавленного пункта

Реже: `items` (пункты списком), `find_hot_key(&клавиша)`, `find_item(буква)`, `box_size()`, `bar_positions()`.

Как работает меню у пользователя. F10 (команда `cm::MENU`) или щелчок открывает строку меню; Alt+буква — сразу нужное меню (в любой раскладке: Alt+Ф и Alt+А одинаковы); стрелки, Enter и Esc — внутри; пункт выбирается и горячей буквой. Пункты с запрещённой командой серые.

> Клавиши пунктов меню ловятся **раньше** полей ввода. Не назначайте пунктам буквы, Enter или Ctrl+стрелки — их перестанут получать поля.

`MenuItem` — пункт меню

Что это. Один пункт: `Command` (команда), `SubMenu` (вложенное меню) или `Separator` (черта). Обычно создаётся методами `Menu`, а не вручную. Методы: `name()`, `is_separator()`, `enabled(&команды)`, `help_ctx()`.

`MenuBar` — строка меню

Что это. Верхняя строка экрана с названиями меню. Программа создаёт её сама; её id — `app.menu_bar_id`. Поле `menu` можно заменить, чтобы поменять меню на ходу:

```
use std::rc::Rc;
use tvision::*;

fn replace_menu(app: &mut Program, m: Menu) {
    if let Some(id) = app.menu_bar_id {
        if let Some(bar) = app.find_mut::<MenuBar>(id) {
            bar.menu = Rc::new(m);
        }
    }
}
```

Реже: `MenuBar::new(границы, меню)`, `active`, `item_at(x)`.

Контекстное меню

Что это. Меню, которое всплывает там, где щёлкнули правой кнопкой. Открывается так: `ctx.popup_menu(меню, точка)` из представления или `app.ctx.popup_menu(меню, точка)` из обработчика приложения. Точка — **экранная**.

```
use std::rc::Rc;
use tvision::*;

const CM_COPY_PATH: u16 = 131;

/// Правая кнопка мыши по свободному месту рабочего стола
/// (такие щелчки доходят до обработчика приложения).
fn context_menu(app: &mut Program, ev: &mut Event) {
    if let Event::MouseDown(m) = ev {
        if m.buttons & event::MB_RIGHT != 0 {
            let menu = Rc::new(Menu::new().item("~К~опировать путь",
CM_COPY_PATH).separator().item("~М~озаика", cm::TILE));
            app.ctx.popup_menu(menu, m.pos); // здесь m.pos уже экранная
            ev.clear();
        }
    }
}
```

Выбранный пункт посылает команду обычным путём.

`MenuSession` и `MenuOutcome`

Служебное: пока меню открыто, им управляет «сеанс меню» `MenuSession`, который сообщает итог — `MenuOutcome::Continue` (меню ещё открыто), `Closed` (закрыли без выбора) или `Command(cmd)`. Самим вызывать не нужно. `menu::draw_menu_box` рисует выпадающее окно меню.

`StatusLine` — строка состояния

Что это. Нижняя строка экрана с подсказками клавиш: пункт «F2 Сохранить» и показывает подсказку, и сам срабатывает по F2.

Аналог: `TStatusLine`.

Пунктов может быть несколько **наборов** — для разных мест программы. Какой показывать, решает **номер темы справки** (`help_ctx`) элемента в фокусе: например, в редакторе — одни клавиши, в списке — другие.

```
use tvision::*;

const HC_EDITOR: u16 = 100;
const CM_BUILD: u16 = 140;

fn status() -> Vec<StatusDef> {
    vec![
        // Когда в фокусе элемент с vs.help_ctx == HC_EDITOR:
        StatusDef::range(HC_EDITOR, HC_EDITOR, vec![
            StatusItem::new("~F2~ Сохранить", kb::F2, cm::SAVE),
            StatusItem::new("~F9~ Собрать", kb::f(9), CM_BUILD),
            StatusItem::new("", kb::F10, cm::MENU),
        ]),
        // Во всех остальных случаях (подходит первый набор по порядку):
        StatusDef::all(vec![
            StatusItem::new("~Alt-X~ Выход", kb::ALT_X, cm::QUIT),
            StatusItem::new("", kb::F10, cm::MENU), // невидимые — только клавиши
            StatusItem::new("", kb::ALT_F3, cm::CLOSE),
            StatusItem::new("", kb::F5, cm::ZOOM),
            StatusItem::new("", kb::F6, cm::NEXT),
        ]),
    ]
}

/// Подсказка справа от пунктов.
fn add_hint(app: &mut Program) {
    if let Some(sl) = app.status_line_mut() {
        sl.hint = Some(Box::new(|ctx| if ctx == HC_EDITOR { "Правка текста".into() } else {
String::new() }));
    }
}
```

Что важно знать.

- Пункт с **пустой надписью** не виден, но его клавиша работает. Удобно для F10, Alt+F3, F5, F6.
- Строка состояния видит клавиши **раньше всех** — даже в открытом диалоге. Поэтому F1 из строки состояния открывает справку где угодно. Но поэтому же **нельзя** вешать пункты на Enter, Esc, Tab и буквы — они перестанут работать в полях и диалогах.
- Пункт с запрещённой командой рисуется серым и не срабатывает.

Поля: `defs` (наборы — можно заменить), `hint` (подсказка), `help_ctx`. **Реже:** `StatusLine::new(границы, наборы)`, `items()`.

`StatusDef` — набор пунктов

`StatusDef::all(пункты)` — для всех мест программы; `StatusDef::range(от, до, пункты)` — когда номер темы справки в диапазоне. Поля `min`, `max`, `items`.

`StatusItem` — пункт строки состояния

`StatusItem::new("~F2~ Сохранить", kb::F2, cm::SAVE)`: надпись (часть в тильдах выделяется цветом), клавиша, команда. Поля `text`, `key`, `command`. Пункт с командой 0 — просто надпись.

`HintFn` — подсказка

Функция `номер темы → текст`, которая рисует подсказку в правой части строки состояния (`status_line.hint`, пример выше).

Справочник: проверка ввода

Валидатор следит, чтобы в строку ввода попадало только правильное: цифры в поле «Возраст», дата в формате `дд.мм.гггг`, слово из списка. Его прикрепляют к строке: `InputLine::new(…).with_validator(…)`.

На этой странице: как это работает · `FilterValidator` · `RangeValidator` · `PictureValidator` · `PicResult` · `StringLookupValidator` · `FnValidator` · `Validator`

Как это работает

Валидатор проверяет в два момента:

1. **Пока набирают.** Неправильный символ просто не появляется (например, буква в поле для цифр). Маска может сама дописать точки и скобки.
2. **Когда нажимают ОК.** Если значение не подходит (число вне диапазона, дата не до конца), появляется сообщение, курсор возвращается в поле, а диалог остаётся открытым. «Отмена» и Esc проверку не запускают.

```
use tvision::*;

fn fields(d: &mut Dialog) {
    // Дата: маска, точки дописываются сами.
    d.add(InputLine::new(Rect::new(3, 2, 15, 3),
10).with_validator(PictureValidator::new("##.##.####", true)));
    // Шестнадцатеричное число: только эти символы.
    d.add(InputLine::new(Rect::new(3, 4, 15, 5),
8).with_validator(FilterValidator::new("0123456789ABCDEFabcdef")));
    // Целое от 1 до 100.
    d.add(InputLine::new(Rect::new(3, 6, 15, 7), 3).with_validator(RangeValidator::new(1, 100)));
    // Одно из двух слов.
    d.add(InputLine::new(Rect::new(3, 8, 15, 9), 10).with_validator(StringLookupValidator::new(&["да",
"нет"])));
}
```

Аналоги: `TValidator` и его наследники.

`FilterValidator` — только разрешённые символы

Что это. Пропускает только символы из заданного набора — остальные не набираются.

Когда нужен. Телефоны, коды, числа без проверки диапазона.

```
use tvision::*;

fn filter() {
    let v = FilterValidator::new("0123456789+-() ");
    let mut good = "+7 (900)".to_string();
    assert!(v.is_valid_input(&mut good, false));
    let mut bad = "+7 abc".to_string();
    assert!(!v.is_valid_input(&mut bad, false)); // буквы не пройдут
}
```

Набор символов — поле `valid_chars`.

`RangeValidator` — число в диапазоне

Что это. Целое от `min` до `max`. Пока набирают, пропускает только цифры (и минус, если `min` меньше нуля); при ОК проверяет диапазон.

```
use tvision::*;

fn range() {
    let v = RangeValidator::new(1, 100);
    assert!(v.is_valid("42"));
    assert!(!v.is_valid("0"));
    assert!(!v.is_valid("101"));
}
```

Что важно знать. Значение поля остаётся **строкой** (`Value::Str`). Если нужно сразу число, удобнее `InputLong`.

`PictureValidator` — маска

Что это. Значение должно соответствовать **маске**: `##.##.####` — две цифры, точка, две цифры, точка, четыре цифры. Второй аргумент — `true`: точки, скобки и другие постоянные символы дописываются сами.

```
use tvision::*;

fn masks() {
    let date = PictureValidator::new("##.##.####", true);
    assert!(date.is_valid("05.11.2026"));
    assert!(!date.is_valid("05.11")); // не до конца
    assert!(!date.is_valid("ab.11.2026")); // не цифры

    let yes_no = PictureValidator::new("{Да,Нет}", true); // одно из слов
    assert!(yes_no.is_valid("Да"));

    let broken = PictureValidator::new("[##", false); // ошибка в самой маске
    assert!(!broken.status_ok());
}
```

Символы маски:

Символ	Значит
`#`	цифра
`?`	буква
`&`	буква, станет заглавной
`@`	любой символ
`!`	любой символ, станет заглавным
`*`	повтор: `*3#` — три цифры, `*#` — сколько угодно цифр
`[ ]`	необязательная часть: `#[#]` — одна или две цифры
`{ }`	группа
`,`	«или»: `{Да,Нет}`
`;`	следующий символ — как есть: `;#` — сам знак «#»
всё остальное	постоянный символ (точка, скобка, дефис)

Готовые маски:

Маска	Для чего
`##.##.####`	дата `05.11.2026`
`(###) ###-##-##`	телефон
`#[#][#]`	число из 1–3 цифр
`&*?`	слово с заглавной буквы: «иван» → «Иван»
`*#[.##]`	число с необязательными двумя знаками после точки

Что важно знать. Пустое значение маску **не проходит**. Если поле необязательное, используйте `FnValidator` с проверкой `s.is_empty() || …`.

Реже: `picture(&mut строка, дописывать) -> PicResult` — разобрать строку по маске самому; `status_ok()` — правильна ли маска; текст ошибки — `validate::PICTURE_ERROR`.

`PicResult`

Результат разбора маски: `Complete` (строка полностью подходит), `Incomplete` (подходит, но не до конца — можно продолжать набор), `Empty`, `Error` (не подходит), `Syntax` (ошибка в самой маске), `Ambiguous`, `IncompNoFill`.

`StringLookupValidator` — значение из списка

Что это. При OK проверяет, что введено одно из разрешённых значений: `StringLookupValidator::new(&["да", "нет"])`. Набирать можно что угодно — проверяется только итог. Список — поле `strings`.

Если хочется **выбирать** из списка, а не печатать, — возьмите `Combo` с флагом `cbx::ONLY_LIST`.

`FnValidator` — своя проверка функцией

Что это. Проверка любой функцией: `f` получает текст и возвращает `true`, если всё хорошо; `message` — что сказать, если нет. Срабатывает при OK.

Когда нужен. Почти всегда, когда готовые не подходят: «не пусто», «есть @», «файл существует».

```
use tvision::*;

fn email() -> InputLine {
    InputLine::new(Rect::new(3, 2, 40, 3), 60).with_validator(FnValidator {
        f: |s: &str| s.is_empty() || (s.contains('@') && s.contains('.')),
        message: "Введите адрес вида имя@домен.ru".into(),
    })
}
```

`Validator` — свой валидатор

Что это. Трейт, который реализуют все валидаторы. Свой нужен, если надо проверять и **по ходу набора** (`FnValidator` проверяет только при OK) или исправлять ввод.

Метод	Когда	Что вернуть
`is_valid_input(&self, s, suppress_fill)`	после каждого нажатия	`false` — отменить нажатие; можно изменить `s`
`is_valid(&self, s)`	при OK	`false` — показать сообщение

Метод	Когда	Что вернуть
`error_message(&self)`	для сообщения	текст ошибки (обязательный метод)
`status_ok(&self)`	при создании окна	правилен ли сам валидатор

`suppress_fill = true` приходит, когда пользователь **стирает**: в этот момент ничего не дописывайте, иначе стереть не получится.

```rust
use tvision::*;

/// Логин: латиница и цифры, 3–16 символов, всегда маленькими буквами.
struct Login;

impl Validator for Login {
    fn is_valid_input(&self, s: &mut String, _suppress_fill: bool) -> bool {
        *s = s.to_lowercase(); // исправляем ввод на лету
        s.chars().all(|c| c.is_ascii_alphanumeric()) && s.chars().count() <= 16
    }
    fn is_valid(&self, s: &str) -> bool {
        (3..=16).contains(&s.chars().count())
    }
    fn error_message(&self) -> String {
        "Логин: от 3 до 16 латинских букв и цифр".into()
    }
}

fn login() {
    let mut s = "Ivan".to_string();
    assert!(Login.is_valid_input(&mut s, false));
    assert_eq!(s, "ivan");
    assert!(!Login.is_valid("iv")); // короче трёх
    let _field = InputLine::new(Rect::new(3, 2, 22, 3), 16).with_validator(Login);
}
```

Справочник: элементы управления

Всё, что кладут в диалоги и окна. Координаты элементов считаются от левого верхнего угла окна или диалога; строка 0 и столбец 0 — рамка (урок 4).

На этой странице: `Button` · `StaticText` · `Label` · `InputLine` · `InputLong` · `Cluster`: флажки и переключатели · `ScrollBar` · `ListBox` · `History` · `Combo` · `ProgressBar` · окна сообщений

`Button` — кнопка

Что это. Кнопка с надписью. Нажатие (щелчком, пробелом, горячей буквой или Enter для кнопки по умолчанию) посылает команду.

Аналог: `TButton`.

```
use tvision::*;

const CM_APPLY: u16 = 150;

fn buttons(d: &mut Dialog) {
    d.add(Button::new(Rect::new(4, 10, 16, 12), "O~K~", cm::OK, bf::DEFAULT)); // по Enter
    d.add(Button::new(Rect::new(18, 10, 32, 12), "~П~рименить", CM_APPLY, bf::NORMAL));
    d.add(Button::new(Rect::new(34, 10, 46, 12), "Отмена", cm::CANCEL, bf::NORMAL)); // и по Esc
}
```

Как создать. `Button::new(прямоугольник, надпись, команда, флаги)`. Высота прямоугольника — **2**: вторая строка — тень кнопки.

Что важно знать.

- `cm::OK`, `cm::CANCEL`, `cm::YES`, `cm::NO` **закрывают** диалог. Другие команды идут дальше — в `AppHandler` (или в `modal_hook`, если диалог модальный).
- `bf::DEFAULT` — кнопка «по умолчанию», её нажимает Enter. Такая кнопка в диалоге одна.
- Если команда кнопки запрещена (`disable_command`), кнопка серая и не нажимается.

Флаги `bf`: `NORMAL`, `DEFAULT`, `LEFT_JUST` (надпись слева, а не по центру), `BROADCAST` (вместо команды разослать сообщение соседям — чтобы кнопка «поговорила» с другими элементами, не закрывая диалог), `GRAB_FOCUS` (забирать фокус при нажатии мышью).

Поля и методы: `title`, `command`, `flags`, `am_default`, `press(ctx)` — нажать из программы.

`StaticText` — текст

Что это. Надпись. Длинный текст сам переносится по словам; `\n` — новая строка; `\x03` в начале — по центру.

Аналог: `TStaticText` (и `TColoredText` из Dialog Design).

```rust
use tvision::*;

fn texts(d: &mut Dialog) {
    d.add(StaticText::new(Rect::new(2, 2, 40, 4), "\x03Длинный текст переносится по словам в пределах прямоугольника."));
    d.add(StaticText::colored(Rect::new(2, 5, 40, 6), "Внимание!", color::attr(14, 4))); // жёлтый на красном
}
```

- `StaticText::new(прямоугольник, текст)` — цвет как у текста диалога;
- `StaticText::colored(прямоугольник, текст, цвет)` — свой цвет;
- поля `text` и `attr` можно менять на ходу: `app.find_mut::<StaticText>(id).unwrap().text = "…".into()`.

`ColoredText` — другое имя того же типа.

`Label` — подпись

Что это. Подпись к полю. Щелчок по ней или Alt+горячая буква переводят курсор в поле, а пока поле в фокусе, подпись подсвечена.

Аналог: `TLabel`.

```rust
use tvision::*;

fn labeled(d: &mut Dialog) {
    let name = d.add(InputLine::new(Rect::new(12, 2, 40, 3), 60));
    d.add(Label::new(Rect::new(2, 2, 11, 3), "~И~мя:", Some(name))); // Alt+И → в поле
}
```

`Label::new(прямоугольник, текст, id_поля)`. Поля: `text`, `link`, `light`.

`InputLine` — строка ввода

Что это. Поле для одной строки текста. Умеет выделение мышью и Shift+стрелками, копирование и вставку (в том числе из системного буфера обмена), режим замены (Ins), историю и проверку.

Аналог: `TInputLine`.

```rust
use tvision::*;

fn input(d: &mut Dialog) -> ViewId {
    let mut il = InputLine::new(Rect::new(3, 2, 30, 3), 80); // не длиннее 80 символов
    il.set_text("42"); // начальный текст (выделен — набор заменит его)
    d.add(il)
}

fn read(app: &Program, id: ViewId) -> String {
    app.find::<InputLine>(id).map(|il| il.text()).unwrap_or_default()
}
```

Метод / поле	Что
`InputLine::new(прямоугольник, макс_длина)`	высота — 1; длинный текст прокручивается
`text()`, `set_text(s)`	текст
`with_validator(v)`	проверка ввода

Метод / поле	Что
`history_id`	номер списка истории
данные для диалога	`Value::Str(текст)`

Реже: `cursor_pos()`, `set_cursor_pos(i)`, `select_all(вкл)`, `validator`.

Клавиши: ←/→, Home/End, Ctrl+←/→ (по словам), Shift+стрелки (выделить), Backspace, Del, Ins (вставка/замена), Ctrl+Y (очистить), Ctrl+Ins (копировать), Shift+Ins (вставить), Shift+Del (вырезать).

`InputLong` — поле для числа

Что это. Строка ввода для целого числа в заданных пределах. Значение — сразу число (`Value::Int`), а не строка.

Аналог: `TInputLong` из Dialog Design.

```
use tvision::*;

fn number(d: &mut Dialog) -> ViewId {
    // макс. длина 8, от −100 до 1000, можно вводить «$1F»; "Счётчик" — для сообщения об ошибке
    let n = d.add(InputLong::new(Rect::new(14, 2, 24, 3), 8, -100, 1000, il::HEX, "~С~чётчик"));
    d.add(Label::new(Rect::new(2, 2, 13, 3), "~С~чётчик", Some(n)));
    n
}

fn values() {
    let mut f = InputLong::new(Rect::new(0, 0, 10, 1), 8, 0, 100, 0, "N");
    f.set_value(500); // больше предела — обрежется
    assert_eq!(f.value(), 100);
    assert_eq!(parse_long("$1F"), Some(31)); // «$» — шестнадцатеричное
}
```

`InputLong::new(прямоугольник, макс_длина, мин, макс, флаги, название)`. Методы `value()`, `set_value(v)`, `range_check()`. Флаги `il`: `HEX` (можно `$1F`), `BLANK_EQ_ZERO` (пусто = 0), `DISPLAY_HEX` (показывать в шестнадцатеричном виде). Функция `parse_long(s)` разбирает число.

`Cluster` — флажки и переключатели

Что это. Группа вариантов. Один тип — три вида:

Как создать	Что это	Значение
`Cluster::check_boxes(прямоугольник, &надписи)`	флажки `[X]` — сколько угодно	биты: бит `i` — отмечен вариант `i`
`Cluster::radio_buttons(прямоугольник, &надписи)`	переключатели `(•)` — ровно один	номер выбранного
`Cluster::multi_check_boxes(…)`	флажки с несколькими состояниями (`[ ]`, `[+]`, `[-]`)	по несколько бит на флажок

Аналоги: `TCheckBoxes`, `TRadioButtons`, `TMultiCheckBoxes`. Имена `CheckBoxes`, `RadioButtons`, `MultiCheckBoxes` тоже есть — это тот же `Cluster`.

```rust
use tvision::*;

fn clusters(d: &mut Dialog) {
    // 4 флажка в прямоугольнике высотой 2 — два столбца по 2.
    d.add(Cluster::check_boxes(Rect::new(3, 2, 30, 4), &["~К~ниги", "~С~порт", "~М~узыка",
"~И~гры"]));
    d.add(Cluster::radio_buttons(Rect::new(32, 2, 50, 4), &["~Д~а", "~Н~ет"]));
}

fn marks() {
    let mut c = Cluster::check_boxes(Rect::new(0, 0, 20, 2), &["a", "b"]);
    c.press(1); // отметить второй
    assert!(c.mark(1) && !c.mark(0));
    assert_eq!(c.value, 0b10);
}
```

Что важно знать.

- Высота прямоугольника — сколько вариантов в одном столбце. Если вариантов больше, они переходят в следующий столбец.
- Клавиши: стрелки — между вариантами, пробел — отметить, горячая буква — сразу нужный вариант.
- Проверить флажок `i` по значению: `value & (1 << i) != 0`.

Поля и методы: `value`, `sel` (вариант под курсором), `strings`, `mark(i)`, `press(i)`, `set_button_state(маска, вкл)` (сделать варианты недоступными), `button_state(i)`. **Реже:** `kind` (`ClusterKind`), `enable_mask`, `multi_mark(i)`; `Cluster::multi_check_boxes(прямоугольник, &надписи, состояний, флаги, символы)` — флаги `cf::ONE_BIT`, `TWO_BITS`, `FOUR_BITS`, `EIGHT_BITS` (сколько бит на флажок), символы — по одному на состояние, например `" +-"`.

`ScrollBar` — полоса прокрутки

Что это. Полоса с бегунком: показывает, какая часть списка видна, и позволяет прокручивать мышью.

Аналог: `TScrollBar`.

Как использовать. Полосу создают и кладут **в то же окно или диалог**, что и список, а её id передают списку:

```rust
let sb = d.add(ScrollBar::new(Rect::new(30, 2, 31, 8)));   // шириной 1 — вертикальная
let list = ListBox::new(Rect::new(3, 2, 30, 8), 1, Some(sb));
```

В окне удобнее `window.standard_scroll_bar(true)` — полоса сразу на правой рамке.

Поля `value`, `min`, `max`, `pg_step`, `ar_step` и метод `set_params(…)` — для своих прокручиваемых представлений; списки, таблицы и редактор управляют полосой сами.

`ListBox` — список

Что это. Список строк с выбором. Может быть в несколько столбцов.

Аналог: `TListBox`.

```rust
use tvision::*;

fn languages(d: &mut Dialog) -> ViewId {
    let sb = d.add(ScrollBar::new(Rect::new(30, 2, 31, 8)));
    let mut lb = ListBox::new(Rect::new(3, 2, 30, 8), 2, Some(sb)); // 2 столбца
    lb.set_items(["Rust", "C++", "Pascal", "Go", "Zig"].iter().map(|s| s.to_string()).collect());
    lb.focus_item(2); // выбрать «Pascal»
    d.add(lb)
}

fn chosen(d: &Dialog, id: ViewId) -> Option<String> {
    d.win.group.find::<ListBox>(id)?.selected_text().map(String::from)
}
```

Метод / поле	Что
`ListBox::new(прямоугольник, столбцов, полоса)`	
`set_items(строки)`	содержимое
`focused`, `focus_item(i)`	номер выбранной строки
`selected_text()`	текст выбранной строки
данные для диалога	`Value::Int(номер)`

Реже: `items`, `top_item`, `range()`, `palette`. Пустой список показывает `<empty>` (`listbox::EMPTY_TEXT`, с русским языком — «<пусто>»).

Что важно знать. Enter и двойной щелчок рассылают сообщение `cm::LIST_ITEM_SELECTED` соседям; буква — переход к строке на эту букву. **Список сам не растягивается** с окном — задайте `grow_mode = gf::GROW_HI_X | gf::GROW_HI_Y`.

`History` — история ввода

Что это. Маленький значок `▐↓▌` справа от строки ввода. Щелчок по нему (или ↓ в строке) открывает список того, что вводили раньше.

Аналог: `THistory`.

```rust
use tvision::*;

fn with_history(d: &mut Dialog) {
    let mut il = InputLine::new(Rect::new(3, 2, 30, 3), 80);
    il.history_id = Some(5); // номер списка истории — любой, но одинаковый у значка
    let id = d.add(il);
    d.add(History::new(Rect::new(30, 2, 33, 3), id, 5)); // значок 3×1 справа
}

fn store() {
    history_add(9, "первый");
    history_add(9, "второй");
    assert_eq!(history_str(9, 0).as_deref(), Some("второй")); // 0 — самое свежее
    assert_eq!(history_count(9), 2);
}
```

Что важно знать. Строки запоминаются, когда нажата кнопка диалога или фокус ушёл из поля. В каждом списке не больше 32 строк (`history::HISTORY_LIMIT`), повтор переносится в начало.

Функции: `history_add(номер, строка)`, `history_count(номер)`, `history_str(номер, i)`, `history_list(номер)`, `clear_history()`. **Реже:** `HistoryWindow` — само всплывающее окно со списком (`HistoryWindow::new(границы, номер)`, `selection()`); значок открывает его сам.

`Combo` — выпадающий список

Что это. Значок рядом со строкой ввода, который открывает список готовых вариантов. Выбранный вариант попадает в строку.

Аналог: `TCombo` из Dialog Design.

```
use tvision::*;

fn city(d: &mut Dialog) {
    let il = d.add(InputLine::new(Rect::new(3, 2, 25, 3), 30));
    let mut cb = Combo::new(Rect::new(25, 2, 26, 3), il, cbx::ONLY_LIST, &["Москва", "Тверь",
"Казань"]);
    cb.activate_char('▼'); // свой значок
    d.add(cb);
}
```

`Combo::new(прямоугольник, id_строки, флаги, &варианты)`. Флаги `cbx`: `ONLY_LIST` — ввести можно **только** вариант из списка (буквы ищут по списку); `NO_TRANSFER` — не участвовать в обмене данными диалога; `DISPOSES_LIST` — для совместимости. Методы: `new_list(варианты)`, `activate_char(c)`; поля `list`, `show_char`, `activate`. **Реже:** `ComboWindow` — открывающееся окно списка (`selection()`).

`ProgressBar` — полоса выполнения

Что это. Полоса, которая показывает, сколько работы сделано. В оригинальном Turbo Vision её нет.

```
use tvision::*;

fn bars(d: &mut Dialog) {
    d.add(ProgressBar::new(Rect::new(3, 2, 43, 3), 200)); // от 0 до 200, по центру «25%»
    d.add(ProgressBar::new(Rect::new(3, 4, 43, 5),
10).with_style(ProgressStyle::Blocks).with_caption("{value} из {max} файлов"));
    d.add(ProgressBar::new(Rect::new(3, 6, 43, 7), 0).indeterminate()); // «бегающий» блок
}

fn values() {
    let mut p = ProgressBar::new(Rect::new(0, 0, 20, 1), 200);
    p.set_value(50);
    assert_eq!(p.percent(), 25);
    p.step(1000); // больше максимума — обрежется
    assert!(p.is_done());
}
```

Метод	Что
`ProgressBar::new(прямоугольник, максимум)`	полоса от 0 до максимума
`set_value(v)`, `step(шаг)`, `value()`	значение
`percent()`, `fraction()`, `is_done()`	сколько сделано
`with_caption("{value} из {max}")`	своя надпись: `{value}`, `{min}`, `{max}`, `{percent}`
`with_style(ProgressStyle::Smooth / Blocks)`	плавная или блоками ▉
`indeterminate()`	когда сколько осталось — неизвестно

Реже: `with_range(мин, макс)`, `set_range`, `with_colors(пустая, заполненная)` (нужно, если полоса в обычном окне, а не в диалоге), `text()`, поля `style`, `caption`, `show_text`, `indeterminate`, `colors`; палитра `progress::PROGRESS_PALETTE`.

Как обновлять во время долгой работы. В модальном диалоге — функцией `exec_dialog_idle` (пример). Если работа идёт в отдельном потоке — см. главу 7 и примеры `examples/progress.rs`, `examples/json_load.rs`. Ещё один способ — сообщение `ctx.message(cm::PROGRESS_SET, Info::Progress { id, value })`.

Окна сообщений

Что это. Готовые маленькие диалоги: сообщение с кнопками и окно для ввода одной строки.

Аналоги: `messageBox`, `inputBox`.

```
use tvision::*;

fn dialogs(app: &mut Program) {
    // Вопрос: функция возвращает команду нажатой кнопки.
    if message_box(app, "\x03Удалить файл?", mf::CONFIRMATION | mf::YES_BUTTON | mf::NO_BUTTON) ==
cm::YES {
        // …удалить…
    }
    // Сообщение побольше — своего размера.
    message_box_rect(app, Rect::new(0, 0, 60, 12), "Длинное сообщение\nв несколько строк", mf::ERROR |
mf::OK_BUTTON, true);
    // Ввод строки: Some(текст) — нажали OK, None — отмена.
    if let Some(name) = input_box(app, "Новый файл", "~И~мя:", "без имени.txt", 100) {
        let _ = name;
    }
}
```

Функция	Что делает
`message_box(app, текст, флаги) -> команда`	сообщение 40×9 по центру
`message_box_rect(app, прямоугольник, текст, флаги, по_центру)`	сообщение своего размера
`input_box(app, заголовок, подпись, начальный_текст, макс_длина)`	строка ввода; `Some(текст)` или `None`
`build_message_box(прямоугольник, текст, флаги) -> Dialog`	собрать, не показывая (например, для `ctx.exec` из представления)

Флаги `mf`. Заголовок: `INFORMATION` («Информация»), `WARNING`, `ERROR`, `CONFIRMATION`. Кнопки: `OK_BUTTON`, `YES_BUTTON`, `NO_BUTTON`, `CANCEL_BUTTON`; сочетания `YES_NO_CANCEL`, `OK_CANCEL`. Флаги складываются знаком `|`. Первая кнопка — «по умолчанию» (Enter).

Что важно знать. Текст переносится по словам; `\n` — новая строка; `\x03` в начале — по центру. Надписи кнопок и заголовков по-русски — после `locale::set_language(Language::Ru)`; сами тексты хранятся в `MsgBoxText`.

Справочник: таблица, дерево, прокрутка

Представления для большого количества данных.

На этой странице: `Table` · `TableModel` · `VecTableModel` · `FnTableModel` · `TableColumn`, `Align` · `SelectionMode` · `TableColors` · `TreeView` · `TreeNode` · `TreePath`, `TreeLoader` · `CheckState` · `TreeGlyphs` · `TreeColors` · `Outline`, `Node` · `Scroller` · `Terminal`

`Table` — таблица

Что это. Таблица со строкой заголовков, линиями между столбцами, прокруткой, сортировкой щелчком по заголовку, правкой ячеек и выделением строк. В оригинальном Turbo Vision её нет.

Когда нужна. Когда у данных несколько столбцов: контакты, файлы, записи из базы. Для одного столбца хватит `ListBox`.

Таблица не хранит данные сама — она берёт их у **модели** (`TableModel`) и спрашивает только видимые ячейки. Поэтому даже миллион строк показывается быстро.

```
use tvision::*;

fn people_window(app: &mut Program) -> ViewId {
    let mut w = Window::new(Rect::new(0, 0, 60, 16), "Люди", 1);
    let v = w.standard_scroll_bar(true);
    let h = w.standard_scroll_bar(false);
    let rows = vec![
        vec!["Анна".to_string(), "34".to_string(), "Тула".to_string()],
        vec!["Борис".to_string(), "27".to_string(), "Казань".to_string()],
    ];
    let model = VecTableModel::new(&["Имя", "Возраст", "Город"], rows);
    let mut t = Table::new(w.interior(), Box::new(model), Some(v), Some(h))
        .with_columns(vec![TableColumn::new(16), TableColumn::new(8).align(Align::Right),
TableColumn::new(16)])
        .row_numbers(true) // столбец с номерами строк
        .striped(true); // строки через одну другого цвета
    t.vs.grow_mode = gf::GROW_HI_X | gf::GROW_HI_Y; // растягиваться вместе с окном
    let id = w.add(t);
    app.insert_window(w);
    id
}

/// Текст ячейки под курсором.
fn current_cell(app: &Program, id: ViewId) -> Option<String> {
    let t = app.find::<Table>(id)?;
    let (row, col) = t.cursor();
    Some(t.model().cell(row, col))
}
```

Как создать. `Table::new(прямоугольник, Box::new(модель), вертикальная_полоса, горизонтальная_полоса)`, затем настройки цепочкой.

Настройки (цепочкой при создании или полями потом):

Метод (поле)	Что включает	По умолчанию
`with_columns(v)` (`columns`)	ширина и выравнивание столбцов	ширина 12
`header(вкл)` (`show_header`)	строка заголовков	да

Метод (поле)	Что включает	По умолчанию
`grid(вкл)` (`show_grid`)	линии между столбцами	да
`row_numbers(вкл)` (`show_row_numbers`)	номера строк слева	нет
`striped(вкл)` (`zebra`)	«зебра»	нет
`selection(SelectionMode::Row)` (`selection_mode`)	курсор на всю строку	на ячейку
`sortable(вкл)` (`allow_sort`)	сортировка щелчком по заголовку	да
`editable(вкл)` (`allow_edit`)	правка ячеек (F2, Enter или буква), если модель разрешает	нет
`multi_select(вкл)` (`multi_select`)	выделение нескольких строк (Shift+стрелки, Ctrl+щелчок, пробел)	нет
`frozen(n)` (`frozen_cols`)	первые `n` столбцов не уезжают при прокрутке вбок	0
`resizable(вкл)` (`allow_resize`)	менять ширину столбцов мышью	да
`with_colors(c)` (`colors`)	свои цвета	по окну

Главные методы:

Метод	Что делает
`cursor() -> (строка, столбец)`, `set_cursor(строка, столбец)`	курсор
`model()`, `model_mut()`	модель — чтобы прочитать или изменить данные
`set_model(m)`	заменить модель целиком
`refresh()`	**после изменения данных**: поправить курсор и прокрутку
`auto_width(n)`	подобрать ширину столбцов по первым `n` строкам
`selected_rows()`	выделенные строки (при `multi_select`)

Реже: `set_columns(v)`, `resize_column(столбец, изменение)`, `sort_by(столбец, по_возрастанию)`, `sort_state()`, `select_rows(&строки)`, `clear_selection()`, `is_editing()`, `copy_text()`, `row_number_base`.

Что важно знать.

- **Таблица сама не растягивается** с окном — задайте `grow_mode`, как в примере.
- При сортировке щелчком таблица переставляет строки **модели**. Если вы храните данные ещё и отдельно (как в уроке 9), номера строк разойдутся — либо выключите сортировку (`sortable(false)`), либо держите данные только в модели.
- Enter таблица не забирает: в диалоге он нажмёт кнопку по умолчанию, в окне — вернётся к окну (урок 9).
- Сообщения соседям: `cm::TABLE_CURSOR_MOVED` (курсор сдвинулся), `cm::TABLE_CELL_EDITED` (ячейку изменили), `cm::LIST_ITEM_SELECTED` (Enter или двойной щелчок).
- Ctrl+C и Ctrl+Ins копируют ячейку или выделенные строки.

Подробнее — глава 7, пример — `cargo run --example table`.

`TableModel` — откуда таблица берёт данные

Что это. Трейт: таблица спрашивает у модели число строк и столбцов и текст каждой видимой ячейки. Свою модель пишут, когда данные удобнее вычислять или брать из своего хранилища, а не копировать в `Vec<Vec<String>>`.

Метод	Обязателен	Что вернуть
`row_count()`, `col_count()`	да	размеры
`cell(строка, столбец)`	да	текст ячейки
`header(столбец)`	нет	заголовок (по умолчанию `A`, `B`, …)
`editable(строка, столбец)`	нет	можно ли править (по умолчанию нет)
`set_cell(строка, столбец, текст)`	нет	записать; `false` — не принято
`sort(столбец, по_возрастанию)`	нет	отсортировать строки; `false` — не умею

```
use tvision::*;

/// Таблица умножения 1000×1000 — данные не хранятся, а вычисляются.
struct Multiply;

impl TableModel for Multiply {
    fn row_count(&self) -> usize {
        1000
    }
    fn col_count(&self) -> usize {
        1000
    }
    fn cell(&self, r: usize, c: usize) -> String {
        ((r + 1) * (c + 1)).to_string()
    }
    fn header(&self, c: usize) -> String {
        (c + 1).to_string()
    }
}

fn model() {
    assert_eq!(Multiply.cell(2, 3), "12");
    let _t = Table::new(Rect::new(0, 0, 40, 10), Box::new(Multiply), None, None);
}
```

`VecTableModel` — данные в памяти

Что это. Готовая модель: заголовки и строки как `Vec<Vec<String>>`. `VecTableModel::new(&["Имя", "Возраст"], строки)`; `.editable(true)` — разрешить правку. Сортирует сама (числа — как числа). Поля `headers`, `rows`, `editable`: после изменения `rows` через `model_mut()` вызовите `table.refresh()`.

`FnTableModel` — ячейки из функции

Что это. Самый короткий способ сделать «вычисляемую» таблицу без своего типа: `FnTableModel::new(строк, столбцов, |строка, столбец| текст)` и `.headers(&["…"])`.

```
use tvision::*;

fn squares() -> Table {
    let m = FnTableModel::new(1_000_000, 2, |r, c| if c == 0 { r.to_string() } else { (r *
r).to_string() }).headers(&["n", "n²"]);
    Table::new(Rect::new(0, 0, 30, 10), Box::new(m), None, None)
}
```

Функции модуля: `column_letters(i)` — имя столбца как в Excel (0 → `A`, 26 → `AA`); `natural_cmp(a, b)` — сравнение «как числа, если обе строки — числа».

`TableColumn` и `Align` — столбцы

`TableColumn::new(ширина)` и цепочкой: `.title("…")`, `.align(Align::Left / Center / Right)`, `.limits(мин, макс)` (пределы ширины при растягивании мышью), `.visible(false)` (скрыть), `.resizable(false)`, `.sortable(false)`.

`SelectionMode`

`Cell` — курсор на одной ячейке, ←/→ ходят по столбцам; `Row` — курсор на всю строку, ←/→ прокручивают вбок.

`TableColors`

Свои цвета таблицы: `normal`, `zebra`, `header`, `header_sorted`, `cursor`, `cursor_inactive`, `selected`, `grid`, `row_number`. Обычно не нужны — таблица берёт цвета у окна. Запасные — `table::DEFAULT_TABLE_COLORS`.

`TreeView` — дерево

Что это. Дерево с раскрывающимися ветками: `▸` — свёрнуто, `▾` — раскрыто. Умеет загружать ветки только при раскрытии, флажки, быстрый поиск набором букв. В оригинале его нет.

Когда нужно. Каталоги, разделы документа, структура JSON, настройки по группам. Полное описание — глава 18, пример — `cargo run --example treeview`.

```
use tvision::*;

fn project_tree(r: Rect) -> TreeView {
    let roots = vec![
        TreeNode::new("src")
            .expanded(true) // раскрыта сразу
            .child(TreeNode::new("main.rs"))
            .child(TreeNode::new("widgets").child(TreeNode::new("tree.rs"))),
        TreeNode::new("Cargo.toml"),
    ];
    TreeView::new(r, roots, None, None)
}

fn tree_ops() {
    let mut t = project_tree(Rect::new(0, 0, 30, 10));
    assert_eq!(t.row_count(), 4); // src, main.rs, widgets (свёрнута), Cargo.toml
    assert!(t.select_path(&[0, 1, 0])); // встать на tree.rs (widgets раскроется)
    assert_eq!(t.current().unwrap().text, "tree.rs");
    t.push(Some(&[0]), TreeNode::new("lib.rs")); // добавить в src
    assert!(t.find_text("lib.rs").is_some());
}
```

Клавиши: ↑ ↓ — по строкам; → — раскрыть или к первому вложенному; ← — свернуть или к родителю; `+` `-` `*` — раскрыть, свернуть, раскрыть всё внутри; Enter — раскрыть или «выбрать» (на листе); пробел — флажок; буквы — поиск.

Главные методы:

Метод	Что делает
`TreeView::new(прямоугольник, узлы, вертикальная_полоса, горизонтальная_полоса)`	дерево
`current()`, `cursor_path()`	узел под курсором и путь к нему
`select_path(путь)`	поставить курсор на узел (раскрыв всё выше)
`node(путь)`, `node_mut(путь)`	узел по пути
`push(родитель, узел)`, `insert(родитель, место, узел)`, `remove(путь)`	добавить, вставить, удалить
`expand(путь)`, `collapse(путь)`, `expand_all()`, `collapse_all()`	раскрыть, свернуть
`find_text(текст)`, `find(|узел| …)`	найти узел
`refresh()`	после правки узлов через `node_mut`

Настройки цепочкой: `.checkboxes(true)` — флажки; `.lines(false)` — без линий веток; `.with_loader(функция)` — загрузка веток при раскрытии; `.enter_toggles(false)` — Enter всегда «выбирает»; `.glyphs(…)`, `.with_colors(…)`.

Реже: `roots()`, `roots_mut()`, `set_roots(v)`, `path_texts(путь)` (тексты от корня), `for_each(f)`, `toggle`, `set_expanded`, `expand_subtree(путь, глубина)`, `row_count()`, `row_path(i)`, `cursor_row()`, `set_cursor_row(i)`, `check_state(путь)`, `set_checked(путь, вкл)`, `toggle_check(путь)`, `checked_paths()` (отмеченные листья); поля `show_lines`, `checkboxes`, `enter_toggles`, `glyphs`, `colors`, `v_scroll`, `h_scroll`.

Сообщения соседям: `cm::TREE_CURSOR_MOVED`, `cm::TREE_NODE_TOGGLED`, `cm::TREE_CHECK_CHANGED`, `cm::LIST_ITEM_SELECTED`.

`TreeNode` — узел дерева

Что это. Один узел: текст, вложенные узлы и признаки. Собирается цепочкой:

Метод	Что задаёт
`TreeNode::new(текст)`	узел
`.child(узел)`, `.children(vec)`	вложенные
`.expanded(true)`	раскрыт сразу
`.lazy()`	вложенные загрузит функция `TreeLoader` при раскрытии
`.checked(true)`	флажок отмечен
`.icon('{')`	значок перед текстом
`.color(цвет)`	свой цвет текста
`.data("…")`	ваши данные: путь к файлу, ключ записи

Поля открыты: `text`, `children`, `expanded`, `lazy`, `checked`, `icon`, `color`, `data`. Помощники: `has_children()`, `check_state()`, `set_checked_deep(вкл)`, `count()`.

`TreePath` и `TreeLoader`

`TreePath` — путь к узлу: номера по порядку от корня (`Vec<usize>`). `[0]` — первый узел верхнего уровня, `[0, 2]` — его третий вложенный.

`TreeLoader` — функция, которая возвращает вложенные узлы для узла с `.lazy()`. Вызывается один раз — при первом раскрытии. Так дерево каталогов открывается мгновенно: читается только то, что раскрыли.

```
use tvision::*;

/// Бесконечное дерево: у каждого узла по три вложенных, создаются по требованию.
fn infinite(r: Rect) -> TreeView {
    TreeView::new(r, vec![TreeNode::new("корень").lazy()], None, None)
        .with_loader(|_path, node| (1..=3).map(|i| TreeNode::new(&format!("{}.{}", node.text,
i)).lazy()).collect())
}
```

`CheckState`

Состояние флажка: `Checked` (отмечен), `Unchecked`, `Partial` (отмечена только часть вложенных — показывается `[-]`).

`TreeGlyphs`

Символы, которыми рисуется дерево: линии `│ ├ └ ─`, стрелки `▸ ▾`, содержимое флажков. Готовые наборы: `TreeGlyphs::UNICODE` (по умолчанию) и `TreeGlyphs::ASCII` — для терминалов без псевдографики.

`TreeColors`

Свои цвета дерева: `normal`, `cursor`, `cursor_inactive`, `lines`. Обычно не нужны — дерево берёт цвета у окна. Запасные — `treeview::DEFAULT_TREE_COLORS`.

`Outline` и `Node` — дерево из оригинала

Что это. Перевод `TOutline` из Turbo Vision — для программ, переносимых с C++ или Pascal. Для новых программ удобнее `TreeView`.

```
use tvision::*;

fn outline(r: Rect) -> Outline {
    let root = Node::with_children("Проект", vec![Node::new("main.rs"), Node::with_children("docs",
vec![Node::new("guide.md")], false)], true);
    Outline::new(r, None, None, vec![root])
}
```

`Node::new(текст)`, `Node::with_children(текст, вложенные, раскрыт)`; `Outline::new(прямоугольник, горизонтальная_полоса, вертикальная_полоса, узлы)` — обратите внимание: сначала **горизонтальная**. Поля `roots`, `foc`; методы `update()` (после правки `roots`), `row_count()`, `node_at(i)`, `focused_node()`, `focused_path()`, `set_expanded(i, вкл)`, `expand_all(i)`. Служебное: флаги `outline::ov::*`, функция `create_graph`.

`Scroller` — прокрутка для своих представлений

Что это. Помощник для своего представления, которое больше экрана (длинный текст, большая картинка): хранит, насколько прокручено (`delta`), и связывает представление с полосами прокрутки.

Аналог: `TScroller` (там от него наследовались, здесь его кладут полем в свою структуру).

```
use tvision::*;

/// Просмотр длинного текста.
pub struct TextView {
    pub vs: ViewState,
    pub scroller: Scroller,
    pub lines: Vec<String>,
}

impl TextView {
    pub fn new(r: Rect, h: Option<ViewId>, v: Option<ViewId>, lines: Vec<String>) -> Self {
        let mut vs = ViewState::new(r);
        vs.options |= of::SELECTABLE;
        let mut scroller = Scroller::new(h, v);
        let w = lines.iter().map(|l| text::str_width(l)).max().unwrap_or(0) as i32;
        scroller.set_limit(w, lines.len() as i32, r.size()); // размер содержимого
        TextView { vs, scroller, lines }
    }
}

impl View for TextView {
    tvision::impl_view_basics!();

    fn draw(&self, dc: &mut DrawCtx) {
        let a = dc.color(1);
        let d = self.scroller.delta; // насколько прокручено
        let w = self.vs.size.x.max(0) as usize;
        for y in 0..self.vs.size.y {
            let mut b = DrawBuffer::new(w);
            b.move_char(0, ' ', a, w);
            if let Some(l) = self.lines.get((d.y + y) as usize) {
                // строка данных = прокрутка + строка на экране
                let s: String = l.chars().skip(d.x.max(0) as usize).collect();
                b.move_str_limited(0, &s, a, w);
            }
            dc.write_line(0, y, self.vs.size.x, 1, &b);
        }
    }

    fn palette(&self) -> &'static [u8] {
        color::palettes::SCROLLER
    }

    fn handle_event(&mut self, ev: &mut Event, ctx: &mut Ctx) {
        view::default_handle_event(&self.vs, ev, ctx);
        let size = self.vs.size;
        self.scroller.handle(ev, size, self.vs.is_focused()); // стрелки, колесо, полосы
        self.scroller.sync(size, ctx); // передать полосам новое положение
    }
}
```

Метод / поле	Что
`Scroller::new(горизонтальная, вертикальная)`	id полос (или `None`)
`set_limit(ширина, высота, размер_окна)`	размер содержимого
`delta`	насколько прокручено (столбцов, строк)
`handle(ev, размер, в_фокусе)`	обработать прокрутку

Метод / поле	Что
`sync(размер, ctx)`	обновить полосы
`scroll_to(x, y, размер)`, `limit`	прокрутить в точку; размер содержимого

`Terminal` — окно журнала

Что это. Окно для вывода строк одна за другой, как в консоли: новые строки добавляются снизу, старые удаляются, когда их больше заданного числа. Удобно для журнала событий.

Аналог: `TTerminal`.

```
use tvision::*;

fn log_window(app: &mut Program) -> ViewId {
    let mut w = Window::new(Rect::new(0, 0, 60, 15), "Журнал", 0);
    let v = w.standard_scroll_bar(true);
    let log = w.add(Terminal::new(w.interior(), None, Some(v), 1000)); // хранить 1000 строк
    app.insert_window(w);
    log
}

fn log(app: &mut Program, id: ViewId, msg: &str) {
    if let Some(t) = app.find_mut::<Terminal>(id) {
        t.write_line(msg);
    }
}
```

Методы: `write_line(текст)`, `write(текст)` (`\n` — новая строка), `clear()`, `lines()`; поля `max_lines`, `scroller`.

Справочник: текстовый редактор

Многострочный редактор текста: курсор, выделение, отмена, поиск и замена, буфер обмена. Есть в трёх видах — сам редактор, поле для диалога и окно для файла.

На этой странице: `EditWindow` · `Editor` · `Pos` · `Memo` · `FileEditor` · `Indicator` · `ef`, `ec` · функции

Какой выбрать:

Нужно	Берите
открыть и править файл в окне	`EditWindow` — всё готово
несколько строк текста в диалоге (комментарий, адрес)	`Memo`
текст в своём окне, без файла	`Editor`

`EditWindow` — окно для файла

Что это. Готовое окно редактора файла: текст, полосы прокрутки, строка и столбец курсора в нижней рамке, имя файла в заголовке. Само сохраняет (F2 / `cm::SAVE`, «Сохранить как» — `cm::SAVE_AS`) и при закрытии изменённого файла спрашивает «Сохранить?».

Аналог: `TEditWindow`.

```
use std::path::Path;
use tvision::*;

fn open_file(app: &mut Program, file: &Path) -> ViewId {
    let r = app.desktop().group.vs.extent(); // во весь рабочий стол
    app.insert_window(EditWindow::new(r, file, 1)) // пустой путь — новый файл
}

fn text_of(app: &Program, id: ViewId) -> Option<String> {
    Some(app.find::<EditWindow>(id)?.editor().editor.text())
}
```

`EditWindow::new(прямоугольник, путь, номер)`. Методы `editor()` и `editor_mut()` — внутренний `FileEditor`; поля `win`, `editor_id`. Пример — `cargo run --example tvedit`.

`Editor` — редактор

Что это. Сам редактор текста. Остальные — `Memo`, `FileEditor`, `EditWindow` — построены на нём.

Аналог: `TEditor`.

```rust
use tvision::*;

fn notes_window(app: &mut Program) {
    let mut w = Window::new(Rect::new(5, 2, 65, 18), "Заметки", 1);
    let h = w.standard_scroll_bar(false);
    let v = w.standard_scroll_bar(true);
    let mut ed = Editor::new(w.interior(), Some(h), Some(v), None);
    ed.set_text("Пишите здесь…");
    w.add(ed);
    app.insert_window(w);
}

fn editing() {
    let mut ed = Editor::new(Rect::new(0, 0, 40, 10), None, None, None);
    ed.set_text("первая строка\nвторая строка");
    ed.set_select(Pos::new(1, 0), Pos::new(1, 6)); // выделить «вторая»
    assert_eq!(ed.selected_text(), "вторая");
    ed.insert_text("третья", false); // заменить выделенное
    assert_eq!(ed.line(1), "третья строка");
    ed.undo(); // отменить
    assert_eq!(ed.line(1), "вторая строка");
}
```

Метод	Что делает
`Editor::new(прямоугольник, горизонтальная, вертикальная, индикатор)`	редактор; полосы и индикатор — id (или `None`)
`text()`, `set_text(s)`	весь текст (строки через `\n`); `set_text` очищает историю отмены
`line_count()`, `line(i)`	число строк, строка
`cursor()`	где курсор (`Pos`)
`set_select(от, до)`	выделить (`set_select(p, p)` — просто поставить курсор)
`selected_text()`, `selection()`	выделенный текст и его границы
`insert_text(s, выделить)`	вставить на место курсора (выделенное заменится)
`undo()`, `redo()`	отменить, повторить
`search(что, флаги) -> bool`	найти от курсора и выделить
`modified`	изменён ли текст

Реже: `delete_select()`, `track_cursor(по_центру)` (прокрутить к курсору), `scroll_to(x, y)`, `col_to_x`, `x_to_col` (номер символа ↔ экранный столбец), поля `overwrite`, `auto_indent`, `can_undo`, `delta`, `find_str`, `replace_str`, `editor_flags`, `h_scroll`, `v_scroll`, `indicator`, `palette`.

Что важно знать.

- Редактор в фокусе сам выполняет **и разрешает** команды `cm::UNDO`, `REDO`, `CUT`, `COPY`, `PASTE`, `CLEAR`, `SELECT_ALL`, `FIND`, `REPLACE`, `SEARCH_AGAIN` — поставьте их в меню, и пункты будут серыми, когда действие невозможно.
- Отмена многоуровневая; вставка поверх выделения отменяется одним шагом.
- Клавиши — привычные (стрелки, Shift+стрелки, Ctrl+Ins/Shift+Ins, Ctrl+Z) и из Turbo Pascal (Ctrl+K B, Ctrl+Q F…). Полная таблица — глава 9.
- Данные для диалога — `Value::Str(весь текст)`.

`Pos` — место в тексте

Строка `line` и номер символа в ней `col` (оба с нуля). `Pos::new(строка, символ)`. Это **не экранные координаты**: табуляция занимает несколько клеток, а считается одним символом.

`Memo` — многострочное поле в диалоге

Что это. Редактор для диалога: Tab в нём переходит к следующему полю (а не вставляет табуляцию), данные — весь текст.

Аналог: `TMemo`.

```
use tvision::*;

fn comment_dialog() -> Dialog {
    let mut d = Dialog::centered(50, 14, "Комментарий");
    let v = d.add(ScrollBar::new(Rect::new(46, 2, 47, 10)));
    let memo = d.add(Memo::new(Rect::new(3, 2, 46, 10), None, Some(v)));
    d.add(Label::new(Rect::new(2, 1, 12, 2), "~Т~екст", Some(memo)));
    d.add(Button::new(Rect::new(13, 11, 25, 13), "О~К~", cm::OK, bf::DEFAULT));
    d.set_data(&Value::List(vec![Value::Str("Первая строка\nвторая".into())]));
    d
}
```

`Memo::new(прямоугольник, горизонтальная, вертикальная)`; внутренний редактор — поле `editor`.

`FileEditor` — редактор файла

Что это. Редактор, привязанный к файлу: загружает, сохраняет, спрашивает имя для «Сохранить как», а при закрытии изменённого текста — «Сохранить?». Обычно его не создают сами, а берут готовым в `EditWindow`.

Аналог: `TFileEditor`.

Метод / поле	Что
`FileEditor::new(прямоугольник, гор., верт., индикатор, путь)`	существующий файл загружается сразу
`load_file()`, `save_file()`	`true` — получилось, иначе текст ошибки в `last_error`
`file_name`	путь (пустой — «Без имени»)
`title()`	имя для заголовка окна
`editor`	сам редактор

`Indicator` — строка и столбец

Что это. Маленький индикатор `12:5` (строка:столбец) и `*`, если текст изменён, — обычно в нижней рамке окна редактора. `EditWindow` создаёт его сам. `Indicator::new(прямоугольник)`, поля `location`, `modified`; редактор обновляет его сам, если получил его id.

Аналог: `TIndicator`.

`ef` и `ec`

`ef` — флаги поиска и замены: `CASE_SENSITIVE` (учитывать регистр), `WHOLE_WORDS_ONLY` (только слова целиком), `PROMPT_ON_REPLACE` (спрашивать перед каждой заменой), `REPLACE_ALL`, `DO_REPLACE`, `BACKUP_FILES` (при сохранении оставлять копию `.bak`).

`ec` — внутренние команды редактора (`ec::CHAR_LEFT` и т. п.), как в оригинале. Обычно не нужны.

Функции модуля

- `editor::find_dialog(текст, флаги)`, `editor::replace_dialog(что, на_что, флаги)` — диалоги поиска и замены (редактор открывает их сам по `cm::FIND` и `cm::REPLACE`);
- `editor::display_width(s)` — ширина строки на экране с табуляциями;
- `editor::absolute(путь)` — полный путь;
- `editor::TAB_SIZE` — ширина табуляции (8), `editor::UNTITLED` — имя нового файла.

Справочник: готовые диалоги — файлы, цвета, справка

Диалоги, которые нужны почти каждой программе, уже написаны.

На этой странице:

- **Файлы:** `FileDialog` · `ChDirDialog` · части диалогов · функции
- **Цвета:** `ColorDialog` · `ColorGroup`, `ColorItem` · `Color256Dialog` · части диалогов
- **Справка:** `HelpFile` · `HelpWindow` · `HelpViewer` · `HelpTopic`, `Paragraph`, `CrossRef`

`FileDialog` — выбор файла

Что это. Стандартный диалог «Открыть» / «Сохранить как»: поле для имени, список файлов и каталогов, сведения о выбранном файле.

Аналог: `TFileDialog`.

```rust
use tvision::*;

fn open(app: &mut Program) -> Option<std::path::PathBuf> {
    // шаблон, заголовок, подпись поля, кнопки, номер истории
    let d = FileDialog::new("*.txt", "Открыть файл", "~И~мя", fd::OPEN_BUTTON, 100);
    let (cmd, d) = app.exec_dialog(d);
    (cmd != cm::CANCEL).then(|| d.file_name()) // полный путь
}
```

`FileDialog::new(шаблон, заголовок, подпись, кнопки, номер_истории)`:

- **шаблон** — какие файлы показывать: `"*.txt"`, `"*"`;
- **кнопки**: `fd::OPEN_BUTTON` («Открыть»), `fd::OK_BUTTON` («ОК» — для «Сохранить как»), `fd::REPLACE_BUTTON`, `fd::CLEAR_BUTTON`, `fd::HELP_BUTTON`; «Отмена» есть всегда;
- **номер истории** — диалог запоминает введённые имена; у каждого диалога свой номер.

Что важно знать.

- Проверяйте `cmd != cm::CANCEL`, а не `cmd == cm::OK`: кнопки «Открыть» и «ОК» возвращают `cm::FILE_OPEN`, «Заменить» — `cm::FILE_REPLACE`.
- Если ввести в поле шаблон (`*.rs`) или каталог и нажать Enter, диалог перейдёт туда и останется открытым.
- `d.file_name()` — полный путь к выбранному файлу.

Реже: поля `wildcard`, `directory`, `dlg`; методы `read_directory()`, `FileDialog::normalize(путь)`; флаг `fd::NO_LOAD_DIR`.

`ChDirDialog` — смена каталога

Что это. Диалог выбора каталога с деревом каталогов. На ОК меняет текущий каталог программы.

Аналог: `TChDirDialog`.

```rust
use tvision::*;

fn change_dir(app: &mut Program) -> Option<std::path::PathBuf> {
    let (cmd, d) = app.exec_dialog(ChDirDialog::new(cd::NORMAL, 0));
    (cmd == cm::OK).then(|| d.directory())
}
```

`ChDirDialog::new(флаги, номер_истории)`; флаги `cd::NORMAL`, `cd::NO_LOAD_DIR`, `cd::HELP_BUTTON`; `directory()` — выбранный каталог.

Части файловых диалогов

Из этих частей собраны диалоги выше; отдельно нужны редко.

- **`FileList`** — список файлов в два столбца (`TFileList`): `set_entries(…)`, `focused_entry()`; поля `lb`, `entries`, `activated`.
- **`FileEntry`** — запись о файле: `name`, `is_dir`, `size`, `modified`; `display()` — как показывать в списке.
- **`FileInfoPane`** — строки сведений внизу диалога (`TFileInfoPane`): поля `path`, `entry`.
- **`DirListBox`** — дерево каталогов (`TDirListBox`): `new_directory(путь)`, `focused_path()`; поля `lb`, `paths`, `dir`, `cur`, `activated`.
- **`FileDialogText`** — надписи диалогов («Открыть», «Неверное имя файла»...); по-русски — с `locale::set_language(Language::Ru)`.

Функции для файлов

```rust
use tvision::widgets::filedlg::{is_wild, wild_match};

fn wild() {
    assert!(wild_match("*.rs", "main.rs")); // подходит ли имя под шаблон
    assert!(wild_match("a?c", "abc"));
    assert!(is_wild("*.txt") && !is_wild("a.txt")); // шаблон ли это
}
```

В модуле `tvision::widgets::filedlg`: `wild_match(шаблон, имя)`, `is_wild(s)`, `read_directory(каталог, шаблон)` (список файлов), `civil_time(время)` (год, месяц, день, час, минута).

`ColorDialog` — настройка цветов

Что это. Диалог, в котором пользователь сам перекрашивает программу: слева — что красить (рабочий стол, меню, окна, диалоги...), справа — цвета текста и фона и образец. Есть кнопки для выбора из 256 цветов.

Аналог: `TColorDialog`.

```rust
use tvision::*;

fn colors(app: &mut Program) {
    let (cmd, d) = app.exec_dialog(ColorDialog::new(&app.palette, standard_color_groups()));
    if cmd == cm::OK {
        app.palette = d.palette.clone(); // применить ко всей программе
    }
}
```

`ColorDialog::new(&палитра, группы)` правит **копию** палитры; чтобы применить, присвойте `d.palette` программе. `standard_color_groups()` — стандартный список того, что можно красить. Палитра — обычный `Vec<Attr>`, её легко сохранить в файл.

Реже: `current_attr()`, `set_current_attr(a)`, поля `groups`, `dlg`.

`ColorGroup` и `ColorItem`

Что это. Строки левой части диалога: **группа** («Меню», «Диалоги») и **элемент** внутри неё («Обычный текст», «Выбранный пункт») с номером цвета в палитре программы.

```
use tvision::*;

fn my_groups() -> Vec<ColorGroup> {
    let mut g = standard_color_groups();
    // Своя группа: пары «название, номер цвета в палитре программы».
    g.push(ColorGroup::new("Мой редактор", &[("Текст", 6), ("Выделение", 7)]));
    g
}
```

Поля: у `ColorGroup` — `name`, `items`, `index`; у `ColorItem` — `name`, `index`.

`Color256Dialog` — выбор одного цвета

Что это. Сетка всех 256 цветов: выбрать цвет текста или фона.

```
use tvision::*;

fn pick_background(app: &mut Program, a: Attr) -> Option<Attr> {
    let (cmd, d) = app.exec_dialog(Color256Dialog::new(ColorSel::Background, a));
    (cmd == cm::OK).then(|| d.result()) // `a` с новым фоном
}
```

`Color256Dialog::new(что_выбираем, исходный_цвет)`; `ColorSel::Foreground` или `ColorSel::Background`. `result()` — исходный цвет с выбранным, `color()` — номер выбранного (0–255).

Части цветовых диалогов

- **`ColorSelector`** — сетка 16 цветов (`TColorSelector`): `ColorSelector::new(прямоугольник, ColorSel::…)`, поле `color`.
- **`ColorSel`** — что выбирать: `Foreground` (текст) или `Background` (фон).
- **`ColorDisplay`** — образец текста выбранным цветом (`TColorDisplay`): поля `text`, `color`.
- **`Color256Selector`** — сетка 256 цветов без диалога: `Color256Selector::new(x, y, цвет)`, поле `color`, константы `COLS`, `ROWS`.
- **`ColorDialogText`** — надписи диалогов (переводятся вместе с языком).

`HelpFile` — файл справки

Что это. Справка программы: темы с текстом и ссылками друг на друга. Пишется простым текстом — программа читает его сама.

Аналог: `THelpFile` (в оригинале справку нужно было компилировать утилитой `tvhc`; здесь не нужно).

```rust
use tvision::*;

const SRC: &str = "\
.topic NoContext=0
 Моя программа
Строки без пробела в начале собираются в абзацы. Ссылка: {окна:Windows}.

.topic Windows=10
 Окна
Окна можно двигать мышью.
";

fn help() {
    let h = HelpFile::parse(SRC).expect("ошибка в справке");
    assert!(h.topic_for_context(10).is_some()); // тема с номером 10
    assert!(h.warnings.is_empty()); // ссылки ведут на существующие темы
}
```

Как писать справку:

- `.topic Имя=номер` — начало темы. **Номер** — тот же, что в `vs.help_ctx` элемента: по F1 откроется тема элемента в фокусе;
- строка, которая начинается **без пробела**, — часть абзаца (абзацы переносятся по ширине окна); **с пробелом** — выводится как есть (для заголовков и таблиц);
- `{текст:Тема}` — ссылка на тему `Тема` с текстом `текст`; `{Тема}` — ссылка с именем темы.

Справку удобно хранить в файле и встроить в программу: `HelpFile::parse(include_str!("help.txt"))`. Подробно — глава 10.

Методы: `HelpFile::parse(текст)`, `HelpFile::load(путь)`, `topic_for_context(номер)`, `topic_by_name(имя)`. **Реже:** поля `topics`, `symbols`, `warnings`; `context_constants()` (константы `HC_…` для кода), `HelpFile::invalid_topic()`.

`HelpWindow` — окно справки

Что это. Окно, которое показывает тему и даёт ходить по ссылкам: Tab — к следующей ссылке, Enter — перейти, Esc — закрыть.

Аналог: `THelpWindow`.

```rust
use std::rc::Rc;
use tvision::*;

struct App {
    help: Rc<HelpFile>,
}

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        if ev.command() == Some(cm::HELP) {
            let topic = app.help_ctx(); // номер темы элемента в фокусе
            app.exec_dialog(HelpWindow::new(self.help.clone(), topic));
            ev.clear();
        }
    }
}
```

`HelpWindow::new(справка, номер_темы)`. Чтобы F1 работал везде, добавьте в строку состояния `StatusItem::new("~F1~ Справка", kb::F1, cm::HELP)`, а для открытых диалогов обработайте `cm::HELP` ещё и в `app.modal_hook`. **Реже:** `viewer()`, поля `win`, `viewer`; заголовок — `help::HELP_WIN_TITLE`.

`HelpViewer`

Что это. Сам просмотр темы внутри окна справки — если нужно встроить справку в своё окно. `HelpViewer::new(прямоугольник, гор., верт., справка, номер_темы)`, `switch_to(имя_темы)`; для тестов — `text_lines()`, `selected_target()`; поля `file`, `scroller`.

`HelpTopic`, `Paragraph`, `CrossRef`

Как справка устроена внутри: **тема** (`THelpTopic`) — абзацы `paragraphs` и ссылки `cross_refs`; **абзац** (`TParagraph`) — `text` и `wrap` (переносить ли); **ссылка** (`TCrossRef`) — где в тексте (`offset`, `length`) и на какую тему (`target`). Тема-заглушка «справки нет» — `help::NO_HELP`.

Справочник: файловые панели и операции с файлами

Для программ вроде Norton Commander или Far: панели со списком файлов и копирование в фоне с полосой выполнения. Большой пример — `cargo run --example lfm` (файловый менеджер), подробности — глава 17.

На этой странице: `FilePanel` · `DualPanel` · `PanelEntry` · `FileKind` · `FileClass` · `FileFilter` · `SortMode` · `FilePanelColors` · функции панелей · `FileJob` · `JobKind` · `Conflict` · `JobProgress` · функции `fileops`

`FilePanel` — файловая панель

Что это. Список файлов каталога: ходить по каталогам, выделять файлы, сортировать, искать набором букв, открывать несколько вкладок. Файлы раскрашены по типам (архивы, картинки, исходники).

```
use std::path::Path;
use tvision::*;

fn panel_window(app: &mut Program, dir: &Path) -> ViewId {
    let mut w = Window::new(Rect::new(0, 0, 50, 20), "Файлы", 1);
    let mut p = FilePanel::new(w.interior(), dir);
    p.vs.grow_mode = gf::GROW_HI_X | gf::GROW_HI_Y; // растягиваться вместе с окном
    p.set_sort(SortMode::Time, true); // новые сверху
    p.set_filter("*.o,*.tmp"); // эти файлы скрыть
    let id = w.add(p);
    app.insert_window(w);
    id
}

/// Выделенные файлы — или файл под курсором, если ничего не выделено.
fn selected(app: &Program, id: ViewId) -> Vec<std::path::PathBuf> {
    app.find::<FilePanel>(id).map(|p| p.targets()).unwrap_or_default()
}
```

Клавиши: стрелки, PgUp/PgDn, Home/End; Enter или → — войти в каталог, Backspace или ← — наверх; Ins или пробел — выделить файл; Ctrl+S или Alt+буква — поиск по имени; Ctrl+R — перечитать каталог.

Главные методы:

Метод	Что делает
`FilePanel::new(прямоугольник, каталог)`	панель
`path()`, `set_path(путь)`, `go_up()`	текущий каталог
`current()`	файл под курсором (`PanelEntry`)
`targets()`	выделенные файлы или файл под курсором
`marked()`, `toggle_mark()`, `mark_glob("*.rs", вкл)`, `clear_marks()`	выделение
`set_sort(вид, обратный)`	сортировка
`set_filter("*.o,*.tmp")`, `toggle_hidden()`	что скрыть
`refresh()`	перечитать каталог

Реже: `change_dir(путь, Some(имя))`, `enter()`, `entries()`, `cursor()`, `set_cursor(i)`, `select_name(имя)`, `select_path(путь)`, `quick_find(текст)`, `invert_marks()`, `filter()`, `resort()`, вкладки (`new_tab`, `close_tab`, `next_tab`, `prev_tab`, `select_tab`, `tab_count`, `active_tab`, `tab_paths`), `set_listing(…)` и `listing_title()` (показать произвольный список файлов, например результаты поиска), `calc_dir_sizes()`, `dir_size(путь)`, `history()`, `error()`, `set_message(s)`; поля `sort`, `sort_reverse`, `sort_mix_dirs`, `sort_case_sensitive`, `show_hidden`, `colors`; `filepanel::HISTORY_LIMIT`.

Что важно знать.

- Enter на **файле** ничего не открывает сам — панель посылает команду `cm::FILE_PANEL_EXECUTE`, а что делать с файлом, решает ваша программа (пример — в `DualPanel`).
- Панель сама не растягивается с окном — задайте `grow_mode`.
- Цвета берутся у окна, как у таблицы.

`DualPanel` — две панели

Что это. Две панели рядом, как в Norton Commander: Tab переключает между ними.

```
use std::path::Path;
use tvision::*;

struct App {
    panels: ViewId,
}

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        // Enter на файле в любой из панелей.
        if ev.command() == Some(cm::FILE_PANEL_EXECUTE) {
            let file = app.find::<DualPanel>(self.panels).and_then(|d| d.active().current().map(|e|
e.path.clone()));
            if let Some(f) = file {
                message_box(app, &format!("Открыть {}?", f.display()), mf::CONFIRMATION |
mf::YES_NO_CANCEL);
            }
            ev.clear();
        }
    }
}

fn start(app: &mut Program, left: &Path, right: &Path) -> App {
    let r = app.desktop().group.vs.extent();
    App { panels: app.insert_window(DualPanel::new(r, left, right)) }
}
```

Метод	Что делает
`DualPanel::new(прямоугольник, левый_каталог, правый_каталог)`	
`active()`, `active_mut()`	панель, в которой курсор
`other()`, `other_mut()`	другая панель (например, куда копировать)
`swap()`	поменять панели местами
`sync_other()`	открыть в другой панели тот же каталог
`toggle_single()`	одна панель или две

Реже: `left_id()`, `right_id()`, `active_id()`, `other_id()`, `panel_mut(id)`, `switch(ctx)`, `layout()`, поля `single`, `group`.

`PanelEntry` — файл в панели

Поля: `name`, `path`, `kind` (`FileKind`), `size`, `modified`, `mode` (права), `executable`, `hidden`. Методы: `is_dir()`, `is_parent()` (это `..`), `extension()`, `class()` (`FileClass`), `perms_string()` (`rwxr-xr-x`); `PanelEntry::from_path(путь, имя)` — создать без панели.

`FileKind`

Что это за запись: `Parent` (`..`), `Dir`, `File`, `Link { to_dir }`, `Other` (устройство, канал).

`FileClass`

Тип файла — для цвета: `Normal`, `Dir`, `Exec`, `Link`, `Archive`, `Image`, `Audio`, `Video`, `Document`, `Source`, `Data`, `Temp`. Определяется по расширению функцией `class_by_name(имя)`.

`FileFilter` — что скрыть

Что это. Шаблоны файлов, которые панель **не показывает**; `!шаблон` — исключение.

```rust
use tvision::*;

fn filters() {
    let f = FileFilter::parse("*.o,*.tmp,!keep*");
    assert!(f.hides("main.o"));
    assert!(!f.hides("keep.tmp")); // исключение
    assert!(!f.hides("main.rs"));
    assert_eq!(class_by_name("photo.JPG"), FileClass::Image);
}
```

Методы: `parse(s)`, `hides(имя)`, `is_empty()`, `text()`. Шаблоны: `*`, `?`, `[abc]`, `[a-z]`, `[!x]` (функция `glob_match(шаблон, имя, учитывать_регистр)`).

`SortMode`

Как сортировать: `Name` (по имени), `Ext` (по расширению), `Size`, `Time`, `Unsorted`. `letter()` — буква для рамки, `SortMode::ALL` — все виды.

`FilePanelColors`

Свои цвета панели (`frame`, `cursor`, `marked`, `classes` и другие). Обычно не нужны. `FilePanelColors::derive(рамка, рамка_активная, текст, курсор)` — собрать набор из четырёх цветов.

Функции панелей

Функция	Пример результата
`format_size_short(n)`	`12.3M` — размер в 7 символов
`format_size_full(n)`	`1 234 567`
`format_time_short(t)`	`05.11.26 14:30`
`format_time_full(t)`	`05.11.2026 14:30:00`
`available_roots()`	диски на Windows; `/`, домашний каталог, `/mnt/*`, `/media/*` на Unix

`FileJob` — копирование, перенос, удаление

Что это. Операция над файлами в **отдельном потоке**: программа не «замирает», можно показывать полосу выполнения и остановить в любой момент.

```
use std::path::PathBuf;
use tvision::fileops::{self, Conflict, FileJob, JobKind};
use tvision::*;

/// Скопировать с окном прогресса и кнопкой «Стоп».
fn copy(app: &mut Program, sources: Vec<PathBuf>, dest: PathBuf) -> fileops::JobProgress {
    let mut job = FileJob::start(JobKind::Copy, sources, Some(dest), Conflict::Skip);
    let mut d = Dialog::centered(50, 8, "Копирование");
    let bar = d.add(ProgressBar::new(Rect::new(2, 2, 48, 3), 100));
    d.add(Button::new(Rect::new(19, 4, 31, 6), "~С~топ", cm::CANCEL, bf::DEFAULT));
    // Пока диалог открыт, эта функция вызывается снова и снова.
    let (cmd, _) = app.exec_dialog_idle(d, |app, _| {
        let p = job.progress();
        app.find_mut::<ProgressBar>(bar)?.set_value(p.percent() as i64);
        p.finished.then_some(cm::OK) // закончилось — закрыть диалог
    });
    if cmd != cm::OK {
        job.cancel(); // нажали «Стоп»
    }
    job.wait() // итог: ошибки, пропущенные файлы
}

fn blocking() {
    let dir = std::env::temp_dir().join(format!("guide-fileops-{}", std::process::id()));
    std::fs::create_dir_all(dir.join("from")).unwrap();
    std::fs::write(dir.join("from/a.txt"), "привет").unwrap();
    // Без потока — например, в тесте:
    let r = FileJob::run_blocking(JobKind::Copy, vec![dir.join("from/a.txt")], Some(dir.clone()),
Conflict::Overwrite);
    assert!(r.errors.is_empty() && dir.join("a.txt").exists());
    let _ = fileops::remove_any(&dir);
}
```

Метод	Что делает
`FileJob::start(вид, файлы, куда, конфликт)`	начать в потоке (`куда` для удаления — `None`)
`progress()`	сколько сделано (`JobProgress`)
`cancel()`	остановить
`wait()`	дождаться и получить итог
`is_finished()`	закончилось ли
`FileJob::run_blocking(…)`	то же без потока

Что важно знать. «Куда» — как у команд `cp` и `mv`: если это существующий каталог, файлы кладутся внутрь, иначе (для одного файла) — это новое имя. Недокопированный файл при ошибке или остановке удаляется.

`JobKind`

`Copy` (копировать), `Move` (перенести), `Delete` (удалить).

`Conflict`

Что делать, если файл в месте назначения уже есть: `Overwrite` (заменить) или `Skip` (пропустить). Узнать заранее, какие файлы уже есть, — функция `fileops::conflicts`.

`JobProgress`

Снимок хода операции: `total_bytes`, `done_bytes`, `total_files`, `done_files`, `current` (какой файл сейчас), `errors`, `skipped`, `finished`, `cancelled`, `counting` (ещё считается объём); `percent()` — сколько процентов сделано.

Функции `fileops`

Функция	Что делает
`conflicts(файлы, куда)`	какие файлы в месте назначения уже есть
`target_path(файл, куда)`	куда попадёт файл
`dir_size(путь)`	размер каталога: (байт, файлов)
`is_inside(a, b)`	лежит ли `a` внутри `b` (нельзя копировать каталог в себя)
`remove_any(путь)`	удалить файл или каталог целиком

Справочник: терминал, буфер обмена, время, язык

Связь программы с операционной системой. Обычно всё это работает само — страница нужна, если вы пишете тесты, хотите свой буфер обмена или переводите надписи.

На этой странице: `Backend` · `RawEvent` · `default_backend` · `HeadlessBackend` · `UnixBackend`, `WindowsBackend` · `AnsiWriter`, `InputParser`, `Parsed` · `Clipboard` · `ClipboardBackend` · готовые буферы · `DateTime` · язык

`Backend` — терминал

Что это. Трейт, через который программа общается с терминалом: узнаёт размер экрана, получает нажатия и выводит готовый кадр. Сама программа о терминале больше ничего не знает — поэтому одна и та же программа работает в Linux, Windows, в тестах и даже в графическом окне.

Когда нужен. Почти никогда: `Program::with_terminal` выбирает терминал сам. Свой — если выводить нужно куда-то ещё (в окно, по сети).

```
use std::time::Duration;
use tvision::platform::{Backend, RawEvent};
use tvision::*;

/// Заготовка своего терминала.
struct MyBackend {
    size: Point,
}

impl Backend for MyBackend {
    fn size(&self) -> Point {
        self.size // размер экрана
    }
    fn read(&mut self, _timeout: Duration) -> Option<RawEvent> {
        None // ждать нажатие не дольше timeout; None — ничего не было
    }
    fn flush(&mut self, _screen: &ScreenBuffer, _cursor: Option<Point>, _insert: bool) {
        // показать кадр `screen`
    }
}

fn custom() {
    let mut app = Program::new(Box::new(MyBackend { size: Point::new(80, 25) }), None, None);
    app.pump(&mut program::NoHandler);
}
```

Необязательные методы: `suspend()`, `resume()` (вернуть терминал в обычный режим и обратно), `system_clipboard()`, `last_frame()`.

`RawEvent`

То, что приходит от терминала: `Key(KeyEvent)` (клавиша), `Mouse { pos, buttons, mods, wheel }` (что сейчас с мышью — двойной щелчок и прочее программа вычисляет сама), `Resize(размер)` (окно терминала изменило размер) и `Command(команда)` — команда от самого «экрана»: например, графическое окно tvision-gui присылает `cm::QUIT`, когда окно закрывают крестиком.

`default_backend`

`platform::default_backend()` — терминал для текущей системы. Его вызывает `Program::with_terminal`.

`HeadlessBackend` — невидимый терминал

Что это. Терминал для тестов: нажатия берутся из очереди, экран хранится в памяти. Подробно — урок 10.

```
use tvision::platform::HeadlessBackend;
use tvision::*;

fn headless() {
    let mut b = HeadlessBackend::new(80, 25);
    b.push_key(kb::F10); // открыть меню
    b.push_key(kb::ESC); // закрыть
    b.push_key(kb::ALT_X); // выйти
    let menu = Menu::new().submenu("~Ф~айл", Menu::new().item_key("~В~ыход", cm::QUIT, kb::ALT_X,
"Alt-X"));
    let mut app = Program::new(Box::new(b), Some(menu), None);
    app.pump(&mut program::NoHandler);
    assert!(!app.is_running()); // вышли
    assert!(app.screen_text().contains("Файл")); // меню на экране
}
```

Метод / поле	Что
`HeadlessBackend::new(ширина, высота)`	терминал
`push_key(клавиша)`, `push_click(x, y)`	добавить нажатие / щелчок в очередь
`input`	сама очередь — можно класть любые `RawEvent`
`esc_budget`	когда очередь пуста, терминал сам «нажимает» Esc, чтобы закрыть забытые окна; `0` — не нажимать

Реже: `frame`, `cursor`, `flushes`, `size`.

`UnixBackend` и `WindowsBackend`

Настоящие терминалы: `UnixBackend` — Linux, macOS, BSD; `WindowsBackend` — консоль Windows 10 и новее. Создаются сами через `with_terminal`. При завершении программы (и даже при панике) терминал возвращается в обычный вид.

`AnsiWriter`, `InputParser`, `Parsed`

Детали для тех, кто пишет свой терминал, понимающий ANSI-коды: `AnsiWriter` превращает кадр в коды (`render(…)`, `invalidate()`, `with_mode(…)`; функция `ansi::push_sgr`), `InputParser` разбирает байты от терминала (`parse(&байты) -> Parsed`: `Event(…)`, `Incomplete` — дочитать, `Skip(n)` — пропустить). Константы `ansi::ENTER_SEQ`, `ansi::LEAVE_SEQ`.

`Clipboard` — буфер обмена

Что это. Буфер обмена программы. Он связан с буфером операционной системы: скопированное в программе можно вставить в браузер, и наоборот. Строки ввода и редактор пользуются им сами (Ctrl+Ins, Shift+Ins).

```rust
use tvision::platform::clipboard::Clipboard;
use tvision::*;

fn clipboard(app: &mut Program) {
    app.ctx.clipboard.set("текст"); // скопировать
    let s = app.ctx.clipboard.get(); // вставить
    let _ = s;
}

fn internal() {
    let mut c = Clipboard::internal(); // только внутри программы
    c.set("привет");
    assert_eq!(c.get(), "привет");
}
```

Из обработчика приложения — `app.ctx.clipboard`, из представления — `ctx.clipboard`. Методы `get()`, `set(текст)`, `has_text()`, `backend_name()` (что используется: `"windows"`, `"wl-clipboard"`, `"osc52"`…). Создать: `Clipboard::system()` (с буфером системы), `Clipboard::internal()` (только свой), `Clipboard::with_backend(…)`.

Как связан с системой:

Система	Как
Windows	напрямую
macOS	программы `pbcopy` / `pbpaste`
Linux (Wayland)	`wl-copy` / `wl-paste`
Linux (X11)	`xclip` или `xsel`
нет ничего (например, по SSH)	копирование через терминал (OSC 52), вставка — из своего буфера

`ClipboardBackend` — свой буфер обмена

Что это. Трейт источника буфера: `get()`, `set(текст)`, `name()`. Свой — если буфер должен быть где-то ещё.

```rust
use tvision::platform::clipboard::{Clipboard, ClipboardBackend};

/// Буфер обмена в файле.
struct FileClipboard(std::path::PathBuf);

impl ClipboardBackend for FileClipboard {
    fn get(&mut self) -> Option<String> {
        std::fs::read_to_string(&self.0).ok()
    }
    fn set(&mut self, text: &str) -> bool {
        std::fs::write(&self.0, text).is_ok()
    }
    fn name(&self) -> &'static str {
        "file"
    }
}

fn use_file(app: &mut tvision::Program) {
    app.ctx.clipboard = Clipboard::with_backend(Box::new(FileClipboard("/tmp/clip.txt".into())));
}
```

Готовые буферы обмена

`CommandClipboard` — через программы `pbcopy`, `wl-copy`, `xclip`, `xsel` (`CommandClipboard::detect()` найдёт доступную); `Osc52Clipboard` — через терминал (только копирование); `WinClipboard` — Windows. Функция `clipboard::base64_encode` — служебная.

`DateTime` — местное время

Что это. Дата и время: поля `year`, `month`, `day`, `hour`, `minute`, `second`. Модуль `tvision::platform::time`.

```rust
use tvision::platform::time;

fn clock() -> String {
    let t = time::now();
    format!("{:02}.{:02}.{} {:02}:{:02}", t.day, t.month, t.year, t.hour, t.minute)
}

fn epoch() {
    let t = time::utc_from_secs(86_400); // сутки после 1 января 1970
    assert_eq!((t.year, t.month, t.day), (1970, 1, 2));
}
```

Функции: `time::now()` — сейчас, `time::local(момент)` — местное время для `SystemTime`, `time::utc_from_secs(секунды)`.

Язык интерфейса: `Language`

Что это. Язык надписей, которые рисует **сама библиотека**: кнопки окон сообщений («ОК», «Отмена»), файловые диалоги, сообщения валидаторов, справка. По умолчанию — английский. Надписи **вашей** программы вы пишете сами на любом языке.

```rust
use tvision::locale::{self, Language};

fn russian() {
    locale::set_language(Language::Ru); // обычно — первой строкой main
    assert_eq!(locale::tr("~Y~es"), "~Д~а");
    locale::set_language(Language::En);
}
```

Что	Описание
`Language`	`En` (английский), `Ru` (русский)
`locale::set_language(язык)`	выбрать
`locale::language()`	текущий
`locale::tr(текст)`	перевести надпись библиотеки (если перевода нет — вернёт как есть)

Горячие буквы работают в любой раскладке: Alt+Ф и Alt+A — одно и то же (глава 13).

Справочник: данные — JSON, SQLite, MVC

Чтение и запись JSON, работа с базой SQLite и каркас «модель — представление — контроллер» для программ, которые показывают и правят записи. Уроки — 11. MVC и 12. Рецепты, примеры — `mvc_contacts`, `mvc_todo`, `mvc_books`, `mvc_csv`, `sqlite_query`, `json_settings`.

На этой странице: `Json` · `parse_json` · `JsonError` · `Connection` · `SqlValue` · `SqlError`, `Rows` · `Schema` · `Field` · `FieldKind` · `Data` · `Record` · `Storage` · `JsonStorage` · `SqliteStorage` · `MemoryStorage` · `open_storage` · `Model` · `ModelWindow` · `ModelTableModel` · диалог записи · `CrudController`

`Json` — значение JSON

Что это. Разобранный JSON: модуль `tvision::json`. Внешних библиотек не нужно.

```
use tvision::json::{parse_json, Json};

fn read_settings() {
    let j = parse_json(r#"{"имя": "Аня", "размер": [80, 25], "тёмная": true}"#).unwrap();
    assert_eq!(j.get("имя").and_then(Json::as_str), Some("Аня"));
    assert_eq!(j.get("размер").and_then(|r| r.at(1)).and_then(Json::as_i64), Some(25));
    assert_eq!(j.get("тёмная").and_then(Json::as_bool), Some(true));
    assert!(j.get("нет такого").is_none());
}

fn write_settings() {
    let mut j = Json::object(vec![("имя", Json::from_str("Аня"))]);
    j.set("размер", Json::Arr(vec![Json::from_i64(80), Json::from_i64(25)]));
    assert_eq!(j.compact(), r#"{"имя": "Аня", "размер": [80, 25]}"#);
    let text = j.pretty(); // с отступами — для файла
    assert!(text.contains("\n  \"имя\""));
}
```

Варианты: `Null`, `Bool(bool)`, `Num(String)` (число хранится текстом — без потери точности), `Str(String)`, `Arr(Vec<Json>)`, `Obj(Vec<(String, Json)>)` (порядок ключей сохраняется).

Метод	Что делает
`get(ключ)`, `at(номер)`	поле объекта, элемент массива (`Option`)
`as_str()`, `as_i64()`, `as_f64()`, `as_bool()`	значение нужного вида (`None` — другой вид)
`as_array()`, `as_object()`, `is_null()`	
`Json::object(пары)`, `from_str`, `from_i64`, `from_f64`	построить
`set(ключ, значение)`	задать поле объекта
`compact()`, `pretty()`	текст в одну строку / с отступами

Реже: `write(…)` (в свой буфер), `cell_text()` (коротко для ячейки таблицы).

`parse_json` и `parse_json_with`

`parse_json(текст) -> Result<Json, JsonError>` — разобрать. `parse_json_with(текст, Some(&mut ход))` — то же, но функция `ход(сделано, всего) -> bool` вызывается по мере разбора (для полосы выполнения); вернула `false` — разбор прерывается (`Err(None)`). Пример с полосой выполнения — `examples/json_load.rs`.

`JsonError`

Ошибка разбора: поля `line`, `col` (с единицы) и `msg`; печатается как «строка 3, столбец 7: ожидалось ...».

`Connection` — база SQLite

Что это. Открытая база SQLite: модуль `tvision::sqlite`, только с признаком `sqlite` (`features = ["sqlite"]`). Работает через системную `libsqlite3` (в Windows — `winsqlite3.dll`); внешних крейтов нет.

```
use tvision::sqlite::{Connection, SqlValue};

fn books() {
    let db = Connection::open_in_memory().unwrap(); // файл: Connection::open("books.db")
    db.execute_batch(
        "CREATE TABLE books (id INTEGER PRIMARY KEY, title TEXT, year INTEGER);
         INSERT INTO books (title, year) VALUES ('Мастер и Маргарита', 1967);",
    )
    .unwrap();
    // Значения — отдельно от текста запроса, на месте «?».
    db.execute("INSERT INTO books (title, year) VALUES (?, ?)", &["Дюна".into(),
1965i64.into()]).unwrap();
    assert_eq!(db.last_insert_rowid(), 2);
    let rows = db.query("SELECT title FROM books WHERE year < ? ORDER BY year", &
[SqlValue::Int(1966)]).unwrap();
    assert_eq!(rows, vec![vec![SqlValue::Text("Дюна".into())]]);
}
```

Метод	Что делает
`Connection::open(путь)`	открыть или создать файл базы
`Connection::open_in_memory()`	база в памяти (для тестов)
`execute(sql, &[параметры])`	команда без результата; вернёт число изменённых строк
`execute_batch(sql)`	несколько команд через `;`, без параметров
`query(sql, &[параметры])`	строки результата `Vec<Vec<SqlValue>>`
`query_rows(sql, &[…])`	то же с именами столбцов (`Rows`)
`last_insert_rowid()`	номер последней вставленной строки
`changes()`	сколько строк изменила последняя команда (даже выполненная через `query`)
`transaction(|db| { … })`	всё внутри — или ничего: при ошибке изменения отменяются

Что важно знать.

- **Никогда не вклеивайте данные в текст запроса** через `format!` — кавычка в имени сломает запрос (а злоумышленник сможет выполнить свой SQL). Пишите `?` и передавайте значения вторым аргументом.
- Имя таблицы или столбца параметром передать нельзя; если оно не ваше, экранируйте функцией `sqlite::quote_ident(имя)`.
- База закрывается, когда `Connection` удаляется.

`SqlValue`

Значение ячейки: `Null`, `Int(i64)`, `Real(f64)`, `Text(String)`. Создаётся через `.into()` из `&str`, `String`, `i64`, `f64`, `bool` (→ 0/1), `Option<…>` (`None` → `Null`). Методы `to_text()`, `as_i64()`, `as_f64()`, `is_null()`.

`SqlError` и `Rows`

`SqlError` — ошибка SQLite: поля `code`, `msg`; печатается как «SQLite (1): near "SELEKT": syntax error». `Rows` — результат `query_rows`: поля `columns` (имена) и `rows`.

`Schema` — схема

Что это. Описание записи для модуля `tvision::mvc`: имя таблицы (для SQLite) и поля. Из схемы получаются столбцы таблицы, диалог правки и проверка ввода.

```
use tvision::mvc::*;

fn schema() -> Schema {
    Schema::new(
        "books",
        vec![
            Field::text("title", "~Н~азвание").required().width(30),
            Field::int("year", "~Г~од"),
            Field::real("price", "~Ц~ена"),
            Field::bool("read", "~П~рочитана"),
            Field::choice("shelf", "~П~олка", &["Дома", "На работе"]),
        ],
    )
}

fn schema_basics() {
    let s = schema();
    assert_eq!(s.index_of("year"), Some(1));
    let r = s.new_record(); // пустая запись: "", пусто, пусто, нет, «Дома»
    assert!(s.validate(&r).is_err()); // название обязательно
    let j = s.to_json(&Record::new(vec!["Дюна".into(), Data::Int(1965), Data::Null, true.into(),
"Дома".into()]));
    assert_eq!(j.compact(), r#"{"title": "Дюна", "year": 1965, "price": null, "read": true, "shelf":
"Дома"}"#);
}
```

Метод	Что делает
`Schema::new(таблица, поля)`	
`index_of(имя)`	номер поля
`new_record()`	пустая запись со значениями по умолчанию
`validate(&запись)`	проверить все поля
`to_json(&запись)`, `from_json(&json)`	запись ↔ объект JSON

Поля `table` и `fields` открыты.

`Field` — поле

Что это. Одно поле записи: имя для хранения (`name`), подпись (`title`, с `~горячей~` буквой), вид и правила.

```rust
use tvision::mvc::*;

fn fields() {
    let age = Field::int("age", "~В~озраст");
    assert_eq!(age.parse(" 42 ").unwrap(), Data::Int(42));
    assert_eq!(age.parse("").unwrap(), Data::Null); // необязательное — можно пусто
    assert_eq!(age.parse("сорок").unwrap_err(), "В поле «Возраст» нужно целое число.");

    let mail = Field::text("email", "Почта").check(|s| s.contains('@'), "В адресе нет «@».");
    assert!(mail.parse("a@b").is_ok() && mail.parse("ab").is_err());

    let vip = Field::bool("vip", "VIP");
    assert_eq!(vip.display(&Data::Bool(true)), "✓");
}
```

Создать	Вид
`Field::text(имя, подпись)`	строка
`Field::int(…)`	целое число
`Field::real(…)`	дробное (запятая или точка)
`Field::bool(…)`	да/нет
`Field::choice(…, &[варианты])`	один из вариантов (всегда обязателен)

Уточнение	Что делает
`.required()`	нельзя оставить пустым
`.max_len(n)`	длина строки ввода (по умолчанию 100)
`.width(n)`	ширина столбца таблицы
`.check(функция, сообщение)`	своя проверка текста; для пустого необязательного поля не вызывается

Реже: `parse(текст)` (текст → значение или сообщение об ошибке), `validate(&значение)`, `display(&значение)` (текст ячейки), `default_value()`, `plain_title()` (подпись без тильд), `options()`, `from_json(…)`; тип `FieldCheck`.

`FieldKind`

Вид поля: `Text`, `Int`, `Real`, `Bool`, `Choice(Vec<String>)`. Обычно задаётся конструктором `Field::…`.

`Data` — значение поля

`Null` (пусто), `Int(i64)`, `Real(f64)`, `Text(String)`, `Bool(bool)`. Создаётся через `.into()` из `&str`, `String`, `i64`, `f64`, `bool`. Методы: `as_str()`, `as_i64()`, `as_f64()`, `as_bool()`, `is_null()`, `to_text()` (для строки ввода), `to_json()`, `compare(&другое)` (для сортировки: пустые первыми, текст без учёта регистра).

`Record` — запись

Поля `id: Option<i64>` (номер в хранилище; `None` — ещё не сохранена) и `values: Vec<Data>` (по порядку полей схемы). `Record::new(значения)`, `get(i)` (значение), `text(i)` (значение текстом).

`Storage` — хранилище

Что это. Трейт «где лежат записи». Модель вызывает его методы и не знает формата. Каждый метод **сразу** сохраняет изменение.

Метод	Что делает
`load(&схема)`	прочитать все записи
`insert(&схема, &mut запись)`	добавить, записать новый номер в `запись.id`
`update(&схема, &запись)`	сохранить по `запись.id`
`delete(&схема, id)`	удалить
`name()`	имя для заголовка окна

Своё хранилище (CSV, сервер по сети) — реализуйте эти пять методов.

`JsonStorage`

Файл JSON — массив объектов с ключом `"id"`. `JsonStorage::new(путь)`, `path()`. Нет файла — нет записей; объект без `"id"` (или с повторяющимся) получает номер при чтении; ключи не из схемы сохраняются; файл переписывается целиком через временный файл.

`SqliteStorage`

Таблица `схема.table` в базе SQLite (признак `sqlite`). `SqliteStorage::open(путь)`, `SqliteStorage::in_memory()`, `connection()` — для своих запросов. Таблица создаётся сама (`id INTEGER PRIMARY KEY` + столбцы `TEXT`, `INTEGER`, `REAL`); недостающие столбцы добавляются. `Bool` хранится как 0/1.

`MemoryStorage`

Записи только в памяти — для тестов: `MemoryStorage::new()`, `MemoryStorage::with_records(записи)`.

`open_storage`

`open_storage(путь) -> Result<Box<dyn Storage>, String>` — хранилище по расширению: `.json` → `JsonStorage`; `.db`, `.sqlite`, `.sqlite3` → `SqliteStorage` (без признака `sqlite` — ошибка с подсказкой).

`Model` — модель

Что это. Записи в памяти + хранилище. Каждое изменение: проверка по схеме → сохранение → изменение в памяти. Если что-то не удалось, ничего не меняется.

```rust
use tvision::mvc::*;

fn model() {
    let schema = Schema::new("t", vec![Field::text("name", "Имя").required(), Field::int("n",
"Число")]);
    let mut m = Model::open(schema, Box::new(MemoryStorage::new())).unwrap();
    m.add(Record::new(vec!["Вера".into(), Data::Int(3)])).unwrap();
    m.add(Record::new(vec!["Анна".into(), Data::Int(1)])).unwrap();
    m.sort_by(0, true); // по имени
    let i = m.add(Record::new(vec!["Борис".into(), Data::Null])).unwrap();
    assert_eq!(i, 1); // встала между Анной и Верой
    assert_eq!(m.get(0).unwrap().id, Some(2)); // id — от хранилища
    assert!(m.add(Record::new(vec!["".into(), Data::Null])).is_err()); // имя обязательно
    m.remove(2).unwrap(); // Вера
    assert_eq!(m.len(), 2);
}
```

Метод	Что делает
`Model::open(схема, хранилище)`	создать и прочитать записи
`into_ref()`	завернуть в `ModelRef` (= `Rc<RefCell<Model>>`)
`len()`, `is_empty()`, `get(i)`, `records()`	записи
`add(запись)`	добавить; вернёт номер строки
`update(i, запись)`	заменить (id сохраняется); вернёт новый номер строки
`remove(i)`	удалить; вернёт удалённую запись
`sort_by(поле, по_возрастанию)`	сортировать (и потом держать порядок)
`reload()`	перечитать из хранилища

Реже: `schema()`, `index_of_id(id)`, `revision()` (растёт при каждом изменении), `storage_name()`, `sort_state()`, `unsort()`.

`ModelWindow` — окно с таблицей

Что это. Окно с таблицей записей модели. Таблица растягивается с окном, щелчок по заголовку сортирует модель, Enter и двойной щелчок посылают `CM_RECORD_EDIT`. Если модель изменилась (её изменили не через окно), окно обновляется само.

Метод / поле	Что
`ModelWindow::new(прямоугольник, заголовок, модель)`	окно; в заголовке ещё имя файла и число записей
`current()`	номер записи под курсором
`select(i)`	поставить курсор
`refresh()`	обновить таблицу и заголовок
`table()`, `table_mut()`	сама `Table`
поля `win`, `table`, `model`, `caption`	

`ModelTableModel` и `model_table`

`ModelTableModel { model }` — `TableModel` над моделью: для своего окна. `model_table(прямоугольник, модель, полоса_в, полоса_г)` — готовая `Table` со столбцами по схеме (ширина из `Field::width`, числа справа, `✓` по центру).

Диалог записи

Функция	Что делает
`edit_record(app, &схема, &запись, заголовок)`	показать диалог; `Some(запись)` — нажали OK
`record_dialog(&схема, &запись, заголовок)`	только собрать `Dialog` (строки ввода, флажки, переключатели по видам полей)
`record_value(&схема, &запись)`	данные для `Dialog::set_data`
`record_from_value(&схема, &value)`	запись из `Dialog::get_data` (или сообщение об ошибке)

Строки ввода проверяются при OK по правилам поля; ошибка — окно с сообщением, диалог остаётся открытым.

`CrudController` — контроллер

Что это. Команды «Добавить», «Изменить», «Удалить» для одного окна `ModelWindow`: показывает диалог, меняет модель, ставит курсор на изменённую запись, сообщает об ошибках хранилища.

```
use tvision::mvc::*;
use tvision::*;

struct App {
    crud: CrudController,
}

impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        self.crud.handle(app, ev);
    }
    fn idle(&mut self, app: &mut Program) {
        self.crud.update_commands(app);
    }
}

fn start(app: &mut Program) -> App {
    let schema = Schema::new("notes", vec![Field::text("text", "~Т~екст").required().max_len(200)]);
    let model = Model::open(schema, Box::new(MemoryStorage::new())).unwrap().into_ref();
    let r = app.desktop().group.vs.extent();
    let mut crud = CrudController::open(app, r, "Заметки", model);
    crud.add_title = "Новая заметка".into();
    crud.confirm_delete = false; // удалять без вопроса
    App { crud }
}
```

Метод / поле	Что
`CrudController::open(app, прямоугольник, заголовок, модель)`	вставить окно и создать контроллер
`CrudController::new(модель, id_окна)`	для окна, вставленного вами
`handle(app, ev) -> bool`	выполнить команду записи, если окно активно
`update_commands(app)`	разрешить/запретить команды по состоянию
`add(app)`, `edit(app)`, `delete(app)`	то же, что команды, — вызвать напрямую
`window(app)`, `is_active(app)`	окно контроллера; активно ли оно
поля `model`, `window`, `add_title`, `edit_title`, `confirm_delete`	

Команды: `CM_RECORD_ADD` (180), `CM_RECORD_EDIT` (181), `CM_RECORD_DELETE` (182). Свои команды программы нумеруйте так, чтобы не пересечься с ними.

Справочник: раскладки

Модуль `tvision::layout`: расстановка элементов без координат, как в Qt. Урок — 13. Раскладки, пример — `cargo run --example layout_demo`.

На этой странице: `Layout` · `HBox`, `VBox` · `Grid` · `Form` · `Item` · `Spacer` · `Policy`, `SizePolicy` · `Alignment` · `Margins` · окна и группы · свой элемент · как считается

`Layout` — раскладка

Что это. Описание того, как расставить элементы: строка, столбец, сетка или форма, с промежутками, полями и распорками. Создаётся построителями `HBox::new()`, `VBox::new()`, `Grid::new()`, `Form::new()`; методы вызываются цепочкой.

```
use tvision::layout::*;
use tvision::*;

fn settings() -> Dialog {
    let mut d = Dialog::centered(40, 10, "Параметры");
    let path = d.add(InputLine::new(Rect::default(), 60));
    let browse = d.add(Button::new(Rect::default(), "~О~бзор...", cm::OK, bf::NORMAL));
    let lp = d.add(Label::new(Rect::default(), "~П~апка:", Some(path)));
    let opts = d.add(Cluster::check_boxes(Rect::default(), &["~С~крытые файлы", "~П~одкаталоги"]));
    let ok = d.add(Button::new(Rect::default(), "О~К~", cm::OK, bf::DEFAULT));
    d.set_layout(
        VBox::new()
            .margin(1)
            .add(HBox::new().add(lp).add(path).add(browse)) // подпись, поле, кнопка — в строку
            .space(1)
            .add(opts)
            .stretch(1)
            .add(HBox::new().stretch(1).add(ok)),
    );
    d.adjust_size();
    d
}

fn check() {
    let d = settings();
    let ok = d.win.group.children.iter().rev().find(|c| c.as_any().is::<Button>()).unwrap();
    assert_eq!(ok.vs().bounds().b.x, d.win.group.vs.size.x - 2); // у правого края (рамка + поле)
}
```

Метод	Что делает
`add(элемент)`	добавить: `ViewId`, вложенную раскладку, `Spacer` или `item(…)`
`stretch(n)`	пружина с коэффициентом `n` (как `addStretch`)
`space(n)`	пустое место ровно `n` клеток (как `addSpacing`)
`spacing(n)`	промежуток между элементами (у сетки и формы — по обеим осям)
`hspacing(n)`, `vspacing(n)`	промежуток между столбцами / строками
`margins(слева, сверху, справа, снизу)`, `margin(n)`	поля вокруг
`size_hint(&группа)`, `min_size(&группа)`	желаемый и минимальный размер раскладки
`apply(&mut группа, прямоугольник)`	расставить один раз (обычно не нужно: это делает окно)
`geometry(&группа, прямоугольник)`	где окажется каждый элемент — без изменений (для тестов)

Методы сетки и формы — ниже.

`HBox` и `VBox` — строка и столбец

Что это. `HBox::new()` — элементы слева направо (`QHBoxLayout`), `VBox::new()` — сверху вниз (`QVBoxLayout`). Промежуток по умолчанию: в строке 1 клетка, в столбце 0.

```
use tvision::layout::*;
use tvision::*;

fn toolbar() {
    let mut g = Group::new(Rect::new(0, 0, 50, 2));
    let a = g.insert(Box::new(Button::new(Rect::default(), "~Н~овый", cm::OK, bf::NORMAL)));
    let b = g.insert(Box::new(Button::new(Rect::default(), "~О~ткрыть", cm::OK, bf::NORMAL)));
    let c = g.insert(Box::new(Button::new(Rect::default(), "~В~ыход", cm::QUIT, bf::NORMAL)));
    // Две кнопки слева, одна справа.
    g.set_layout(HBox::new().add(a).add(b).stretch(1).add(c));
    assert_eq!(g.view(c).unwrap().vs().bounds(), Rect::new(40, 0, 50, 2));
}
```

Типичные приёмы:

Чего хотим	Как
кнопки справа	`HBox::new().stretch(1).add(ok).add(cancel)`
кнопки по центру	`HBox::new().stretch(1).add(ok).add(cancel).stretch(1)`
кнопки внизу окна	в `VBox` перед рядом кнопок — `.stretch(1)`
подпись, поле, кнопка «...»	`HBox::new().add(label).add(input).add(button)` — строка ввода заберёт лишнее
два списка 1:2	`HBox::new().add(item(a).stretch(1)).add(item(b).stretch(2))`

`Grid` — сетка

Что это. Элементы по строкам и столбцам (`QGridLayout`); элемент может занимать несколько клеток. Ширина столбца — по самому широкому элементу в нём, высота строки — по самому высокому. Промежутки: 1 между столбцами, 0 между строками.

```
use tvision::layout::*;
use tvision::*;

fn grid() {
    let mut g = Group::new(Rect::new(0, 0, 41, 6));
    let b: Vec<ViewId> = (0..3).map(|_| g.insert(Box::new(Button::new(Rect::default(), "X", cm::OK,
bf::NORMAL)))).collect();
    let wide = g.insert(Box::new(Button::new(Rect::default(), "Широкая", cm::OK, bf::NORMAL)));
    g.set_layout(
        Grid::new()
            .cell(0, 0, item(b[0]).expand())
            .cell(0, 1, item(b[1]).expand())
            .cell(0, 2, item(b[2]).expand())
            .span(1, 0, 1, 3, item(wide).expand()) // на три столбца
            .col_stretch(0, 1)
            .col_stretch(1, 1)
            .col_stretch(2, 1),
    );
    assert_eq!(g.view(b[1]).unwrap().vs().bounds().a.x, 14); // (41 - 2) / 3 = 13 + промежуток
    assert_eq!(g.view(wide).unwrap().vs().bounds().b.x, 41);
}
```

Метод	Что делает
`cell(строка, столбец, элемент)`	элемент в одну клетку
`span(строка, столбец, строк, столбцов, элемент)`	на несколько клеток
`col_stretch(столбец, n)`, `row_stretch(строка, n)`	как делить лишнее место
`add(элемент)`	в новую строку, в столбец 0

Что важно знать. Элемент в клетке растягивается на всю клетку, только если его политика позволяет расти; иначе он желаемого размера в левом верхнем углу клетки (поменять — `align_h` / `align_v`). Пустые строки и столбцы места и промежутков не занимают.

`Form` — форма

Что это. Строки «подпись — поле» (`QFormLayout`): столбец подписей шириной по самой длинной, поля — на всю оставшуюся ширину. Промежутки: 1 между подписью и полем и 1 между строками — как в диалогах Turbo Vision.

Метод	Что делает
`row(подпись, поле)`	строка формы; поле — любой элемент или раскладка (например, `HBox` из строки ввода и кнопки истории)
`row_wide(элемент)`	элемент на обе колонки (флажок, пояснение)

Форма — это сетка из двух столбцов, поэтому для неё работают и методы сетки.

`Item` и `item` — уточнения элемента

Что это. Элемент раскладки с уточнениями. Обычно создаётся неявно (`.add(id)`), а явно — функцией `item(…)`, чтобы что-то поменять:

```
.add(item(table).stretch(2).min(20, 5))
.add(item(title).align_h(Alignment::Center))
.add(item(code_input).fixed_width(8))
```

Метод	Что делает
`stretch(n)`	коэффициент растяжения
`expand()`	тянуться во все стороны
`policy(по_ширине, по_высоте)`	своя политика вместо политики элемента
`hint(w, h)`	свой желаемый размер
`fixed(w, h)`, `fixed_width(w)`, `fixed_height(h)`	ровно такой размер
`min(w, h)`, `max(w, h)`, `min_width`, `max_width`, `min_height`, `max_height`	пределы
`align_h(a)`, `align_v(a)`, `center()`	выравнивание вместо растягивания (`Alignment`)

`Spacer` — распорка

Пустое место (`QSpacerItem`). В строках и столбцах удобнее `.stretch(n)` и `.space(n)`; `Spacer` — для сетки и особых случаев.

Создать	Что
`Spacer::horizontal()`	пружина по ширине
`Spacer::vertical()`	пружина по высоте
`Spacer::fixed(w, h)`	ровно `w × h`
`Spacer::new(w, h, политика_ш, политика_в)`	любая

Рядом с распорками промежуток раскладки не ставится.

`Policy` и `SizePolicy`

`Policy` — как элемент относится к месту по одной оси (`QSizePolicy::Policy`):

Значение	Растёт	Сжимается
`Fixed`	нет	нет
`Minimum`	да	нет (не меньше желаемого)
`Maximum`	нет	да
`Preferred`	да	до минимума (у элементов — до желаемого)
`Expanding`	да, **в первую очередь**	до минимума (у элементов — 5 по ширине, 3 по высоте)
`Ignored`	да, в первую очередь	желаемый размер не учитывается

`SizePolicy { h, v }` — пара политик. Готовые: `SizePolicy::FIXED`, `PREFERRED` (по умолчанию: ширина `Preferred`, высота `Fixed`), `EXPANDING_H` (строка ввода), `EXPANDING` (таблица, список).

`Alignment`

Как поставить элемент, если ему дали больше места, чем он хочет: `Fill` (по умолчанию: растянуть, если политика позволяет), `Start` (влево / вверх), `Center`, `End` (вправо / вниз). С `Start`, `Center` и `End` элемент всегда желаемого размера.

`Margins`

Поля раскладки: `left`, `top`, `right`, `bottom`. Задаются методами `margins(…)` и `margin(n)`.

Раскладка в окне и группе

Метод	Что делает
`Window::set_layout(раскладка)`, `Dialog::set_layout(…)`	расставлять элементы внутри рамки; пересчёт — при каждом изменении размера окна
`Window::adjust_size()`, `Dialog::adjust_size()`	размер окна по желаемому размеру раскладки (левый верхний угол на месте; диалог с `of::CENTERED` встанет по центру при открытии)
`Group::set_layout(раскладка)`	то же для любой группы, без отступа под рамку
`Group::apply_layout()`	пересчитать сейчас — после скрытия элемента или смены надписи
поле `Group::layout`	сама раскладка (`Option<Box<Layout>>`)

Что важно знать.

- Окно нельзя сделать меньше минимума раскладки (и меньше 16 × 6).
- Элементы, которых нет в раскладке, двигаются по своему `grow_mode`.
- Скрытые элементы (`sf::VISIBLE` снят) места не занимают.

Свой элемент в раскладке

Раскладка спрашивает у представления два метода трейта `View`:

```
impl View for Gauge {
    // …
    fn size_hint(&self) -> Option<Point> {
        Some(Point::new(self.label.chars().count() as i32 + 12, 1))   // желаемый размер
    }
    fn size_policy(&self) -> layout::SizePolicy {
        layout::SizePolicy::EXPANDING_H                               // тянуться по ширине
    }
}
```

`size_hint` по умолчанию — `None`: тогда желаемым считается размер, с которым элемент был создан. `size_policy` по умолчанию — `SizePolicy::PREFERRED`.

Как считаются размеры

1. **Измерение.** Для каждого элемента — желаемый размер, минимум, максимум и политика по обеим осям; у вложенной раскладки — сумма по длине и наибольшее поперёк, плюс промежутки и поля.
2. **Раздача места вдоль строки:** * есть коэффициенты растяжения — элементы без них получают желаемый размер, остальное делится по коэффициентам (не меньше минимумов, не больше максимумов); * иначе лишнее получают `Expanding`, а если таких нет — все, кто может расти; поровну, до своего максимума; * места не хватает — сжимаются сжимаемые, пропорционально запасу до минимума.
3. **Поперёк строки** элемент занимает всё место, если может расти и выравнивание `Fill`, иначе — желаемый размер с выравниванием.

Это тот же порядок, что у раскладок Qt, поэтому привычные приёмы (пружины, коэффициенты, `Expanding`) работают так же.

Указатель классов

A — Align 114 · AnsiWriter 134 · AppHandler 92 · Attr 68 · AttrPair 69 · Alignment 149

B — Backend 133 · Background 93 · Button 104

C — Cell 71 · ChDirDialog 123 · CheckBoxes 106 · CheckState 116 · Clipboard 135 · ClipboardBackend 135 · Cluster 106 · ClusterKind 106 · Color256Dialog 125 · Color256Selector 125 · ColorDialog 124 · ColorDialogText 125 · ColorDisplay 125 · ColoredText 104 · ColorGroup 125 · ColorItem 125 · ColorMode 70 · ColorSel 125 · ColorSelector 125 · Combo 109 · ComboWindow 109 · CommandSet 84 · Conflict 132 · Connection 139 · CrossRef 127 · CrudController 144 · Ctx 82

D — Data 141 · DateTime 136 · Desktop 92 · Dialog 94 · DialogPalette 95 · DirListBox 124 · DrawBuffer 71 · DrawCtx 84 · DualPanel 129

E — Editor 119 · EditWindow 119 · Event 73

F — FileClass 130 · FileDialog 123 · FileDialogText 124 · FileEditor 121 · FileEntry 124 · FileFilter 130 · FileInfoPane 124 · FileJob 131 · FileKind 130 · FileList 124 · FilePanel 128 · FilePanelColors 130 · Field 140 · FieldKind 141 · Form 148 · FilterValidator 100 · FnTableModel 113 · FnValidator 102 · Frame 94 · FrameInfo 94

G — Grid 147 · Group 85

H — HBox 147 · HeadlessBackend 134 · HelpFile 125 · HelpTopic 127 · HelpViewer 127 · HelpWindow 126 · HintFn 99 · History 108 · HistoryWindow 108

I — Indicator 121 · Info 76 · InputLine 105 · InputLong 106 · InputParser 134 · Item 148

J — JobKind 131 · JobProgress 132 · Json 138 · JsonError 139 · JsonStorage 142

K — Key 74 · KeyEvent 74

L — Label 105 · Language 136 · Layout 146 · ListBox 107

M — Margins 149 · Memo 121 · Menu 96 · MenuBar 97 · MenuItem 97 · MenuOutcome 98 · MenuSession 98 · MemoryStorage 142 · Message 76 · Model 142 · ModelTableModel 143 · ModelWindow 143 · Mods 74 · MouseEvent 75 · MsgBoxText 110 · MultiCheckBoxes 106

N — Node 116 · NoHandler 92

O — Outline 116

P — PanelEntry 130 · Paragraph 127 · Parsed 134 · Phase 84 · PicResult 102 · PictureValidator 101 · PlaceFn 84 · Point 67 · Policy 149 · Pos 121 · Program 90 · ProgressBar 109 · ProgressStyle 109

R — RadioButtons 106 · RangeValidator 101 · RawEvent 133 · Record 141 · Rect 67 · Request 84

S — Schema 140 · ScreenBuffer 71 · ScrollBar 107 · Scroller 117 · SelectionMode 114 · SizePolicy 149 · SortMode 130 · Spacer 148 · SqlError 140 · SqliteStorage 142 · SqlValue 140 · StaticText 104 · StatusDef 99 · StatusItem 99 · StatusLine 98 · Storage 142 · StringLookupValidator 102

T — `Table` 111 · `TableColors` 114 · `TableColumn` 114 · `TableModel` 113 · `Terminal` 118 · `TreeColors` 116 · `TreeGlyphs` 116 · `TreeLoader` 115 · `TreeNode` 115 · `TreePath` 115 · `TreeView` 114

U — `UnixBackend` 134

V — `VBox` 147 · `Validator` 102 · `Value` 82 · `VecTableModel` 113 · `View` 78 · `ViewId` 81 · `ViewState` 80

W — `Window` 93 · `WindowsBackend` 134

Модули констант

Модуль	Что в нём	Где описан
`cm`	команды	события 76
`kb`	клавиши	события 75
`sf`, `of`, `gf`, `dm`	флаги представлений	представления 81
`wf`	что можно делать с окном	окна 94
`bf`	флаги кнопки	кнопка 104
`mf`	окна сообщений	окна сообщений 110
`cf`, `cbx`, `il`	флажки, выпадающий список, поле для числа	элементы 104
`ef`, `ec`	поиск в редакторе	редактор 122
`fd`, `cd`	файловые диалоги	готовые диалоги 123
`color::palettes`	палитры	цвета 70
`mvc::CM_RECORD_*`	команды записей	данные 144

Все номера команд и константы — в главе 16.